台灣自編大學教科書

地圖建構與同步定位 (SLAM)

Simultaneous Localization and Mapping



大學教科書

第六篇

第20–24章

Visual SLAM 與視覺導航機器人

李世淵 (機器人叫獸)

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第六篇 Visual SLAM 與視覺導航機器人

第 20 章

Visual SLAM 入門

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

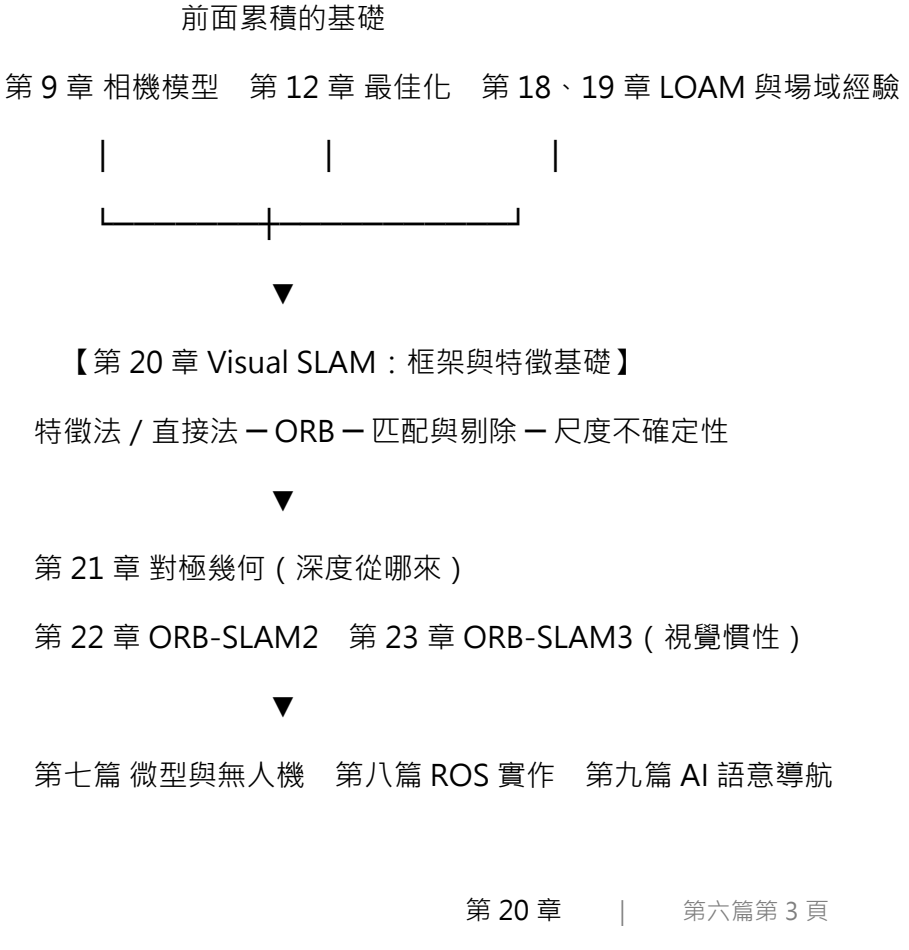
大學智慧機器人學程 / 高中 AI 機器人社團課程

本章在全書的位置

第六篇換了感測器，但沒有換思路。第五篇用雷射「量」出三維世界，並在第 19 章把那套技術送進了每天要跑一年的巡檢現場；本篇改用相機「看」。然而同學很快會發現，LOAM 的四個步驟——提取特徵、前端快速估計、後端精細最佳化、閉環修正——在視覺 SLAM 裡一步都沒少。差別只在於特徵的性質：光達的特徵是幾何的，一出生就帶著三維座標；相機的特徵是外觀的，只有二維像素位置，深度得另外想辦法。

這個差別造就了本篇的兩個主題。其一是「怎麼挑特徵、怎麼配對」，也就是本章 20.3 與 20.4 節的內容；其二是「沒有深度怎麼辦」，這是 20.5 節的尺度不確定性，也是第 21 章對極幾何要正面回答的問題。另外值得先放在心上的是：第 19.5 節量到的那件事——長走廊因為幾何退化而讓光達說不清自己走了多遠——恰恰是相機的主場，因為牆上的海報、門牌與地板紋理都是清楚的視覺特徵。在那之前，本章先把整個框架搭起來，讓同學知道後面四章各自在補哪一塊。

圖 20-0　第 20 章的連結地圖



學習目標

第五篇我們用雷射「量」出了三維世界；從本篇起，改用相機「看」。感測器換了，但同學會發現核心思想完全相通——特徵、配準、最佳化、閉環，只是從幾何點雲換成了影像像素。本章是第六篇的入口，建立視覺 SLAM 的整體框架與特徵技術基礎。學完本章之後，同學應能夠達成下列目標：

1. 說明視覺 SLAM 的基本流程，並指出各階段對應本書的哪些章節。
2. 比較特徵法與直接法兩大流派的原理、優缺點與適用場合。
3. 說明 ORB、SIFT、SURF、FAST、BRIEF 的角色與特性，並理解為何 ORB 成為 SLAM 的主流選擇。
4. 區分特徵追蹤與特徵匹配的差異，並說明常見的誤匹配剔除策略。
5. 解釋單目 SLAM 的尺度不確定性，以及三種解決途徑。
6. 比較雙目 SLAM 與 RGB-D SLAM 的差異與適用場景。
7. 舉例說明視覺 SLAM 在 AR / VR 與機器人中的應用。

核心概念

中文名詞	英文名詞	概念說明
視覺 SLAM	Visual SLAM	以相機為主要感測器的同步定位與建圖。
視覺里程計	Visual Odometry	由連續影像估計相機運動，會累積漂移。
特徵法	Feature-based / Indirect	提取特徵點並匹配，再由幾何關係求解運動。
直接法	Direct Method	不提特徵，直接以像素亮度差估計運動。
關鍵點	Keypoint	影像中顯著且可重複偵測的位置，如角點。
描述子	Descriptor	描述關鍵點周圍外觀的向量，用於比對。

中文名詞	英文名詞	概念說明
尺度不變性	Scale Invariance	物體遠近改變時仍能偵測到同一特徵的能力。
旋轉不變性	Rotation Invariance	影像旋轉後仍能正確匹配的能力。
漢明距離	Hamming Distance	兩個二元描述子相異位元的個數，比對極快。
特徵匹配	Feature Matching	在兩張影像間找出對應的同一特徵點。
特徵追蹤	Feature Tracking	在連續影像中持續追蹤同一特徵的位置。
尺度不確定性	Scale Ambiguity	單目相機無法確定地圖真實大小的先天限制。
詞袋模型	Bag of Words	把影像表示為視覺單詞的統計，用於地點辨識。

20.1 視覺 SLAM 的基本流程

第 9.6 節我們畫過一條知識階梯：影像處理 → 特徵提取 → 多視圖幾何 → 視覺里程計 → 視覺 SLAM。本節把這條階梯展開成完整的系統流程。

五個階段

1. 感測器資料讀取與預處理：讀取影像、去畸變（第 9.2 節的相機校正）。若是雙目或 RGB-D，還需做立體匹配或深度對齊。
2. 視覺里程計（前端）：估計相鄰影像之間的相機運動，並三角測量出局部的三維點。這一階段快速但會累積漂移（第 18 章的光達里程計是它的雷射版本）。
3. 後端最佳化：對一批位姿與地圖點做非線性最佳化（第 12 章的 Bundle Adjustment），消除累積誤差。
4. 閉環偵測：辨識「這個地方我來過」，產生閉環約束以修正全域漂移（第 12、15 章）。視覺 SLAM 常用詞袋模型來做這件事。
5. 建圖：依應用需求輸出地圖——稀疏的特徵點地圖、稠密的點雲、或語意地圖（第 1、3 章）。

圖 20-1 視覺 SLAM 的標準流程

影像輸入（單目 / 雙目 / RGB-D）

↓

① 預處理：去畸變、立體匹配（第 9 章）

↓

② 視覺里程計（前端）：特徵提取與匹配 → 估計相機運動（第 21 章）

↓

③ 後端最佳化：Bundle Adjustment 精修位姿與地圖點（第 12 章）

↓

④ 閉環偵測：詞袋模型辨識重訪地點 → 全域修正（第 22 章）

↓

⑤ 建圖輸出：稀疏點雲 / 稠密地圖 / 語意地圖（第 32 章）

與光達 SLAM 的對照

把這個流程與第 18 章的 LOAM 並排，會發現架構驚人地一致：兩者都是「特徵提取 → 前端快速估計 → 後端精細最佳化 → 閉環修正」。差別只在於特徵的性質：

- 光達的特徵是幾何的（邊緣、平面），直接帶有三維座標與距離。
- 視覺的特徵是外觀的（角點與其周圍紋理），只有二維像素座標，深度需要另外求得。

這個差別導致了視覺 SLAM 的核心難題：如何從「沒有深度的二維觀測」中，恢復出三維的相機運動與場景結構。這正是第 21 章對極幾何要回答的問題。

20.2 特徵法與直接法

視覺 SLAM 有兩條主要技術路線，理解它們的分歧，就理解了這個領域的地形。

特徵法（Feature-based，又稱間接法）

特徵法的流程是三個步驟：

6. 偵測：在影像中找出角點等關鍵點。
7. 描述：為每個關鍵點計算描述子，記錄它周圍長什麼樣。
8. 匹配：依描述子的相似度，在不同影像間找出對應的點。

特徵法在視覺里程計中被廣泛採用，因為它對相機運動、光線變化與動態物體具有穩健性。它之所以稱為「間接」法，是因為它不直接使用像素值，而是先把影像抽象成「一組特徵點」這個中間表示，再用幾何方法求解運動。

- 優點：對光線變化穩健（描述子多有一定的亮度不變性）、可處理較大的視角變化、易於做閉環偵測（用描述子做詞袋比對）、成熟穩定。
- 缺點：計算特徵與描述子耗時、丟棄了大部分像素資訊、在紋理稀少處（白牆）找不到特徵而失效（第 9 章的「白牆詛咒」）。

直接法（Direct Method）

直接法完全跳過特徵提取，直接使用像素亮度。它的核心假設是灰度不變性：同一個空間點在不同影像中的亮度應該相同。於是估計運動就變成：找一組相機運動，使得「把上一幀的像素依這個運動投影到當前幀後，亮度差最小」。

- 優點：不需計算特徵與描述子，速度可以很快；能利用影像中所有有梯度的像素（不只角點），因此在紋理稀少但有微弱梯度的場景仍可運作；能自然地產生半稠密或稠密地圖。
- 缺點：灰度不變性假設脆弱——光線變化、自動曝光、非朗伯表面都會破壞它；對大幅度運動收斂困難（需要好的初值）；較難做閉環偵測。

代表系統包括 LSD-SLAM 與 DSO（Direct Sparse Odometry）。

比較項目	特徵法（間接法）	直接法
使用的資訊	特徵點與描述子	像素亮度
核心假設	特徵可重複偵測與匹配	灰度不變性
對光線變化	較穩健	敏感
弱紋理場景	易失效	仍可利用微弱梯度
大幅運動	較能處理	易發散，需好初值
閉環偵測	容易（詞袋模型）	較困難
地圖密度	稀疏	半稠密或稠密
代表系統	ORB-SLAM 系列	LSD-SLAM、DSO

表 20-1　特徵法與直接法的比較

本書第六篇以特徵法為主線（第 21 至 23 章的 ORB-SLAM 系列），因為它成熟穩定、資源豐富、也最適合教學。但同學應知道直接法的存在——近年也出現結合兩者優點的混合方法。

20.3　ORB、SIFT、SURF、FAST、BRIEF

這五個名詞常一起出現，但它們的角色其實不同：有的只負責「偵測」、有的只負責「描述」、有的兩者都做。釐清這一點是本節的第一個重點。

偵測器與描述子：分工不同

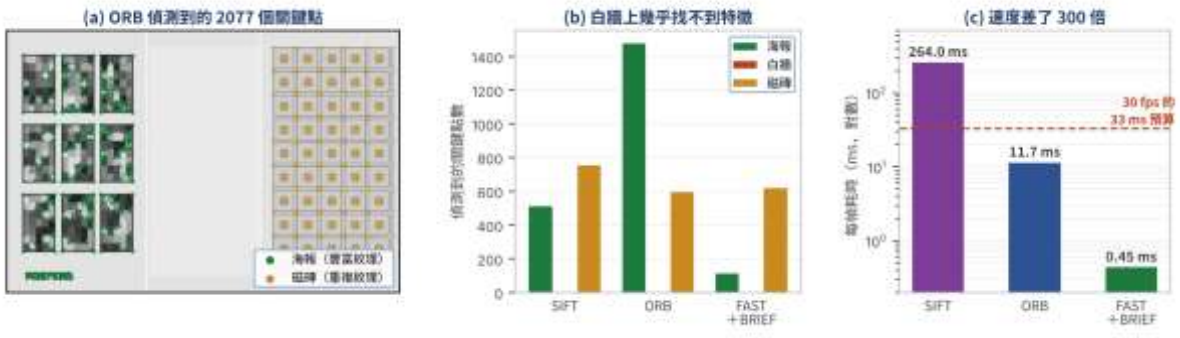
- FAST：只是偵測器。它極快地找出角點，但不產生描述子。
- BRIEF：只是描述子。它為給定的關鍵點產生二元描述向量，但不負責找點。
- SIFT、SURF、ORB：兩者兼具，既偵測也描述。

實務上這些方法可以獨立、分開地作為偵測器或描述子使用，不同組合會有不同的效能表現。這解釋了為何文獻中會出現「FAST-BRIEF」「SIFT-SURF」這類組合寫法。

五大演算法簡介

- SIFT（尺度不變特徵轉換）：經典中的經典。它以高斯差分找出不同尺度下的極值點，產生 128 維的描述子，具有優異的尺度與旋轉不變性。SIFT 本質上把特徵偵測與描述向量產生合併為單一的關鍵推導步驟。缺點是計算量大。
- SURF（加速穩健特徵）：SIFT 的加速版，以積分影像與盒式濾波器近似高斯運算。整體而言 SURF 的計算需求低於 SIFT。
- FAST（加速分段測試特徵）：它簡單而快速，把一個像素與其周圍 Bresenham 圓上的像素比較，先檢查該像素是否比圓上連續 N 個像素更亮或更暗；若是，它就可能是一個角點。演算法接著採用非極大值抑制來辨識並保留最具鑑別度的關鍵點。
- BRIEF（二元穩健獨立基本特徵）：以「隨機挑選點對比較亮度」產生二元字串描述子。優點是產生極快、比對只需算漢明距離（位元運算），極為高效。缺點是原始版本不具旋轉與尺度不變性。
- ORB（定向 FAST 與旋轉 BRIEF）：把 FAST 與 BRIEF 結合並各自改良——為 FAST 角點加上方向資訊（定向 FAST），並讓 BRIEF 描述子隨方向旋轉（旋轉 BRIEF），因而獲得旋轉不變性。

圖 20-2　同一面牆、三種特徵演算法：數量、分布與代價



為了讓比較有對照組，本書合成了一面教室牆，刻意分成三個區域：左側是九張各不相同的海報（豐富且獨特的紋理）、中間是一片幾乎無紋理的白牆、右側是整排一模一樣的磁磚（重複紋理）。這三種區域正好對應視覺 SLAM 會遇到的三種處境。

圖(a) 把 ORB 偵測到的 2,077 個關鍵點畫在影像上，結果一目了然：海報區密密麻麻，磁磚區也不少，而中間那片白牆上，一個點都沒有。圖(b) 把三種演算法的分區統計並列，白牆區的數字分別是 SIFT 1 個、ORB 0 個、FAST 0 個——這就是第 9 章「白牆詛咒」最直接的證據。注意這不是哪個演算法不夠好，而是沒有亮度變化的地方本來就沒有資訊可以提取，三種方法一視同仁地失效。

圖(c) 則是選擇 ORB 的理由。在同一張 960×640 的影像上，SIFT 每幀要 264 毫秒、ORB 11.7 毫秒、單純的 FAST 偵測只要 0.45 毫秒。SIFT 比 ORB 慢了二十餘倍，比 FAST 慢了近六百倍。紅色虛線標出的是 30 fps 系統每幀 33 毫秒的總預算——而這 33 毫秒裡還得塞進匹配、位姿求解與地圖更新。SIFT 單是提取特徵就超支八倍，ORB 則只用掉三分之一。案例 20-1 所說「速度的代價與回報」，講的就是這條紅線。

不過圖(b) 也藏著一個陷阱，值得同學先記下來：磁磚區的特徵數量其實很可觀，ORB 在那裡找到 598 個點、FAST 甚至找到 621 個。數量多不代表有用——下一節的圖 20-3 會揭曉，這些點幾乎全是災難的來源。

這張圖的實驗條件

影像為 960×640 灰階合成圖，ORB 上限設 2000 點，SIFT 與 FAST 用 OpenCV 預設參數，耗時為同一張影像重複五次的平均。

絕對數字會隨影像大小、參數與硬體而變，重要的是數量級的差距：SIFT 與 ORB 之間差一個數量級以上，這個結論相當穩健。

ORB 原論文報告它比 SURF 快約 14 倍、SURF 又比 SIFT 快約 24 倍（20.3 節），本實驗的 SIFT / ORB 比值與這個量級一致。

同學可在實作 20-1 第一部分用自己拍的教室照片重跑，並特別找一面白牆與一片磁磚地板來重現圖(b) 的三種處境。

為什麼 ORB 成為 SLAM 的主流

答案是速度。ORB 的開發者報告它比 SURF 快 14 倍，而 SURF 又比 SIFT 快 24 倍；這個較新的演算法在表現上也大致相當，某些情況下甚至好得多。另有文獻指出 ORB 的提取速度約比 SURF 快 15 倍，同時保持對旋轉與尺度變化的穩健性，這個效率使它特別適合即時應用。

在視覺 SLAM 的各種特徵提取演算法中，ORB 特徵的優勢在於提取與匹配快速，而相較 SIFT 與 SURF 並未損失太多精度。ORB 因此成為 SLAM 系統中主流的特徵點偵測方法——它以部分精度與穩健度換取運算速度的提升，這個取捨對即時系統極為划算。

ORB 描述子的巧思

值得一提 ORB 描述子的設計細節。在 ORB 中，BRIEF 描述子是 256 位元的向量，每個位元代表同一個 31×31 像素區塊中兩個 5×5 像素視窗平均強度比較的結果；一個區塊中有 676 個這樣的視窗，因此有超過二十萬種可能的成對比較。為了最大化描述子的鑑別力，ORB 從這二十萬種組合中，挑選出「離平均最遠且彼此相關性最低」的 256 個比較結果。

這個設計展現了一種重要的工程思維：不是隨機選，而是用資料驅動的方式挑出最有資訊量的比較。同學若日後設計自己的特徵或指標，這個「最大化鑑別力、最小化冗餘」的原則值得參考。

演算法	角色	描述子形式	速度	不變性
SIFT	偵測 + 描述	128 維浮點	最慢	尺度、旋轉
SURF	偵測 + 描述	64 / 128 維浮點	中	尺度、旋轉
FAST	僅偵測	無	極快	無（原始版）
BRIEF	僅描述	二元字串	極快	無（原始版）
ORB	偵測 + 描述	256 位元二元	快	旋轉，部分尺度

表 20-2 五種特徵演算法的比較

20.4 Feature Tracking 與 Feature Matching

找到特徵之後，下一步是在不同影像間建立對應關係。這裡有兩種思路，同學常混淆，本節釐清它們。

特徵匹配（Matching）：比對描述子

匹配的做法是：在兩張影像中各自獨立提取特徵與描述子，然後對第一張的每個特徵，在第二張中找描述子最相似的那一個。

- 相似度度量：浮點描述子（SIFT、SURF）用歐氏距離；二元描述子（ORB、BRIEF）用漢明距離——只需計算兩個位元串有幾位不同，用 XOR 加位元計數即可完成，速度極快。這是二元描述子的最大優勢。
- 適用場景：兩張影像差異較大時（視角改變、時間間隔長、閉環偵測），匹配是唯一可行的方法。

特徵追蹤（Tracking）：在鄰近位置找

追蹤的做法是：在第一張影像提取特徵後，不在第二張重新提取，而是假設特徵只移動了一小段距離，直接在原位置附近搜尋最相似的影像區塊。典型方法是光流（第 9.4 節）。

- 優點：不需計算描述子，速度更快；在連續影像中效果好。
- 限制：只適用於影像變化小的情況（高幀率、緩慢運動）；追蹤久了會累積漂移，特徵可能「跟丟」或漂到別的地方。

誤匹配：必須面對的現實

無論匹配或追蹤，都必然產生錯誤的對應。原因包括：重複紋理（磁磚、欄杆、樹葉，處處長得一樣）、視角變化太大、光線改變、動態物體。而在幾何求解中，少數幾個嚴重的誤匹配就可能把結果拉偏——這是視覺 SLAM 最常見的失敗原因之一。

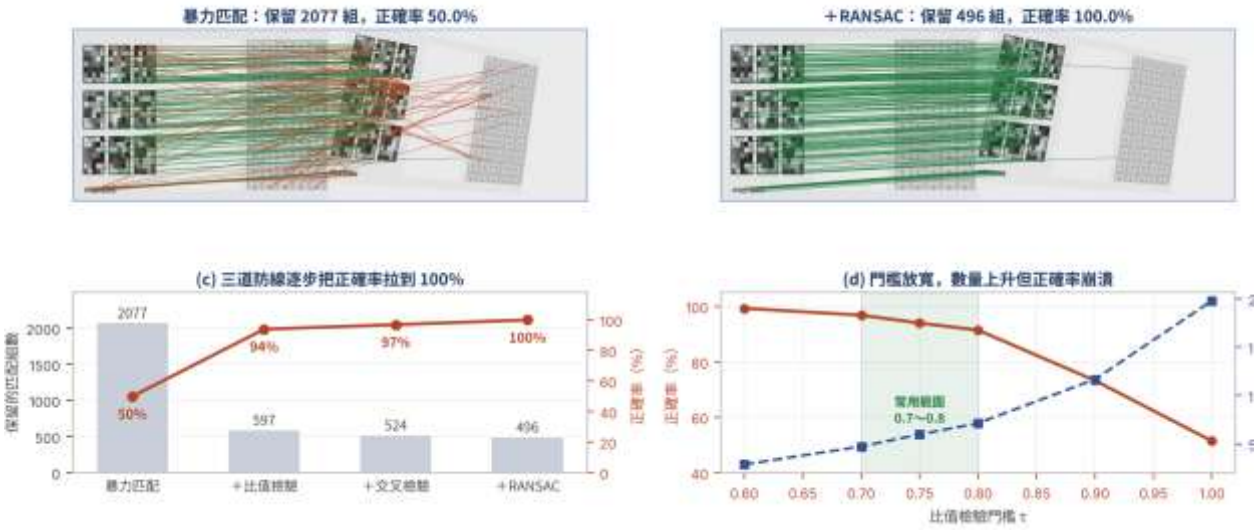
三種誤匹配剔除策略

9. 比值檢驗：對每個特徵，找出最相似與次相似的兩個候選。若兩者的距離很接近，表示「難以區分」，這個匹配不可靠，予以捨棄。這是 SIFT 論文提出的經典做法，簡單而有效。

10. 交叉檢驗：A 影像的特徵 a 匹配到 B 的特徵 b，同時 b 反過來也匹配到 a，才認可。單向的匹配視為不可靠。

11. 幾何驗證（RANSAC）：最重要的一招。利用「所有正確匹配必須符合同一個相機運動」這個幾何約束，反覆隨機取樣求解並統計符合者，把不符合的判為離群點剔除。實務上以 RANSAC 估計本質矩陣與內點集合，是從含有離群點的特徵對應中估計相機相對位姿的標準做法。這個方法將在第 21 章詳細展開。

圖 20-3　誤匹配的三道防線：從 50% 正確率到 100%



誤匹配剔除的效果通常只能靠眼睛看連線順不順，但如果第二張影像是用一個已知的單應矩陣從第一張變換出來的，那每一組匹配的對錯就有了標準答案——把第一張的特徵點用該矩陣投影過去，落點與匹配到的點相距四個像素以內就算正確。本圖正是這樣建立真值的。

圖(a) 是未經任何剔除的暴力匹配：2,077 組匹配，綠線是正確的、紅線是錯誤的，正確率恰好 50.0%——等於擲硬幣。注意紅線的分布並不均勻，大量紅線從右側磁磚區橫跨到左側海報區，呈現明顯的系統性錯亂。圖(b) 是走完三道防線之後：496 組匹配，正確率 100%，

連線整齊地呈現同一個方向的平移。

圖(c) 把過程拆開。比值檢驗一口氣把 2,077 組砍到 597 組，正確率從 50% 跳到 94%——這一刀最關鍵。交叉檢驗再修到 524 組、96.9%；最後 RANSAC 以幾何一致性把殘餘的外點清掉，剩下 496 組、100%。三道防線總共丟掉了 76% 的匹配，換來的是完全可信的 496 組對應。對後端最佳化而言，這筆交易非常划算：少量正確的約束遠勝大量摻雜錯誤的約束。

圖(d) 回答了討論問題第二題。比值門檻 τ 從 0.6 放寬到 1.0，保留的匹配數從 285 升到 1,981，但正確率從 99.3% 一路崩到 51.4%。曲線在 0.8 之後明顯轉折——這解釋了為什麼文獻上的建議值幾乎都落在 0.7 到 0.8 之間（圖中綠色區塊）。τ 的意義其實很直觀：它問的是「最佳候選有沒有明顯優於次佳候選」，而 $\tau = 1.0$ 等於完全不問，自然退化成暴力匹配。

磁磚區發生了什麼：案例 20-2 的量化版

分區來看，暴力匹配在海報區的正確率是 69.8%，在磁磚區只有 1.0%——598 組匹配裡只有 6 組是對的。重複紋理不是「比較難」，而是幾乎必然出錯。

比值檢驗在兩區的表現也截然不同：海報區從 1,479 組保留 587 組（約四成），磁磚區卻從 598 組只保留 10 組（不到 2%）。

這正是比值檢驗的設計意圖：在處處長得一樣的地方，最佳與次佳候選的距離必然接近，演算法會老實地說「我分不出來」而放棄，而不是硬選一個。

最終 RANSAC 之後，磁磚區只剩 2 組匹配。少，但是對的——這比留下 598 組有 99% 是錯的要安全得多。

20.5 Monocular SLAM 的尺度不確定性

這是單目視覺 SLAM 最根本、也最無法迴避的限制。第 9.1 節從針孔投影公式指出了它的幾何根源，本節說明它的後果與解法。

問題的本質

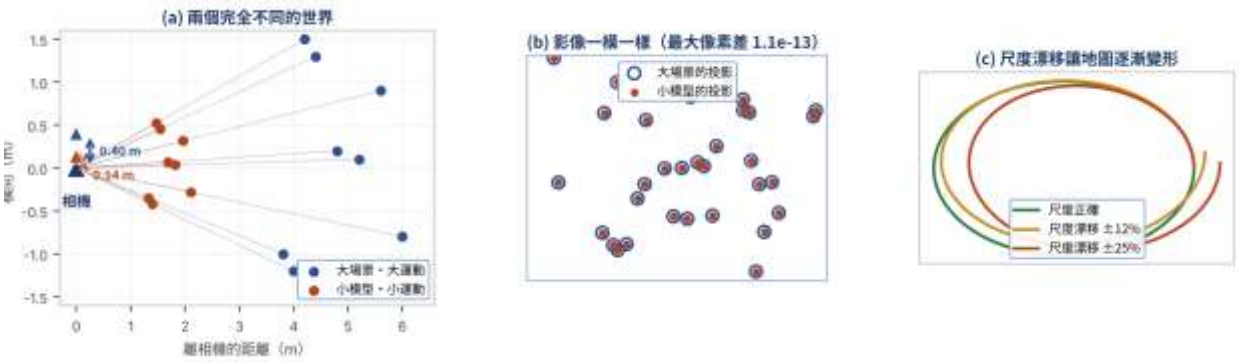
回顧第 9.1 節的投影公式：x = f·X/Z。注意這個式子中，若把場景中所有點的三維座標同時放大 k 倍，同時把相機的所有平移也放大 k 倍，投影到影像上的結果完全相同。

換句話說，單目相機拍到的一段影像序列，可能對應「真實房間中的真實運動」，也可能對應「一個模型屋中的微小運動」——兩者在影像上完全無法區分。單目 SLAM 因此只能恢復出「差一個未知比例因子」的軌跡與地圖。

後果

- 地圖沒有真實尺寸：建出的地圖可以說「這面牆到那面牆是 3.7 個單位」，但不知道一個單位是幾公尺。
- 無法直接用於導航：機器人不知道「前方障礙物到底在 0.5 公尺還是 5 公尺外」，自然無法安全避障。
- 尺度漂移：更麻煩的是，這個未知的比例因子在長時間運行中還會緩慢改變（稱為尺度漂移），使地圖逐漸變形。

圖 20-4　尺度不確定性：單目相機先天無法分辨的兩個世界



20.5 節說「把場景放大 k 倍、相機平移也放大 k 倍，影像完全相同」，這句話可以直接驗算。圖(a) 是側視圖：藍色是一個真實房間，點分布在離相機 3.6 到 6.2 公尺處，相機平移 0.40 公尺；紅色則是把整個場景與平移同時縮小到 0.35 倍——變成一個擺在眼前的模型屋，相機只移動 0.14 公尺。兩者在物理上完全是不同的世界。

圖(b) 把兩組三維點用同一組相機內參投影成影像。藍色空心圈是大場景的投影、紅色實心點是小模型的投影——它們不是「很接近」，而是完全重合。本書實際計算了兩組投影的最大像素差：1.1×10^{-13}，也就是浮點運算的捨入誤差，在數學上就是零。這不是演算法不夠聰明，而是投影方程式本身決定的：單目影像裡根本沒有攜帶尺度資訊。

圖(c) 則展示這個不確定性在實務上的具體樣貌——尺度漂移。綠線是尺度正確時的真實軌跡，一個閉合的橢圓。若每一段位移的尺度因子以隨機漫步方式緩慢偏移 ±12%（黃線）或 ±25%（紅線），軌跡就逐漸變形、首尾無法接合。請注意：這裡的每一步方向都是正確的，錯的只有「每一步走了多遠」，但累積下來足以讓整張地圖失真。

這就是為什麼單目 SLAM 幾乎不會單獨使用。20.5 節的三種解法——加 IMU、改用雙目或 RGB-D、或引入已知尺寸的參考物——本質上都是從外部補進一個「公尺」的定義。值得一提的是，這個限制至今沒有被打破：即使是 20.8 節將介紹的、以基礎模型為核心的最新單目 SLAM 系統，仍然要在 Sim(3) 上最佳化、並在評測時先做尺度對齊——因為它面對的是同一個幾何事實。

尺度與「深度估計」的差別

深度學習模型（如 Depth Anything 類）能從單張影像「猜」出深度，但那是從訓練資料學到的先驗——看到門框就推測約 2 公尺高，而不是量出來的。

這種先驗在一般場景好用，在尺寸異常的場景會系統性出錯：模型屋、放大的道具、空拍的俯視畫面，都是典型的失敗案例。

相對地，IMU 的加速度計量到的是帶物理單位的加速度，雙目的基線是用尺量過的固定距離——這兩者提供的是量測，不是猜測。

實務建議：需要安全保證的場合（避障、抓取、導航）用量測；需要視覺效果的場合（AR 貼圖、稠密重建）可接受先驗。

三種解決途徑

12. 引入 IMU（視覺慣性）：加速度計量測的是有物理單位的加速度，積分後即可提供真實尺度。這是最通用的解法，也是第 7 章與第 23 章的核心主題。代價是需要足夠的運動激勵才能觀測到尺度（第 7.7 節）。
13. 使用雙目或 RGB-D 相機：雙目的基線是已知的固定距離，RGB-D 直接輸出以公尺為單位的深度。硬體上直接解決問題，是最省事的做法（20.6 節）。
14. 已知尺寸的參考物：若場景中有已知大小的物體（如第 4 章的 AprilTag、棋盤格），或已知相機離地高度，就能反推尺度。這在特定應用中很實用。

值得補充的是，近年也有以深度學習從單張影像估計深度（第 9.4 節的 Depth Anything 類模型）來提供尺度先驗的做法，但其精度與可靠度仍不及前三種物理方法。

20.6 Stereo SLAM 與 RGB-D SLAM

第 9.3 節介紹過三種相機取得深度的方式，本節聚焦它們在 SLAM 系統中的差異。

雙目 SLAM

雙目相機以已知基線的兩個鏡頭同時拍攝，由左右影像的視差直接算出深度（第 9.6 節的公式 $Z = f \cdot b/d$）。

- 優點：單幀即可得深度、有絕對尺度、室內外皆可運作（不依賴主動投射）、量測範圍可透過加大基線延伸。
- 限制：需精確校正兩相機的相對位置（外參）、在無紋理表面找不到對應而無深度、深度誤差隨距離平方成長（第 9.6 節）、運算量較大（需做立體匹配）。
- 適用：戶外機器人、無人機、自駕車——凡是需要在陽光下運作、且距離範圍較大的場景。

RGB-D SLAM

RGB-D 相機主動投射紅外線圖案取得稠密深度（第 9.3 節的結構光或主動紅外雙目）。

- 優點：直接輸出每個像素的深度，不需自己做立體匹配；在無紋理的白牆上也有深度（因為是主動投射）；有絕對尺度；能輕易建出稠密的三維地圖。
- 限制：多依賴紅外線，強烈日光下失效；有效範圍通常在 0.2 至 10 公尺；對玻璃鏡面同樣束手無策（與第 8 章光達的弱點相同）；功耗較高。
- 適用：室內服務機器人、桌面級三維重建、室內測繪——這也是第 30 章實作選擇 RGB-D 的原因。

比較項目	單目 SLAM	雙目 SLAM	RGB-D SLAM
絕對尺度	無（需外部提供）	有	有
深度來源	運動三角測量	左右視差	主動投射
無紋理表面	失效	無深度	仍有深度
戶外強光	可用	可用	多半失效
有效距離	不限（但無尺度）	較遠	約 0.2–10 m
地圖密度	稀疏	可半稠密	稠密
硬體成本	最低	中	中

表 20-3　三種視覺 SLAM 配置的比較

ORB-SLAM2 與 ORB-SLAM3（第 22、23 章）的一大優點，就是同時支援這三種模式——同樣一套系統，換個輸入模式即可。這讓同學能在同一個框架下比較三者的差異，是絕佳的教學素材。

20.7　視覺 SLAM 在 AR / VR 與機器人中的應用

第 1 章 1.4 節提過 AR / VR 是 SLAM 四大應用之一，本節深入視覺 SLAM 在其中的角色，並延伸到機器人應用。

AR / VR：口袋裡的視覺 SLAM

AR 眼鏡或手機 AR 要把虛擬物件穩定地「釘」在真實桌面上，前提是裝置必須即時知道自己在空間中的六自由度位姿。業界慣稱這個能力為「空間定位」或「由內而外追蹤」（Inside-Out Tracking），而它的技術本質就是視覺慣性 SLAM。

- 為什麼是視覺而非光達：AR 裝置對重量、功耗、成本極度敏感，相機加 IMU 是唯一可行的組合（第 9.5 節的視覺優勢）。
- 特殊需求：延遲必須極低（否則虛擬物件會「漂移」造成不適）、且必須在使用者快速轉頭時仍維持追蹤——這對視覺 SLAM 是嚴苛考驗，也是 IMU 不可或缺的原因（第 7.6 節）。
- VR 頭盔的演進：早期 VR 需要外部基地台定位，現代頭盔改用機身相機做視覺 SLAM，使用者因此能自由走動——這是 SLAM 直接改變產品形態的例子。

機器人應用

- 室內服務機器人：視覺 SLAM 提供的不只是幾何，還有語意（第 9.7 節）——機器人能辨識「那是門、那是電梯」，這是光達給不了的（第 32 章的主題）。
- 無人機：載重與電力極度受限，相機加 IMU 的組合是最佳選擇（第 26 章的主題）。
- 低速自駕與園區載具：視覺 SLAM 在低速自駕情境中的應用是活躍的研究方向，特徵法因其對相機運動、光線變化與動態物體的穩健性而被廣泛採用。
- 地點辨識與重定位：視覺特徵天生適合做「這裡我來過嗎」的判斷。詞袋技術是最普遍的做法之一，把局部特徵描述子當作視覺單詞使用；而 DBoW2 引入高效的 FAST 偵測器搭配 BRIEF 描述子，以二元單詞袋的形式，相較傳統使用 SURF 與 SIFT 的做法大幅減少特徵提取所需時間，系統更有效率也更穩健。這正是 ORB-SLAM 閉環偵測的技術基礎（第 22 章）。

趨勢：深度學習的加入

傳統手工設計的特徵方法有一個問題：它們無法有效適應各種複雜且無法預期的情況；因此許多研究者採用卷積神經網路來提取影像的深層特徵，應用於視覺里程計、位姿估計與

閉環偵測等各個階段。

這是視覺 SLAM 目前最活躍的方向之一：用學習來的特徵取代手工特徵、用神經網路做地點辨識、乃至於端到端的位姿估計。不過在實務系統中，傳統幾何方法因其可解釋性與穩定性，目前仍是主流；學習方法多作為輔助或用於特定困難場景。第 32 章會回到這個主題，討論 AI 如何與 SLAM 結合。

至此，同學已建立視覺 SLAM 的整體框架。下一章我們深入前端的數學核心——對極幾何：兩張影像之間到底藏著什麼幾何關係？本質矩陣與基礎矩陣是什麼？PnP 問題如何求解？三角測量如何恢復三維點？這些是把「二維像素對應」變成「三維相機運動」的關鍵橋樑。

20.8 特徵之後：2024–2026 年視覺前端的三個轉向

本章介紹的 SIFT、ORB 都是手工設計的特徵，而案例 20-3 已經預告了深度學習帶來的轉變。這個轉變在近兩年明顯加速，而且變化的層次比「換一種特徵」更深。本節整理三個轉向。

一、學習式特徵終於跑得動了

學習式局部特徵（如 SuperPoint）準確度早就超越 ORB，但長年卡在同一個問題上：太慢，而且需要 GPU。這正是圖 20-2(c) 那條 33 毫秒紅線所設下的門檻。2024 年的 XFeat 直接針對這一點而設計——它主打在「一台便宜的筆電 CPU 上即時運行」，不需要專門的硬體加速，而速度比既有的輕量深度特徵最多快五倍，準確度則與 SuperPoint 這類大得多的模型相當。

在匹配端也有類似的演進。LightGlue 一類的方法讓深度匹配器能依影像的難易度自適應地提前結束運算，容易的影像對就少算幾層。兩者合起來的意義是：圖 20-3 那三道防線（比值檢驗、交叉檢驗、RANSAC）正在被一個學會判斷「這組對應可不可信」的網路取代。不過要強調的是，RANSAC 代表的幾何一致性驗證仍然沒有消失，它只是從「主要防線」退

為「最後確認」。

二、從「兩張影像之間」到「直接吐出三維」

第二個轉向更根本。傳統視覺 SLAM 的前端是一條長鏈：提特徵 → 匹配 → 剔除 → 解對極幾何 → 三角測量。而 DUSt3R 與 MASt3R 這一系列工作提出：乾脆讓網路直接從兩張影像吐出每個像素的三維座標，把中間那一整串步驟一次跳過。

建構於其上的 MASt3R-SLAM（CVPR 2025）就是一套完整的單目稠密 SLAM：它以稠密的點圖匹配取代 KD 樹或暴力搜尋，甚至不需要事先知道相機內參。在 TUM RGB-D 資料集上，已校正情況下平均絕對軌跡誤差 0.030 公尺，未校正時 0.060 公尺；在一般桌機搭配高階顯示卡上約可達 15 fps。

但這裡有一個同學一定要看清楚的細節，而且它直接呼應圖 20-4：MASt3R-SLAM 並沒有解決尺度不確定性。該論文明言尺度是預測之間不一致的主要來源，因此在 Sim(3) 上最佳化位姿，單目的評測結果也是在做過尺度對齊之後才報告的。換句話說，就算把整個前端換成基礎模型，單目相機看不出絕對尺度這件事依然成立——因為那是幾何的性質，不是演算法的缺陷。

三、地圖表示：從稀疏點雲到可算繪的場景

第三個轉向與第 17.8 節提過的三維高斯潑濺是同一股潮流，只是在視覺領域更為自然。傳統特徵法 SLAM 建出的是稀疏的特徵點地圖——夠用來定位，但看起來只是一團散點，既無法用於渲染，也不利於後續的語意理解。近兩年的稠密視覺 SLAM 則直接把場景表示成可算繪的形式，讓「定位」與「重建逼真場景」變成同一件事的兩面。

這對第 20.7 節談的 AR / VR 應用意義特別大：AR 需要的從來不只是相機位姿，還需要知道虛擬物件會被哪些真實表面遮擋。稀疏點雲給不了這個資訊，稠密可算繪的表示則天生具備。不過代價也很明確——記憶體與運算量都遠高於稀疏地圖，這也是為什麼機器人導航目前仍以稀疏或半稠密為主。

三個轉向的共同線索

前端元件（XFeat / LightGlue）：學習式特徵終於跨過即時性門檻，開始取代 ORB。

前端結構（MASt3R-SLAM）：整條「特徵—匹配—幾何」的鏈條被端到端網路一次跨過。

地圖表示（稠密可算繪）：地圖從定位用的稀疏點，變成能看、能遮擋、能理解的場景。

沒有改變的事：尺度不確定性仍在（圖 20-4）、幾何一致性驗證仍需要、而白牆上依然沒有資訊可以提取。

共同線索：被取代的是「怎麼算」，不是「算什麼」。本章的四個基本問題——哪裡有特徵、哪些對應可信、深度從哪來、尺度誰給——依然是評斷任何新系統的標準。

承先啟後

本章建立了視覺 SLAM 的整體框架與特徵基礎。圖 20-2 說明了哪裡找得到特徵、以及為什麼是 ORB；圖 20-3 說明了匹配必然出錯、以及三道防線如何把正確率從 50% 救回 100%；圖 20-4 則說明了單目相機先天的極限。這三件事也正是第 22、23 章 ORB-SLAM 系列每一個設計決策的背景。

但我們還沒回答最核心的問題：有了一組可信的二維對應，究竟如何算出相機移動了多少、以及那些特徵點在三維空間的哪裡？下一章進入對極幾何——本質矩陣、基礎矩陣、單應矩陣與三角測量，這是整個視覺 SLAM 的數學心臟，也是圖 20-3 裡 RANSAC 所驗證的那個「幾何約束」的真面目。

公式或流程

一、漢明距離：二元描述子的比對

兩個二元描述子 d_i、d_j 的相似度以漢明距離衡量——即相異位元的個數：

$$Hamming(d_i, d_j) = popcount(d_i \ XOR \ d_j)$$

XOR 後計算 1 的個數（popcount）。對 256 位元的 ORB 描述子而言，這只需數個位元運算指令，遠快於浮點描述子的歐氏距離計算。這正是 ORB 能達到即時效能的關鍵之一。距離越小表示越相似。

二、比值檢驗

對某個特徵，設它與最相似候選的距離為 d_1、與次相似候選的距離為 d_2，則接受此匹配的條件為：

$$d_1 \ / \ d_2 < \tau \quad (\tau \text{ 常取 } 0.7 \text{ 至 } 0.8)$$

直觀理解：若最佳與次佳的差距不夠明顯（比值接近 1），表示這個特徵在影像中有多個相似的候選，難以區分，應予捨棄。這條簡單的判準能剔除大量誤匹配。

三、直接法的光度誤差

直接法不比對描述子，而是最小化光度誤差——把上一幀的像素依相機運動 T 投影到當前幀後的亮度差：

$$T^* = argmin \ \Sigma_i \ || \ I_2(\pi(T \cdot P_i)) \ - \ I_1(u_i) \ ||^2$$

其中 I_1、I_2 是兩幀影像的亮度、u_i 是第一幀中的像素、P_i 是它對應的三維點、$\pi(\cdot)$ 是第 9 章的投影函數。與特徵法比較：特徵法最小化的是「重投影誤差」（幾何距離，以像素為單位），直接法最小化的是「光度誤差」（亮度差）。兩者都是最小平方問題（第 12 章），只是誤差的定義不同。

四、視覺 SLAM 前端處理流程

圖 20-5 特徵法前端的每幀處理流程

① 讀取影像 → 去畸變（第 9.2 節）

↓

② 偵測關鍵點：以 FAST 找角點並做非極大值抑制

↓

③ 計算方向與描述子：定向 FAST + 旋轉 BRIEF → 256 位元描述子

↓

④ 與上一幀（或地圖）匹配：漢明距離比對

↓

⑤ 誤匹配剔除：比值檢驗 → 交叉檢驗 → RANSAC 幾何驗證

↓

⑥ 位姿求解：由通過驗證的匹配估計相機運動（第 21 章）

↓

⑦ 三角測量新的三維點 → 交給後端最佳化（第 12、22 章）

案例

案例 20-1 速度的代價與回報：ORB 為何贏得 SLAM 戰場

如果單看特徵的品質，SIFT 幾乎在各項指標上都優於 ORB——它的尺度不變性更完整、匹配更準確、對視角變化更耐受。那為什麼絕大多數即時 SLAM 系統選擇 ORB?

答案是即時性的殘酷要求。一個以 30 幀每秒運行的 SLAM 系統，每一幀只有約 33 毫秒的預算，而這段時間還得完成特徵提取、匹配、位姿求解、地圖更新等所有工作。ORB 開發者報告它比 SURF 快 14 倍，而 SURF 又比 SIFT 快 24 倍——換算下來，ORB 大約比

SIFT 快兩個數量級。這個差距不是「稍微快一點」，而是「能不能即時運行」的分野。

更關鍵的是，ORB 犧牲的精度在 SLAM 系統中可以被彌補。SLAM 有後端最佳化與閉環修正（第 12 章），即使單次匹配略遜一籌，大量的冗餘觀測與全域最佳化能把誤差壓下來。換句話說：系統層級的設計，補償了元件層級的不足。

這個案例給同學一個重要的工程觀：選擇技術元件時，不能只看該元件的孤立指標，而要看它在整個系統中的貢獻與代價。這與第 8 章「感測器選型是綜合判斷」、第 15 章「設計取向由應用需求決定」一脈相承。

案例 20-2　重複紋理的陷阱：磁磚地板與欄杆

一個在大學課堂上容易重現的失敗實驗：讓機器人在鋪滿相同磁磚的地板上，或沿著整排相同的欄杆移動，執行視覺 SLAM。結果往往是位姿估計突然跳錯，地圖出現嚴重錯位。

原因是重複紋理造成的系統性誤匹配。每一塊磁磚的角點看起來都一模一樣，描述子幾乎相同。演算法在比對時，很可能把「這一塊磁磚的角」匹配到「隔壁那一塊磁磚的角」——而且它「看起來」是一個完美的匹配，描述子距離極小。

這正是比值檢驗存在的意義：當最佳與次佳候選的距離非常接近時（磁磚場景必然如此），比值檢驗會判定「無法區分」而捨棄這個匹配，寧可少一些匹配，也不要錯誤的匹配。若比值門檻設得太寬鬆，這類場景就會出問題。

對照第 13.6 節的長走廊、第 17.3 節的結構退化、第 9 章的白牆詛咒，同學應該看出一個貫穿全書的模式：環境的重複性與特徵匱乏，是 SLAM 的共同天敵，不分光達或視覺。理解這一點，你就能在專題規劃階段預判哪些場域會出問題。

案例 20-3　從手工特徵到深度學習：一個正在發生的轉變

本章介紹的 SIFT、SURF、ORB 都是「手工設計」的特徵——人類研究者根據對影像的理解，設計出偵測與描述的規則。這些方法主導了電腦視覺與 SLAM 二十餘年。

但它們有一個共同弱點：無法有效適應各種複雜且無法預期的情況；因此許多研究者轉而採用卷積神經網路來提取影像的深層特徵，應用於視覺里程計、位姿估計與閉環偵測等階段。學習來的特徵能適應手工規則難以描述的情境——季節變化、日夜差異、天氣影響。

不過同學要注意：這個轉變尚未完成，而且未必會全面取代。傳統幾何方法有幾個學習方法目前難以匹敵的優點：可解釋（出錯時知道為什麼）、不需訓練資料、在未見過的場景中行為可預期、運算資源需求明確。目前主流實務系統仍以幾何方法為骨幹，學習方法作為特定環節的增強。

對本書讀者的建議是：先把傳統方法學紮實。理解 ORB 為何這樣設計、RANSAC 為何有效、Bundle Adjustment 在解什麼——這些原理是不會過時的思考框架。有了這個基礎，再去看深度學習的方法，你會知道它們在取代什麼、又保留了什麼。第 32 章會回到這個主題。

實作活動

實作 20-1　特徵提取、匹配與誤匹配剔除

目的：親手實作視覺 SLAM 前端的完整流程，比較不同特徵演算法的效能，並見證誤匹配剔除的威力。本實作以 Python 搭配 OpenCV，只需一般相機或現成影像即可完成，是第 21 至 23 章的直接暖身。

第一部分：特徵演算法比較（全體）

1. 準備兩張同一場景但視角略有不同的照片（可用手機拍攝教室，前後移動一小步）。
2. 分別用 SIFT、ORB、FAST 偵測關鍵點，把結果畫在影像上並排比較：各偵測到幾個點？分布在哪裡？
3. 計時比較：記錄三種方法在同一張影像上的提取時間，驗證本章提到的速度差異（ORB 遠快於 SIFT）。

4. 對一張大面積白牆的照片重複實驗，觀察三種方法都幾乎偵測不到特徵——再次驗證第 9 章的「白牆詛咒」。
5. 旋轉測試：把第二張影像旋轉 45 度，比較 ORB 與原始 BRIEF 的匹配成功率，驗證「旋轉不變性」的價值。

第二部分：匹配與誤匹配剔除（全體）

6. 用 ORB 對兩張影像做暴力匹配（以漢明距離），畫出所有匹配連線，目視觀察有多少條線明顯是錯的。
7. 套用比值檢驗（門檻 0.75），重新畫出匹配結果，統計剔除了多少匹配、剩下的品質如何。
8. 加入交叉檢驗，再次觀察改善程度。
9. 用 RANSAC 估計兩張影像之間的幾何關係（可先用單應矩陣），把內點與外點以不同顏色顯示，觀察 RANSAC 剔除了哪些匹配。
10. 比值門檻實驗：把門檻設為 0.6、0.75、0.9，比較保留的匹配數量與正確率，找出你的場景最合適的值。

第三部分：重複紋理與追蹤（大學部）

11. 重複紋理實驗：拍攝磁磚地板、欄杆或格子狀的窗戶，執行匹配並觀察誤匹配現象，重現案例 20-2。比較比值檢驗在此類場景的重要性。
12. 光流追蹤：錄製一段緩慢移動的影片，用 OpenCV 的 Lucas-Kanade 光流追蹤特徵點，把軌跡畫出來，觀察哪些點被穩定追蹤、哪些跟丟。
13. 比較追蹤與匹配：對同一段影片，分別用「每幀重新提取並匹配」與「光流追蹤」，比較兩者的耗時與穩定度，對照 20.4 節的討論。

討論問題

1. ORB 與 SIFT 的提取時間相差多少倍？與本章引述的數據是否相符？若你的 SLAM 系統要跑 30 fps，哪一種可行？

2. 比值檢驗剔除了百分之幾的匹配？剩下的匹配品質有明顯提升嗎？若把門檻放寬到 0.9，會發生什麼？
3. 在重複紋理場景中，誤匹配呈現什麼模式（是隨機的，還是系統性地偏向某個方向）？為什麼系統性誤匹配比隨機誤匹配更危險？
4. 光流追蹤在什麼情況下跟丟？這對「相機移動速度」與「幀率」的關係有什麼啟示？
5. （大學部）依表 20-3，若要為大學的室內導覽機器人選視覺配置，你會選單目 + IMU、雙目還是 RGB-D？說明理由。

延伸挑戰（大學部）

其一：實作一個最簡單的視覺里程計——連續讀取影片幀，用 ORB 匹配相鄰幀、以 RANSAC 剔除誤匹配、估計相對運動並累加，把軌跡畫出來。觀察漂移如何隨時間累積（這正是第 21 至 22 章要解決的問題）。其二：實作一個簡易的地點辨識——把一段影片的所有幀提取 ORB 描述子，建立簡單的詞袋表示，測試能否正確找出「同一地點的不同時刻影像」。這是第 22 章閉環偵測的雛形。

延伸閱讀

1. E. Rublee et al., "ORB: An Efficient Alternative to SIFT or SURF," ICCV, 2011.（ORB 原始論文，本章 20.3 節主要依據）
2. D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," IJCV, 2004.（SIFT 經典論文，含比值檢驗的提出）
3. E. Rosten and T. Drummond, "Machine Learning for High-Speed Corner Detection," ECCV, 2006.（FAST 偵測器原始論文）
4. M. Calonder et al., "BRIEF: Binary Robust Independent Elementary Features," ECCV, 2010.（二元描述子的開創性工作）

5. D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," IEEE T-RO, 2012.（DBoW2,ORB-SLAM 閉環偵測的基礎）
6. 高翔、張濤，《視覺 SLAM 十四講》第 7 講（視覺里程計：特徵點法）、第 8 講（直接法）.（華文世界最清楚的特徵法與直接法教材，含完整程式碼）
7. OpenCV 官方文件之 Feature Detection and Description 教學（docs.opencv.org）.（本章實作的主要工具，附完整範例）

小結

1. 視覺 SLAM 的標準流程為：預處理去畸變 → 視覺里程計（前端） → 後端最佳化 → 閉環偵測 → 建圖；其架構與第 18 章的光達 SLAM 高度一致，差別在於視覺特徵只有二維像素座標，深度需另外求得。
2. 特徵法先提取關鍵點與描述子再以幾何求解，對光線變化與動態物體穩健但在弱紋理處失效；直接法以像素亮度最小化光度誤差，能利用微弱梯度並產生稠密地圖，但依賴脆弱的灰度不變性假設。
3. FAST 只偵測、BRIEF 只描述、SIFT / SURF / ORB 兩者兼具；ORB 結合定向 FAST 與旋轉 BRIEF，以 256 位元二元描述子與漢明距離達成極高效率——比 SURF 快約 14 倍、SURF 又比 SIFT 快約 24 倍，因而成為 SLAM 的主流選擇。
4. 匹配比對描述子，適用於視角差異大與閉環偵測；追蹤在鄰近位置搜尋，適用於連續影像且更快。誤匹配必然發生，三種剔除策略為比值檢驗、交叉檢驗與 RANSAC 幾何驗證。
5. 單目 SLAM 有先天的尺度不確定性：場景與運動同時放大 k 倍，影像完全相同；三種解法為引入 IMU、使用雙目或 RGB-D、或利用已知尺寸的參考物。

6. 雙目以已知基線的視差測深，有絕對尺度且室內外皆可用，但怕無紋理且誤差隨距離平方成長；RGB-D 主動投射取得稠密深度，在白牆上仍有效，但怕強光與玻璃、範圍受限。
7. 視覺 SLAM 是 AR / VR 由內而外追蹤的技術本質，也廣泛用於服務機器人、無人機與低速自駕；詞袋模型（如 DBoW2 以二元單詞袋）是地點辨識與閉環偵測的關鍵技術，而深度學習特徵正逐步進入這個領域。

本章建立了視覺 SLAM 的整體框架與特徵基礎。但我們還沒回答最核心的問題：有了兩張影像之間的特徵對應，究竟如何算出相機移動了多少？下一章「對極幾何與相機運動估測」將補上這塊拼圖——本質矩陣與基礎矩陣描述了兩視圖之間的幾何約束、PnP 問題處理三維點到二維影像的對應、三角測量恢復三維座標、RANSAC 則讓這一切在有誤匹配的現實中依然可行。這是視覺 SLAM 最數學、也最優美的一章。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 21 章

對極幾何與相機運動估測

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

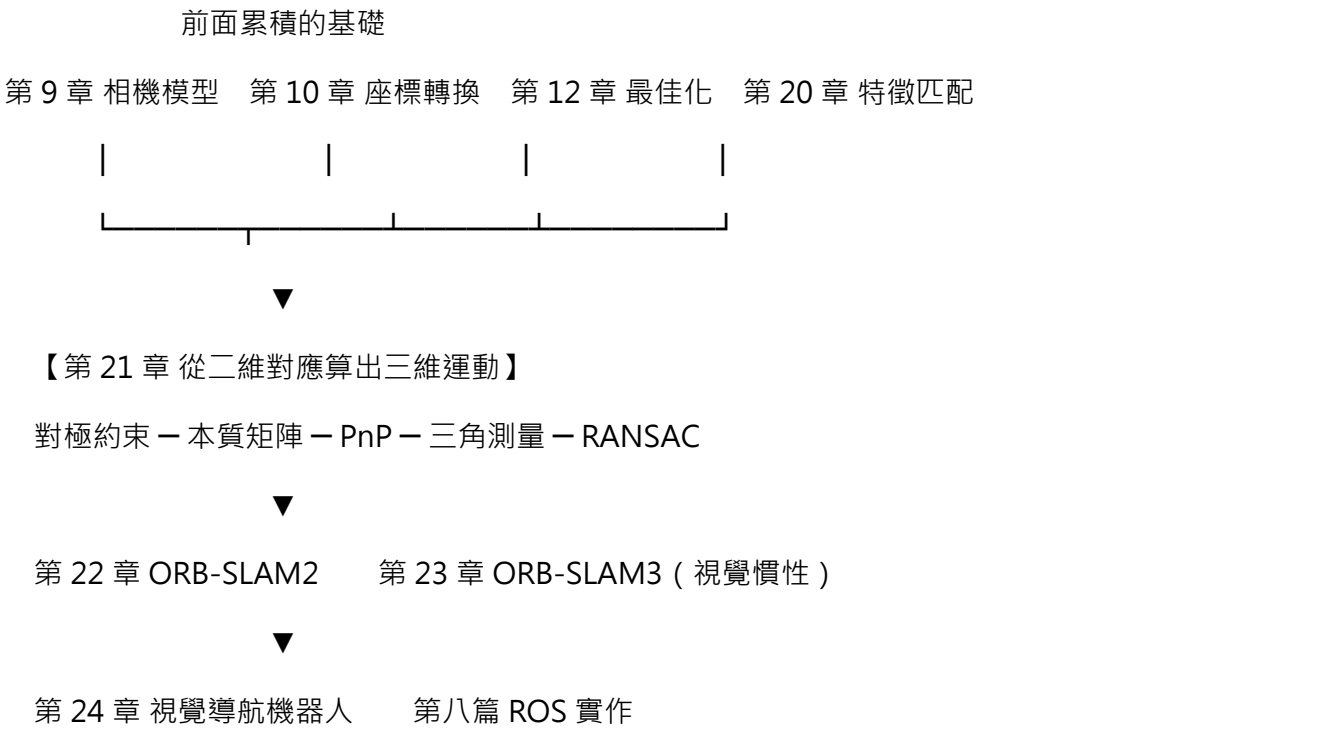
本章在全書的位置

第 20 章教會了機器人「在兩張影像之間找到同一個東西」。本章回答下一個問題：找到了之後，怎麼算出相機移動了多少。

這是全書數學密度最高的一章，但它的回報也最直接——本章的三張實驗圖會證明一件事：對極幾何不是近似、不是經驗法則，而是一組在無雜訊時誤差為零的精確關係。圖 21-2 會示範，給它乾淨的對應點，八點法恢復出的旋轉與平移方向誤差是 0.000 度；四組數學解當中，正深度檢驗會以 223 比 0 的票數挑出唯一正確的那一組。

但同樣重要的是它的脆弱。圖 21-3 會量出雜訊怎麼放大成位姿誤差，以及為什麼基線短的時候深度完全不能信；圖 21-4 則會顯示，只要有一成的錯誤匹配，這套優美的數學就會整個垮掉——這正是 RANSAC 存在的理由，也是第 22 章 ORB-SLAM2 每一個穩健性設計的動機。

圖 21-0　第 21 章的連結地圖



學習目標

上一章我們學會了如何在兩張影像間找出對應的特徵點，但還沒回答最核心的問題：有了這些對應，究竟如何算出相機移動了多少？本章補上這塊拼圖。這是視覺 SLAM 最數學、也最優美的一章——一旦理解對極幾何，你會發現兩張看似平凡的照片裡，竟藏著如此嚴謹的三維約束。學完本章之後，同學應能夠達成下列目標：

8. 說明從兩張影像推估相機運動的整體思路與適用條件。
9. 說明對極幾何的核心元素：基線、對極點、對極線與對極約束。
10. 區分本質矩陣與基礎矩陣的差異，並說明如何由它們恢復相機運動。
11. 說明 PnP 問題的定義、求解方法與它在 SLAM 中的角色。
12. 說明三角測量的原理與其精度受什麼因素影響。
13. 說明 RANSAC 的運作流程，並理解它為何能對抗大量誤匹配。
14. 把影像處理課程的知識銜接到視覺 SLAM 的幾何求解。

核心概念

中文名詞	英文名詞	概念說明
對極幾何	Epipolar Geometry	描述同一場景兩個視角之間內在幾何關係的理論。
基線	Baseline	連接兩個相機光心的直線。
對極點	Epipole	基線與各影像平面的交點。
對極線	Epipolar Line	一影像中的點，在另一影像中對應的一條直線。
對極約束	Epipolar Constraint	對應點必落在對極線上的幾何限制。
本質矩陣	Essential Matrix, E	已知內參時描述兩視圖關係的 3×3 矩陣。
基礎矩陣	Fundamental Matrix, F	未知內參時描述兩視圖關係的 3×3 矩陣。
單應矩陣	Homography, H	描述平面場景或純旋轉下兩視圖關係的矩

中文名詞	英文名詞	概念說明
		陣。
PnP 問題	Perspective-n-Point	由已知三維點與其影像投影求相機位姿。
三角測量	Triangulation	由兩個視角的對應點恢復三維座標。
視差角	Parallax Angle	同一點對兩相機光心的夾角，影響三角測量精度。
隨機取樣一致	RANSAC	以隨機取樣與投票剔除離群點的穩健估計方法。
內點與外點	Inlier / Outlier	符合與不符合估計模型的資料點。

21.1 兩張影像如何推估相機移動

先建立本章的整體圖像。假設相機在位置 A 拍了一張照片，移動到位置 B 又拍了一張。第 20 章已經幫我們找出了兩張影像中的對應特徵點。現在的問題是：從這些對應點，能不能算出「相機從 A 到 B 的旋轉與平移」？

直覺：對應點的移動洩漏了運動

直覺上答案是肯定的。想像你站在教室前方拍一張照，往右走兩步再拍一張：畫面中所有物體都往左移了。若你是往前走，則畫面中的物體會向四周放射狀擴散（近的物體移動大、遠的移動小）。不同的運動，會在影像上產生不同的位移模式——這個模式就是我們要解讀的線索。

三種求解情境

視覺 SLAM 中，依「手上有什麼資訊」不同，會用到三種不同的求解方法。這個分類是本章的骨架，同學務必記牢：

情境	已知資訊	求解方法	本章節次
2D–2D	兩張影像的對應像素點	對極幾何（本質 / 基礎矩陣）	20.2–20.3
3D–2D	已知三維點 + 其影像投影	PnP 問題	20.4
3D–3D	兩組對應的三維點	ICP（第 17 章）	第 17 章

表 21-1 三種相機運動估測情境

15. 2D–2D：系統剛啟動時，還沒有任何三維地圖點，只能靠兩張影像的像素對應來初始化。這是單目 SLAM 的起手式。
16. 3D–2D：一旦地圖建立起來，後續每一幀都能用「地圖中已知的三維點」與「當前影像看到的二維投影」求解位姿。這是 SLAM 運行期間最常用的方法，效率高且穩定。

17. 3D–3D：當兩個視角都有深度資訊（雙目、RGB-D）時，可直接對兩組三維點做配準——這正是第 17 章的 ICP，同學已經學過。

本章聚焦前兩者。理解這個分類，同學就能看懂 ORB-SLAM（第 22 章）的原始碼在什麼時候呼叫哪一種求解器。

21.2 Epipolar Geometry

對極幾何是本章的理論核心。它回答一個看似簡單卻極深刻的問題：第一張影像中的某個點，在第二張影像中可能出現在哪裡？

搜尋範圍從二維降到一維

直覺上，答案似乎是「任何地方」——畢竟我們不知道那個點有多遠。但對極幾何告訴我們一個驚人的結論：它只可能出現在一條特定的直線上。

為什麼？考慮第一張影像中的一個像素點 p_1。它對應的三維點 P，必定落在「從第一個相機光心穿過 p_1 的那條射線」上——只是不知道在射線上的哪個位置（這正是第 9 章的深度遺失）。現在把這整條射線投影到第二張影像上，會得到一條直線。無論 P 實際在射線上的哪個位置，它在第二張影像中的投影，必定落在這條直線上。

這條直線就是對極線（Epipolar Line），而這個約束就是對極約束（Epipolar Constraint）。它的實用價值極大：原本要在整張影像（二維）中搜尋對應點，現在只需沿著一條線（一維）搜尋。這既加快了搜尋，也大幅減少了誤匹配的機會。

對極幾何的四個元素

18. 基線（Baseline）：連接兩個相機光心 O_1、O_2 的直線。它是整個幾何的骨幹。

19. 對極平面（Epipolar Plane）：由兩個光心與空間點 P 三者決定的平面。不同的 P 會產生不同的對極平面，但它們都包含基線——所有對極平面繞著基線形成一束。

20. 對極點（Epipole）：基線與各影像平面的交點，記為 e_1、e_2。幾何意義是「另一個相機的光心在本影像中的投影位置」。有趣的是：同一張影像中所有的對極線，都會通過該影像的對極點。

21. 對極線（Epipolar Line）：對極平面與影像平面的交線。p_1 在第二張影像中的對極線記為 l_2。

圖 21-1　對極幾何的結構

空間點 P

├──── 投影到影像 1 → 像素點 p_1

└──── 投影到影像 2 → 像素點 p_2

O_1 ──────基線──────→ O_2　（兩相機光心）

|　　|

└→ 對極點 e_1　　└→ 對極點 e_2

對極平面 = O_1、O_2、P 三點決定的平面

對極線 l_2 = 對極平面與影像 2 的交線

【核心約束】p_2 必定落在對極線 l_2 上

→ 搜尋範圍從二維（整張影像）降到一維（一條線）

一個生活化的驗證

同學可以親手驗證這個現象：閉上左眼用右眼看前方一枝筆，記住它在視野中的位置；

再閉右眼用左眼看，筆會「跳」到另一個位置。若你把筆前後移動再重複，會發現它在視野中跳動的軌跡，始終沿著一條固定的直線——那就是對極線。人的雙眼，就是一組天然的雙目系統。

21.3 Essential Matrix 與 Fundamental Matrix

對極約束若只是文字描述，無法用來計算。本節把它寫成矩陣形式——這就是本質矩陣與基礎矩陣。

對極約束的矩陣表示

對極約束的數學形式極為簡潔優美。設 p_1、p_2 是兩張影像中的一組對應點（以齊次座標表示），則：

$$p_2^T \cdot F \cdot p_1 = 0 \qquad （基礎矩陣形式，使用像素座標）$$

$$x_2^T \cdot E \cdot x_1 = 0 \qquad （本質矩陣形式，使用歸一化座標）$$

這兩條式子是本章最重要的公式。它們說：對應點與矩陣相乘的結果必須為零——換句話說，任何一組正確的對應點，都必須「滿足」這個約束。反過來，若我們蒐集足夠多的對應點，就能反解出這個矩陣。

兩者的差異

22. 基礎矩陣 F：作用於原始的像素座標，不需要知道相機內參 K（第 9 章）。它同時混合了「相機的內部參數」與「兩相機的相對運動」兩種資訊。自由度為 7。
23. 本質矩陣 E：作用於歸一化座標（已用內參 K 把像素座標轉換過），因此只包含「相對運動」的資訊，更為純粹。自由度為 5（三個旋轉、三個平移，但平移只知方向不知大小，故減一）。

兩者的關係為：

$$E = K_2^T \cdot F \cdot K_1$$

實務上，SLAM 系統的相機都經過校正（第 9.2 節），內參已知，因此多直接使用本質

矩陣。基礎矩陣則在內參未知的情況下使用（例如處理網路上來源不明的照片）。

從 E 恢復相機運動

求出本質矩陣後，可以透過奇異值分解（SVD）把它分解為旋轉 R 與平移 t。但這裡有兩個重要的性質必須注意：

24. 平移只知方向，不知大小：E 的定義中，把 t 乘上任意非零常數，約束仍然成立。這正是第 20.5 節單目尺度不確定性在數學上的體現——對極幾何本身無法告訴我們「相機移動了幾公尺」，只能告訴我們「往哪個方向移動」。
25. 分解有四組解：數學上，分解 E 會得到四組可能的（R, t）組合。要從中挑出正確的那一組，需要用一個物理判準：把對應點三角測量出三維座標後，只有正確的那組解會讓所有點都落在兩個相機的前方（深度為正）。這稱為正深度檢驗（Cheirality Check）。

特殊情況：單應矩陣

對極幾何有一個重要的例外情形。當場景中所有點都落在同一個平面上（例如相機對著一面牆、一張桌面、地板），或者相機只做純旋轉沒有平移時，對極約束會發生退化——本質矩陣的求解變得不穩定甚至無解。

此時應改用單應矩陣（Homography,H），它直接描述「一張影像的點如何映射到另一張影像」：

$$p_2 = H \cdot p_1$$

ORB-SLAM 在初始化時的一個聰明設計，就是同時計算基礎矩陣與單應矩陣，依據兩者的擬合分數自動判斷「這個場景比較像平面還是立體」，再選用合適的模型。這個細節在第 22 章會再提到。

把整條數學鏈實際算一遍

前面三節都是推導。推導讀起來優美，但同學心裡難免有個疑問：這套東西真的算得出來嗎？

本小節用一組可完整重現的合成資料，把「投影 → 八點法 → 分解 → 正深度檢驗 → 三角測量」整條鏈走一遍。

設定是這樣：相機內參為焦距 520 像素、主點在影像中心、解析度 640×480；場景裡隨機散布 240 個三維點，深度介於 4 至 12 公尺；第二個相機相對第一個旋轉約 9 度、平移 0.66 公尺。把這些點投影到兩張影像上，加入 0.3 像素的高斯雜訊，然後假裝不知道真實運動，只用這些二維對應點把它反推回來。

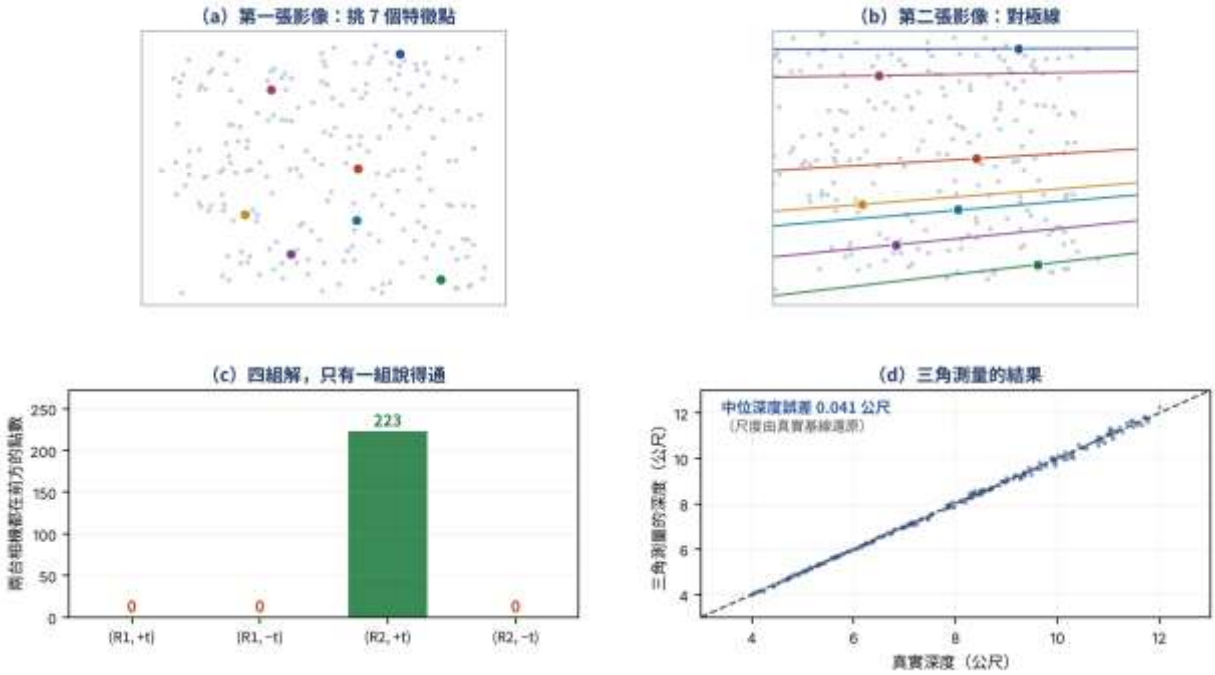


圖 21-2　對極約束的數值驗證：從對應點到三維座標

（a）是第一張影像，從 240 個點裡挑了 7 個標上顏色。（b）是第二張影像，每一條彩色直線就是對應顏色那個點的對極線——由基礎矩陣依 $l_2 = F \cdot p_1$ 算出來的。請注意每個彩色圓點都精確落在同色的直線上：這就是對極約束在影像上的樣子。原本要在整張 640×480 的影像裡找對應點，現在只要沿著一條線找。

（b）還透露了一個細節：七條對極線並不平行，它們微微地朝同一側收斂。延長出去，它們會交於同一點——那就是對極點，也就是第一個相機光心在第二張影像中的投影位置。本例的相機主要是橫向平移加小幅旋轉，所以對極點落在影像外的遠處，線看起來幾乎平行。如果相機是朝正前方推進，對極點就會落在影像中央，所有對極線會像放射狀的光芒一樣從

中心散開——同學可以自己改參數試試。

（c）是本章最能說明「數學歸數學、物理歸物理」的一張圖。把本質矩陣做奇異值分解，數學上會得到四組 (R, t) 組合，它們全都滿足對極約束，純粹從代數無法分辨誰對誰錯。但把每一組拿去三角測量，再檢查「這些三維點是不是在兩台相機的前方」，結果是：第三組有 223 個點深度為正，其餘三組一個都沒有。223 比 0 比 0 比 0——這不是勉強勝出，是壓倒性的。正深度檢驗之所以有效，正是因為錯誤的那三組會把整個場景放到相機背後，而那在物理上根本不可能被拍到。

（d）是最後一步：用挑出來的 (R, t) 把 223 個點三角測量回三維，再與真實深度比較。點幾乎全部貼在對角線上，中位深度誤差 0.041 公尺——在 4 至 12 公尺的場景裡約是千分之四。必須說明的是，橫軸的「真實深度」能對上，是因為我們用已知的真實基線把尺度還原了；若是真正的單目系統，整張圖會保持形狀但整體縮放一個未知倍數，這就是第 20.5 節的尺度不確定性。

零雜訊時，誤差真的是零

把上面的實驗把雜訊設成 0，八點法恢復出的旋轉誤差是 0.0000 度、平移方向誤差也是 0.0000 度。

這件事值得停下來想一想。它表示對極幾何不是一個「夠用的近似」，而是一組精確的代數關係——誤差完全來自量測，不來自方法本身。這與第 13 章的佔據格地圖、第 14 章的粒子濾波那種「本質上就是近似」的方法，是兩種完全不同的東西。

也因此，當同學在實作中發現位姿估得很差時，不該懷疑公式，而該去查三件事：特徵定位的精度、內參校正的準確度、以及有沒有混進錯誤匹配。接下來兩節就分別處理前兩件與第三件。

21.4　PnP 問題

對極幾何處理的是「兩張影像」的情形。但 SLAM 運行起來之後，我們手上已經有一張三維地圖了——此時求位姿有更好的方法。

問題定義

PnP（Perspective-n-Point，透視 n 點）問題的定義是：已知 n 個三維空間點的座標，以及它們在影像上的二維投影位置，求相機的位姿（旋轉 R 與平移 t）。

同學應該覺得眼熟——第 4.6 節介紹 AprilTag 時就用過這個概念：標記的四個角點三維座標已知（正方形邊長已知）、它們在影像上的像素座標也偵測得到，於是能解出相機相對標記的完整 6-DoF 位姿。那正是一個 n = 4 的 PnP 問題。

為什麼 PnP 比對極幾何好用

26. 有絕對尺度：三維點的座標是有單位的（公尺），因此解出的平移也有真實大小——不像對極幾何只能給方向。
27. 不需退化判斷：不會有平面場景退化的問題。
28. 所需點數少：最少三個點就有解（P3P），遠少於對極幾何所需的點數。
29. 效率高：在 SLAM 中，每一幀只需把「地圖中已知的點」投影比對即可，是追蹤階段的主力方法。

常見求解方法

30. P3P：用三個點求解，會得到最多四組解，需第四個點來排除。是許多方法的基礎。
31. EPnP：一種高效的線性解法，能處理任意數量的點，速度快且精度良好，是 OpenCV 與許多 SLAM 系統的預設選擇。
32. 直接線性變換（DLT）：把問題轉為線性方程組求解，概念簡單但精度較差，常作為初值。
33. 非線性最佳化（BA）：以上述方法求得初值後，再最小化重投影誤差（第 12 章）進行精修。這是實務上必做的最後一步。

重投影誤差：PnP 的最佳化目標

PnP 的精確解法，本質上就是最小化重投影誤差——第 12.5 節 Bundle Adjustment 的核心概念：

- 用當前估計的位姿，把每個三維點投影到影像上，得到「預測的像素位置」。
- 與「實際偵測到的像素位置」比較，兩者的差就是重投影誤差。
- 調整位姿，使所有點的重投影誤差平方和最小。

同學可以看出：PnP 其實是「只最佳化相機位姿、固定三維點」的簡化版 BA。而完整的 BA 則同時最佳化位姿與三維點。兩者是同一套數學的不同應用範圍——這也是第 12 章數學工具的又一次現身。

21.5 三角測量

前面兩節都是「由對應點求相機運動」。本節是反過來：已知相機運動，由對應點求三維座標。這就是三角測量（Triangulation），它讓單目 SLAM 能夠建立地圖。

原理：兩條射線的交點

原理極其直觀。已知相機在兩個位置的位姿後：

- 從第一個相機光心，穿過像素 p_1，畫一條射線。三維點 P 必在這條射線上。
- 從第二個相機光心，穿過像素 p_2，再畫一條射線。P 也必在這條射線上。
- 兩條射線的交點，就是 P 的三維位置。

理論上完美，實務上卻有個問題：由於像素座標有量測誤差，兩條射線通常不會精確相交，而是「擦身而過」。因此實際做法是求「離兩條射線都最近的點」，或以最小化重投影誤差的方式求解。

精度的關鍵：視差角

這是本節最重要的觀念。三角測量的精度，取決於兩條射線的夾角——稱為視差角

（Parallax Angle）。

34. 視差角大（相機移動距離相對於物體距離較大）：兩條射線接近垂直相交，交點位置明確，深度估計精準。

35. 視差角小（相機幾乎沒動，或物體非常遠）：兩條射線近乎平行，像素的微小誤差就會讓交點沿射線方向大幅滑動，深度極不準確。

這解釋了幾個重要的實務現象：

- 遠處的點測不準：同樣的相機移動，遠處物體的視差角小得多，因此天空、遠山的深度幾乎無法估計。這與第 9.6 節「雙目深度誤差隨距離平方成長」是同一件事。
- 純旋轉無法建圖：若相機只轉頭不移動，兩個光心重合，視差角為零，完全無法三角測量。這是單目 SLAM 初始化失敗的常見原因——同學在第 30 章實作時，若一直初始化不成功，多半是「只轉不移」。
- 需要足夠的基線：因此 SLAM 系統會等到相機移動足夠距離（產生足夠視差）後，才三角測量新的地圖點。這是關鍵幀選擇策略的考量之一（第 22 章）。

矛盾與解法

這裡存在一個微妙的矛盾：視差角要大，就需要相機移動較遠；但移動越遠，兩張影像的外觀差異越大，特徵匹配就越困難（第 20 章）。

實務系統的解法是：先用小基線的相鄰幀做穩定追蹤，再用相隔較遠的關鍵幀做三角測量。這種「近幀追蹤、遠幀建圖」的策略，正是 ORB-SLAM 區分「普通幀」與「關鍵幀」的原因之一。

量一量：雜訊與基線各值多少

21.5 節說「視差角小的時候深度不準」，這句話對，但不夠具體。不準到什麼程度？基線要多長才夠？本小節用兩組掃描實驗把這兩個問題量出來。

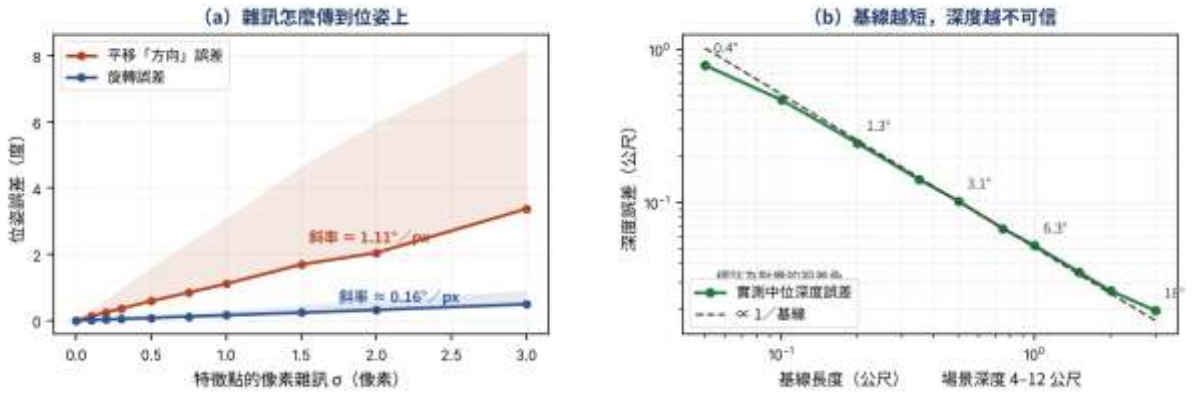


圖 21-3　雜訊如何傳到位姿，以及基線如何決定深度精度

（a）固定場景與運動，只改變特徵點的像素雜訊 σ，每個設定跑 40 次取中位數，淡色帶是第 90 百分位。兩條線都幾乎是直線——位姿誤差與像素雜訊成正比，沒有突然崩潰的門檻。這是好消息：特徵定位精度改善一倍，位姿精度就改善一倍。

真正值得記住的是兩條線的斜率差距。旋轉誤差約 0.16 度 / 像素，平移方向誤差約 1.11 度 / 像素——平移方向對雜訊的敏感度是旋轉的七倍。這個不對稱在實務上影響很大：視覺里程計的軌跡之所以會慢慢偏掉，主要不是因為轉角估錯，而是因為每一步的前進方向都歪了一點點。它也解釋了為什麼第 23 章的視覺慣性 SLAM 要把 IMU 加進來——IMU 的加速度計能對平移方向提供獨立的約束，恰好補在視覺最弱的地方。

（b）固定 0.5 像素的雜訊，改變基線長度，量三角測量的深度誤差。兩軸都是對數刻度，實測點幾乎貼著「與基線成反比」的虛線：基線 5 公分時深度誤差 0.79 公尺（在 4 至 12 公尺的場景裡幾乎沒有參考價值），基線 0.5 公尺時降到 0.10 公尺，基線 3 公尺時降到 0.020 公尺。每個點旁邊標的是對應的視差角，從 0.4 度一路到 18 度。

把這張圖倒過來讀更實用：若同學的專題要求深度誤差在 5 公分以內、場景深度約 10 公尺，那麼基線至少要 1 公尺——也就是說，相機必須移動一公尺之後才適合三角測量新的地圖點。這正是 21.5 節「需要足夠的基線」那句話的具體數字，也是第 22 章關鍵幀選擇策略背後的計算依據。

為什麼實務上用視差角而不用基線

圖 21-3(b) 的橫軸是基線長度，但真正決定精度的是視差角——也就是基線與場景深度的比值。同樣是 0.5 公尺的基線，對 5 公尺外的物體是 5.7 度的視差，對 50 公尺外只剩 0.57 度。

所以 SLAM 系統判斷「能不能三角測量了」時，看的是視差角而非移動距離。常見的門檻是 1 至 2 度：低於這個值，新點先不建，等相機再走一段。

這也說明了為什麼開闊的戶外場景對單目 SLAM 特別不友善——遠處的建築與地平線永遠達不到足夠的視差，能用來建圖的只有近處的路面與路邊物件。

21.6 RANSAC 排除錯誤匹配

前面幾節的數學都很優美，但它們有一個共同的前提：輸入的對應點是正確的。而第 20.4 節告訴我們，誤匹配必然存在。本節介紹對抗誤匹配的關鍵武器。

為什麼誤匹配如此致命

傳統的最小平方法（第 12 章）會把所有資料點的誤差平方和最小化。問題在於：平方會放大大的誤差。一個嚴重的誤匹配，可能產生數十甚至上百像素的誤差，平方之後主導整個目標函數，把估計結果硬生生拉偏。

打個比方：全班九十九位同學的身高都在 160 至 180 公分，但資料中誤植了一筆「1800 公分」。用平均值來估計班級身高，結果會被這一筆離譜的資料嚴重扭曲。最小平方法面對誤匹配，就是這個處境。

RANSAC 的核心思想

RANSAC（RANdom SAmple Consensus，隨機取樣一致）採用完全不同的策略：與其讓所有資料都參與，不如「隨機挑幾個、看看有多少人同意」。

- 隨機取樣：從所有對應點中，隨機挑出剛好足夠求解模型的最少數量（例如求本質矩陣需要 5 或 8 點，求 PnP 需要 3 或 4 點）。
- 求解模型：用這幾個點算出一個候選的相機運動（或矩陣）。
- 投票統計：用這個候選模型檢驗「所有」對應點，計算每個點的誤差。誤差小於門檻的稱為內點（Inlier），記錄內點數量。
- 反覆迭代：回到步驟 1，重複數十到數百次。
- 選出最佳：保留內點數最多的那個模型，並用它的所有內點重新做一次精確的最小平方求解。

為什麼這樣就有效

關鍵洞見在於：如果隨機挑的幾個點「碰巧全是正確的匹配」，那麼算出的模型就是正確的，而其他正確的匹配也都會同意它（成為內點），因此內點數會很高。反之，若挑到了誤匹配，算出的模型是錯的，幾乎沒有其他點會同意，內點數很低。

於是「內點數最多」就成為判斷模型正確與否的有效準則。RANSAC 不試圖修正誤匹配，而是直接把它們排除在外——這種「多數決」的精神，讓它能容忍極高比例的離群點（有時超過一半）。

實務要點

36. 迭代次數：需要多少次才能有高機率至少抽中一次「全是內點」的樣本？這可由機率公式估算（見公式或流程）。內點比例越低、所需樣本點數越多，就需要越多次迭代。
37. 門檻設定：誤差門檻決定了「多接近才算內點」。太嚴則正確的匹配也被排除、太鬆則誤匹配混入。通常依像素雜訊水準設定，例如 1 至 3 像素。

38. 與穩健核函數的關係：RANSAC 是「先剔除再最佳化」的硬性做法；第 12、15 章提過的穩健核函數則是「降低離群點權重」的軟性做法。兩者常搭配使用——先用 RANSAC 剔除明顯的誤匹配，再在最佳化中用穩健核函數處理殘餘的離群值。

39. 通用性：RANSAC 不限於視覺。第 17 章的點雲配準、第 13 章的平面擬合、甚至任何「有離群點的模型擬合」問題都能用它。它是工程師工具箱中最實用的方法之一。

量一量：RANSAC 到底救回了多少

21.6 節說誤匹配「致命」。這一節把「致命」兩個字量出來。

實驗方法是：在 0.5 像素雜訊的正確匹配中，刻意把一部分對應點的第二張影像座標換成隨機位置（模擬誤匹配），再比較兩種做法——直接把全部匹配丟進八點法，以及先用 RANSAC 篩出內點再估。每個比例跑 25 次取中位數。

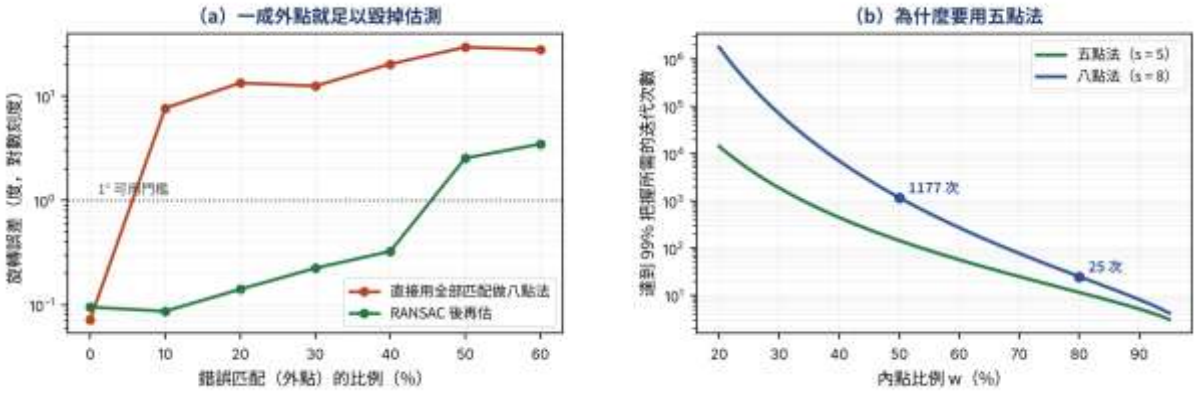


圖 21-4　誤匹配的代價，以及取樣點數為何關鍵

（a）的紅線說明了問題的嚴重性。零外點時，直接八點法的旋轉誤差是 0.07 度，相當精準。但只要混進 10% 的誤匹配，誤差就暴增到 7.68 度——一百倍。到 20% 時是 13.4 度，到 50% 時接近 30 度，也就是完全失去意義。請注意縱軸是對數刻度：這條線不是緩慢劣化，而是從一成外點就直接崩掉。

原因在 21.6 節講過：最小平方法把誤差平方後求和，而一個誤匹配可能造成上百像素的誤差，平方之後主導了整個目標函數。換句話說，這不是「精度稍微下降」，而是最佳化的目標本身被少數幾個點綁架了。

綠線是加上 RANSAC 之後。在 10% 外點時誤差是 0.087 度，甚至比零外點時的直接估計還略好一點（因為 RANSAC 順便剔掉了雜訊較大的點）；一路到 40% 外點都還維持在 0.33 度以內，遠低於 1 度的可用門檻。到 50% 時升到 2.5 度，開始吃力——這與理論一致：內點比例掉到一半時，隨機抽到「八個全是內點」的機率只有 $0.5^8 \approx 0.4\%$。

一個誠實的補充：在 50% 與 60% 外點的情況下，RANSAC 判定為內點的比例分別只有 21% 與 12%，明顯低於真實的內點比例。這表示它找到的是一個偏小的一致集合——模型大致正確，但沒有把所有好點都收進來。實務上的對策是增加迭代次數，或改用對外點更穩健的變形（如 MAGSAC、LO-RANSAC）。

（b）回答一個 21.6 節提過但沒算過的問題：要迭代幾次才夠？依內點比例 w 與最少取樣點數 s，達到 99% 把握所需的次數為 $N = \log(1-0.99) \,/\, \log(1-w^s)$。內點比例 50% 時，八點法需要 1,177 次，五點法只要 145 次；內點比例 80% 時則分別是 25 次與 12 次。這就是為什麼本質矩陣的五點法在實務上幾乎完全取代了八點法——不是因為它比較準，而是因為它讓 RANSAC 便宜了一個數量級。

取樣點數為什麼這麼關鍵

關鍵在於 w^s 這一項：s 出現在指數上。內點比例 50% 時，抽中八個全是內點的機率是 $0.5^8 = 0.39\%$，抽中五個全是內點則是 $0.5^5 = 3.1\%$，差了八倍。

這個觀察在 21.8 節會再出現一次：近年利用單目深度估計的新解法只需要三個點，在同樣 50% 內點比例下只要 34 次迭代——比八點法少了三十四倍。

所以當同學在設計任何 RANSAC 類的演算法時，第一個該問的問題永遠是：我能不能用更少的點把模型定下來？

21.7 從影像處理課程銜接到視覺 SLAM

本章的數學密度較高，本節退一步，幫同學把整條知識脈絡連起來，並說明如何從已學

過的影像處理課程，一步步走到視覺 SLAM。

知識的接續

階段	核心問題	關鍵技術	本書章節
影像處理	如何讓影像更清楚	濾波、邊緣偵測、形態學	先備知識
相機模型	三維如何變成二維	針孔模型、內參、畸變校正	第 9 章
特徵工程	如何找出可重複辨識的點	ORB、描述子、匹配	第 20 章
多視圖幾何	兩張影像藏著什麼幾何關係	對極約束、E / F、PnP、三角測量	本章
視覺里程計	如何連續估計相機軌跡	逐幀求解與累加	第 20、22 章
視覺 SLAM	如何消除漂移、建一致地圖	BA、閉環、關鍵幀	第 22、23 章

表 21-2　從影像處理到視覺 SLAM 的知識階梯

三個關鍵的觀念轉折

- 從「處理影像」到「理解幾何」：影像處理把影像當作二維訊號來處理（濾波、增強）；視覺 SLAM 則把影像視為「三維世界的投影」，關心的是背後的幾何結構。這是最根本的視角轉變。
- 從「單張影像」到「多視圖關係」：本章的對極幾何是這個轉折的核心——單張影像遺失了深度，但兩張影像之間的關係，足以恢復運動與結構。
- 從「精確假設」到「穩健估計」：影像處理的演算法多假設輸入是乾淨的；真實的 SLAM 必須面對誤匹配、雜訊與動態物體，因此 RANSAC 與穩健最佳化成為必備工具。這也呼應第 11 章「以機率處理不確定性」的精神。

學習建議

40. 先建立幾何直覺，再看公式：對極幾何的公式很簡潔，但直覺更重要。務必動手做本章的實作，把對極線畫出來看。
41. 善用視覺化：OpenCV 能畫出對極線、內外點、重投影結果。看到圖，抽象的矩陣就有了意義。
42. 與光達 SLAM 對照：回頭比較第 17 章的 ICP（3D–3D）與本章的 PnP（3D–2D）、對極幾何（2D–2D），你會發現它們是同一個問題在不同資訊條件下的變形。
43. 不必手推所有公式：本質矩陣的 SVD 分解、EPnP 的推導等細節，OpenCV 都已實作。理解「它在解什麼、什麼時候用、會失敗在哪」比背推導更重要。有志研究者可再深入 Hartley 與 Zisserman 的經典教科書。

至此，視覺 SLAM 的數學基礎已備妥。下一章我們把這些零件組裝成一個完整的系統——ORB-SLAM2。它如何用三個執行緒協同工作？關鍵幀與地圖點如何管理？共視圖是什麼？閉環如何偵測與修正？一個真正能跑的視覺 SLAM 系統，即將完整呈現在同學眼前。

21.8 對極幾何之後：2024–2026 年兩視圖估測的三個轉向

本章的數學在 1990 年代就已經成熟，而且至今仍是每一套視覺 SLAM 的底層。但這幾年它的「使用方式」正在改變。本節整理三個值得同學知道的轉向。

一、更少的點：讓單目深度參與求解

第一個轉向利用了一件新事實：單目深度估計網路（第 9 章提過）現在已經相當好用，雖然它給的深度帶有未知的尺度與偏移。2025 年的一項研究把這個「不完美的深度」直接寫進求解器：在估計相對位姿的同時，一併把深度的尺度與偏移當成未知數解出來。

結果是所需的最少對應點從五個降到三個。依 21.6 節的迭代公式，在 50% 內點比例下，三點法只需 34 次迭代，五點法需 145 次，八點法需 1,177 次。論文報告的實測是：在標準資料集上，三點解法的中位旋轉誤差為 0.89 度、耗時 22.4 毫秒；而搭配當前最佳稠密匹配器

的五點法雖然更準（0.56 度），卻需要 184.4 毫秒。也就是說，精度換到八倍的速度——這在嵌入式平台上是關鍵的取捨。

二、更密的匹配：從幾百個特徵到整張影像

第二個轉向來自匹配端。第 20 章介紹的是「偵測特徵點再比對描述子」，一張影像通常得到幾百到一兩千個對應。近年的稠密匹配方法則直接為影像中幾乎每一個像素找對應，一次產生數萬到數十萬組，而且在弱紋理與大視角變化下遠比特徵點穩健。

這對本章的數學產生了一個新問題：RANSAC 原本是為「幾百個對應」設計的，面對數十萬組對應，光是每次迭代要檢驗一遍所有點就慢得無法接受。因此 2025 年出現了專門處理這件事的研究方向——先把稠密匹配「摘要」成少量具代表性的對應，再餵給傳統的兩視圖估測器。這是一個很好的例子：新的前端並沒有淘汰舊的幾何，而是逼著舊的幾何重新設計它的介面。

三、直接回歸：跳過本質矩陣

第三個轉向最激進：乾脆不解本質矩陣，而是訓練一個網路，輸入兩張影像、直接輸出它們之間的相對位姿。

2025 年的一項代表性工作以約 800 萬組影像配對訓練，在室內資料集上的相對位姿精度（AUC@20）達到 75.56，明顯高於同類方法的 65.57；在室內定位任務上的平均中位誤差是 0.04 公尺與 1.02 度，而先前最好的同類方法是 0.36 公尺與 18.38 度。速度方面，它每一對影像只要 42 毫秒（約每秒 24 對），而稠密匹配方法需要 300 毫秒、重建式方法需要 441 毫秒。

這是否意味著本章可以不用學了？恰恰相反，有三個理由。其一，這類網路的訓練標註，正是用本章的方法產生的。其二，它們的評估指標（旋轉誤差、平移方向誤差、AUC）完全建立在本章的位姿表示上。其三，也是最實際的：當它估錯時，你需要本章的知識才能判斷錯在哪裡——是視差不足、是場景退化成平面、還是外點太多。

	傳統對極幾何	稠密匹配 + 幾何	直接回歸位姿
輸入	數百個特徵對應	數萬至數十萬組對應	兩張影像
最少點數	5 至 8（有深度時 3）	同左	不適用
可解釋性	高，每步可追溯	中	低
退化處理	需自行偵測（單應）	同左	隱含於訓練資料
典型耗時	數毫秒	數百毫秒	數十毫秒
目前角色	所有系統的底層	高精度離線重建	快速初始化與重定位

表 21-3　三種兩視圖位姿估測路線的比較

務實的結論是：三條路線正在分工而非互相取代。傳統對極幾何因為快、可解釋、不需要訓練資料，仍然是即時 SLAM 前端的預設選擇；稠密匹配用在離線的高精度重建；直接回歸則適合初始化與重定位這類「要快、容錯高」的場合。第 22、23 章要剖析的 ORB-SLAM 系列，走的是第一條路——也是同學最應該先徹底理解的一條。

公式或流程

一、對極約束

設 p_1、p_2 為兩張影像中的一組對應點（齊次像素座標），K_1、K_2 為相機內參（第 9 章），則對極約束為：

$$p_2^{T} \cdot F \cdot p_1 = 0$$

把像素座標用內參轉為歸一化座標 $x = K^{-1} \cdot p$，則約束改寫為本質矩陣形式：

$$x_2^{T} \cdot E \cdot x_1 = 0$$

兩矩陣的關係與本質矩陣的組成為：

$$E = K_2^{T} \cdot F \cdot K_1 \quad \cdot E = [t]_x \cdot R$$

其中 $[t]_\times$ 是平移向量的反對稱矩陣（第 10 章）、R 是旋轉矩陣。這條式子清楚顯示：本質矩陣正是「平移與旋轉」的緊湊編碼。由 E 分解出 R 與 t 即得相機運動，但 t 只有方向沒有大小（尺度不確定性）。

二、對極線的計算

第一張影像中的點 p_1，在第二張影像中的對極線為：

$$l_2 = F \cdot p_1 \qquad （直線方程式 a·u + b·v + c = 0 的係數）$$

反之，p_2 在第一張影像中的對極線為 $l_1 = F^T \cdot p_2$。實作時把 l_2 畫在第二張影像上，就能親眼看見「對應點必落在這條線上」的約束——這是本章實作最有啟發性的一步。

三、重投影誤差（PnP 與 BA 的目標）

設三維點 P、相機位姿（R, t）、內參 K、實際觀測到的像素 u_obs，則重投影誤差為：

$$e = u_obs - \pi(K, R, t, P)$$

PnP 求解即最小化所有點的重投影誤差平方和：

$$(R^*, t^*) = \arg\min \Sigma_i \, \| u_obs_i - \pi(K, R, t, P_i) \|^2$$

注意：PnP 只最佳化（R, t），三維點固定；完整的 Bundle Adjustment（第 12.5 節）則同時最佳化位姿與三維點。兩者是同一數學框架的不同應用。

四、RANSAC 的迭代次數

設內點比例為 w、每次取樣需要 s 個點、希望以機率 p 至少抽中一次全是內點的樣本，則所需迭代次數為：

$$N = \log(1 - p) \,/\, \log(1 - w^s)$$

舉例：內點比例 $w = 0.5$、求本質矩陣需 $s = 8$ 點、希望成功機率 $p = 0.99$，則 $N \approx 1177$ 次。若改用只需 $s = 5$ 點的方法，N 降至約 145 次——這說明了為何「所需點數少的求解器」在 RANSAC 中如此珍貴，也是五點法比八點法更受青睞的原因。

五、視覺前端的完整求解流程

圖 21-5　從特徵對應到相機位姿的完整流程

① 特徵匹配（第 20 章）→ 得到一組可能含誤匹配的對應點

↓

② 判斷情境：

- 系統初始化，無地圖點 → 走 2D–2D（對極幾何）
- 已有地圖點 → 走 3D–2D(PnP)

↓

③ RANSAC 迭代：隨機取樣 → 求解模型 → 統計內點

↓

④ 取內點數最多的模型，剔除外點

↓

⑤ 用全部內點做非線性最佳化（最小化重投影誤差）精修位姿

↓

⑥ 三角測量：對視差角足夠的新對應點，恢復三維座標並加入地圖

↓

⑦ 輸出相機位姿 + 新增地圖點 → 交給後端（第 22 章）

案例

案例 21-1　為什麼站著原地轉圈，單目 SLAM 就掛了

這是同學實作單目視覺 SLAM 時最常遇到的挫折。拿著手機或相機，系統一直顯示「初

始化中」，怎麼樣都成功不了。檢查了相機校正、特徵數量、光線，全都正常——問題到底出在哪？

答案往往極其簡單：你只在原地轉，沒有走動。

依 21.5 節的原理：純旋轉時，兩次拍攝的光心位置完全重合，基線長度為零，視差角為零。此時從兩個光心穿過對應像素的射線完全重疊，根本沒有交點可言——三角測量在數學上無解，系統當然無法建立初始地圖。

更微妙的是，純旋轉時對極幾何也會退化：平移向量 t 為零，本質矩陣 $E = [t]_x \cdot R$ 也變成零矩陣，求解自然失敗。此時應該用的是單應矩陣（21.3 節）——這也正是 ORB-SLAM 初始化時要同時計算兩種模型並自動選擇的原因。

解法很簡單：平移移動。拿著相機小幅度左右或前後平移數十公分，同時保持特徵在視野中，通常幾秒內就能成功初始化。這個案例呼應第 7.7 節「IMU 初始化需要運動激勵」——兩者的共同教訓是：視覺與慣性系統都需要「有內容的運動」才能觀測到某些量。

案例 21-2　對極線的視覺化：一眼看懂三維約束

對極幾何的公式看起來抽象，但把它畫出來的瞬間，一切都清晰了。這是大學課堂上最受歡迎的示範之一。

做法是：對同一場景拍兩張視角略不同的照片，用 ORB 找出對應點，以 RANSAC 估計基礎矩陣 F，然後對第一張影像中的每個特徵點，用公式二算出它在第二張影像中的對極線並畫出來。

結果會呈現一組壯觀的畫面：第二張影像上出現許多條直線，而每一個對應的特徵點，都精準地落在它自己那條線上。更神奇的是，若相機是往前移動，這些線會全部匯聚到畫面中的某一點——那就是對極點，也就是「另一個相機光心的投影位置」，幾何上正對應著你行進的方向。

這個實驗的教學價值極高：它讓同學親眼看到「兩張平凡的照片裡，竟然藏著如此嚴謹的三維約束」。抽象的矩陣 F，忽然變成了看得見的線。強烈建議教師把這個示範排進課堂，並讓同學自己拍照實作——本章的實作活動就以此為核心。

案例 21-3　RANSAC 的普世價值：超越視覺的方法論

RANSAC 是 1981 年由 Fischler 與 Bolles 提出的方法，原本就是為了電腦視覺中的位姿估計問題（有趣的是，他們提出時處理的正是 PnP 類問題）。但四十餘年來，它的影響力遠遠超出了視覺領域。

它被用在：點雲的平面與圓柱擬合（第 17 章）、光達的地面分離（第 18 章的 LeGO-LOAM）、機器人的感測器校正、生物資訊的序列比對、金融資料的趨勢擬合——任何「資料中混有離群點、需要找出主流模型」的場合。

RANSAC 之所以如此通用，是因為它體現了一個深刻的方法論：當你無法信任每一筆資料時，不要試圖用全部資料求平均，而要尋找「最多資料同意的解釋」。這是一種從「相信資料」轉向「相信共識」的思維轉變。

對同學的啟發是：學習演算法時，除了記住它怎麼做，更要理解它背後的思維模式。RANSAC 的「隨機取樣 + 多數決」、卡爾曼濾波的「按可信度加權」（第 11 章）、圖優化的「把矛盾均攤」（第 12 章）——這些思維框架，遠比具體的公式更能長久受用。

實作活動

實作 21-1　對極幾何與相機運動估測

目的：親手計算基礎矩陣、把對極線畫出來、執行 PnP 與三角測量，並見證 RANSAC 剔除誤匹配的威力。本實作以 Python 搭配 OpenCV，只需一支手機拍照即可完成，是全書最能建立幾何直覺的實驗之一。

第一部分：對極線視覺化（全體，本章核心）

14. 拍攝素材：對教室或走廊拍兩張照片，第二張相對第一張向右平移約 30 公分（注意是平移，不是原地轉）。確保畫面有豐富紋理。
15. 用 ORB 提取特徵並匹配（第 20 章的成果），套用比值檢驗初步剔除誤匹配。
16. 用 OpenCV 的 findFundamentalMat 搭配 RANSAC 估計基礎矩陣 F，同時取得內外點的標記。
17. 依公式二，對第一張影像中的內點，計算它們在第二張影像中的對極線並畫出來；同時把對應的特徵點也畫上。
18. 觀察與驗證：每個對應點是否都落在自己的對極線上？誤差有多大（以像素計）？所有對極線是否匯聚到同一點（對極點）？
19. 對照實驗：改拍一組「原地旋轉」的照片，重複上述流程，觀察估計是否變得不穩定——親身驗證案例 21-1 的退化現象。

第二部分：RANSAC 的威力（全體）

20. 不用 RANSAC（改用最小平方法）估計 F，把對極線畫出來，觀察是否明顯偏移。
21. 比較內點數：RANSAC 判定的內點佔全部匹配的百分之幾？被剔除的外點在影像上是什麼樣的匹配（把它們單獨畫出來看）？
22. 人為加入誤匹配：故意在匹配清單中加入若干隨機的錯誤對應，重跑 RANSAC，觀察它是否仍能正確剔除。加到多少比例才會失敗？
23. 依公式四計算：在你的內點比例下，理論上需要多少次迭代才能有 99% 的成功機率？與 OpenCV 的預設值比較。

第三部分：PnP 與三角測量（大學部）

24. PnP 實作：用第 9 章校正好的相機拍攝一張含棋盤格（或 ArUco 標記）的照片，已知格點的三維座標，用 solvePnP 求相機位姿，並把座標軸投影回影像驗證。
25. 三角測量實作：用第一部分求得的相機運動，對匹配的內點做三角測量（OpenCV 的 triangulatePoints），把恢復的三維點用 matplotlib 或 Open3D 顯示。

26. 視差角實驗：分別用「平移 10 公分」與「平移 50 公分」的兩組照片做三角測量，比較恢復的三維點雲品質（散布程度、深度合理性），驗證 21.5 節視差角對精度的影響。

27. 尺度驗證：量測場景中某兩點的真實距離，與三角測量結果比較——你會發現比例對但絕對值不對，親身體驗單目的尺度不確定性（第 20.5 節）。

討論問題

6. 對極線視覺化的結果中，對應點偏離對極線多少像素？這個誤差來自哪裡（特徵定位誤差？相機校正誤差？）

7. 原地旋轉的那組照片，基礎矩陣的估計為何不穩定？依 21.3 節，此時應該改用什麼模型？

8. RANSAC 剔除的外點，在影像上呈現什麼特徵（是隨機分布，還是集中在某些區域如重複紋理處）？

9. 平移 10 公分與 50 公分的三角測量結果差多少？這對「SLAM 系統何時該三角測量新地圖點」有什麼啟示？

10. （大學部）三角測量恢復的三維點，比例正確但絕對值不對。若要修正，依第 20.5 節你有哪三種選擇？各自的成本如何？

延伸挑戰（大學部）

其一：把本章的零件串成一個完整的兩幀視覺里程計——特徵匹配 → RANSAC 估計本質矩陣 → 分解出 R 與 t → 正深度檢驗選出正確解 → 三角測量建立初始地圖點。這正是 ORB-SLAM 初始化模組的簡化版，完成後你會對第 22 章的系統架構有極深刻的理解。其二：實作 ORB-SLAM 的模型選擇策略——同時計算基礎矩陣與單應矩陣，依擬合分數自動判斷場景是平面還是立體，並用合適的模型初始化。用一面白牆與一個立體場景分別測試，驗證選擇是否正確。

延伸閱讀

8. R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed., Cambridge University Press, 2004.（多視圖幾何的權威聖經，本章所有主題的完整推導來源）
9. M. A. Fischler and R. C. Bolles, "Random Sample Consensus," Communications of the ACM, 1981.（RANSAC 原始論文，提出時即用於位姿估計問題）
10. D. Nistér, "An Efficient Solution to the Five-Point Relative Pose Problem," IEEE TPAMI, 2004.（五點法求本質矩陣，大幅降低 RANSAC 所需迭代次數）
11. V. Lepetit, F. Moreno-Noguer, and P. Fua, "EPnP: An Accurate O(n) Solution to the PnP Problem," IJCV, 2009.（EPnP 原始論文，OpenCV 的預設 PnP 解法）
12. 高翔、張濤，《視覺 SLAM 十四講》第 7 講（視覺里程計：特徵點法）.（對極幾何、PnP、三角測量的華文最佳教材，含完整程式碼）
13. OpenCV 官方文件之 Camera Calibration and 3D Reconstruction 模組（docs.opencv.org）.（findFundamentalMat、solvePnP、triangulatePoints 等函式的使用說明）
14. UZH Vision Algorithms for Mobile Robotics 課程之 Multiple View Geometry 單元（第 2 章介紹）.（免費公開的課程影片與講義，可與本章對照學習）

小結

8. 相機運動估測依已知資訊分三種情境：2D–2D 用對極幾何（系統初始化）、3D–2D 用 PnP（SLAM 運行期間的主力）、3D–3D 用 ICP（第 17 章）。理解這個分類，就能看懂 SLAM 系統何時呼叫哪一種求解器。
9. 對極幾何的核心是對極約束：第一張影像中的一個點，在第二張影像中必落在一條對極線上，使搜尋範圍從二維降到一維。四個元素為基線、對極平面、對極點與對極線，且同一影像中所有對極線都通過對極點。

10. 對極約束的矩陣形式為 $p_2^T \cdot F \cdot p_1 = 0$ 與 $x_2^T \cdot E \cdot x_1 = 0$；基礎矩陣用像素座標且不需內參，本質矩陣用歸一化座標且更純粹。由 $E = [t]_x \cdot R$ 分解可得運動，但平移只知方向（尺度不確定），且有四組解需以正深度檢驗篩選。平面場景或純旋轉時應改用單應矩陣。

11. PnP 由已知三維點與其影像投影求相機位姿，優點是有絕對尺度、不會退化、所需點數少、效率高；常用解法包含 P3P、EPnP、DLT，並以最小化重投影誤差的非線性最佳化精修——它本質上是固定三維點的簡化版 Bundle Adjustment。

12. 三角測量由兩條射線的交點恢復三維座標，精度取決於視差角：視差角小（相機幾乎沒動或物體很遠）時深度極不準；純旋轉時視差角為零，完全無法三角測量——這是單目 SLAM 初始化失敗最常見的原因。

13. RANSAC 以「隨機取樣、求解模型、統計內點、反覆迭代、取內點最多者」對抗誤匹配；它不修正離群點而是直接排除，能容忍極高的離群比例。所需迭代次數由內點比例與取樣點數決定，這也是五點法優於八點法的原因。

14. 從影像處理到視覺 SLAM 有三個觀念轉折：從「處理影像」到「理解幾何」、從「單張影像」到「多視圖關係」、從「精確假設」到「穩健估計」。理解演算法在解什麼、何時用、會失敗在哪，比背推導更重要。

承先啟後

但這些都還是零件——特徵、對極幾何、PnP、三角測量、RANSAC，如何組裝成一個能長時間穩定運行、能建立一致地圖、能從跟丟中恢復的完整系統？下一章我們剖析 ORB-SLAM2：它如何以三個執行緒協同工作、關鍵幀與地圖點如何管理、共視圖如何組織空間關係、閉環如何偵測與修正。建議帶著本章的三張圖去看它的每一個設計決策：關鍵幀為什麼要等視差夠大才建新點（圖 21-3(b)）、為什麼每一步都要跑 RANSAC（圖 21-4(a)）、以及初始化時為什麼要同時算基礎矩陣與單應矩陣再挑一個。一個真正能跑的視覺 SLAM 系

統，即將完整呈現。

本章備齊了視覺 SLAM 的幾何工具。但這些都還是零件——特徵、對極幾何、PnP、三角測量、RANSAC，如何組裝成一個能長時間穩定運行、能建立一致地圖、能從跟丟中恢復的完整系統？下一章我們剖析 ORB-SLAM2：它如何以三個執行緒協同工作、關鍵幀與地圖點如何管理、共視圖如何組織空間關係、閉環如何偵測與修正。一個真正能跑的視覺 SLAM 系統，即將完整呈現。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 22 章

ORB-SLAM2 理論與實作

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

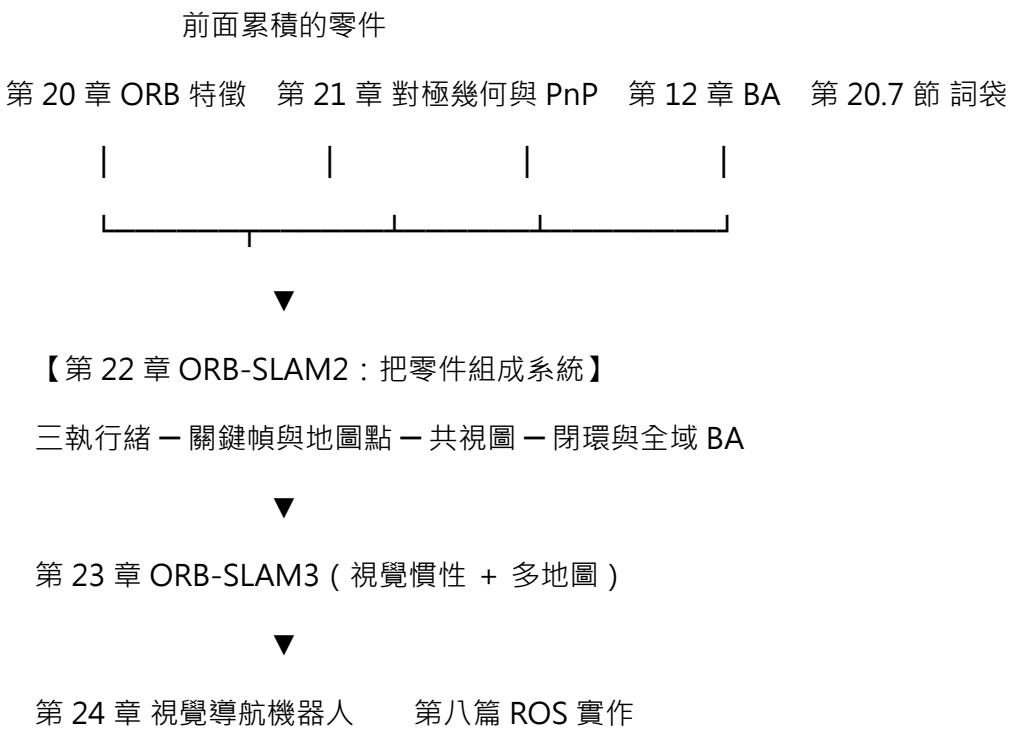
本章在全書的位置

第 20 章給了特徵，第 21 章給了幾何。本章把它們組裝成一個真的能跑的系統。

ORB-SLAM2 值得花一整章來讀，不是因為它的某個演算法特別聰明，而是因為它示範了「把正確的零件組起來」需要多少工程判斷：關鍵幀該什麼時候插、新的地圖點該信到什麼程度、地圖長太大了怎麼辦、閉環偵測錯了會怎樣。這些問題在論文的公式裡找不到答案，只能靠系統設計來回答。

本章的三張圖就是為了把這些工程判斷量化。圖 22-1 會數出新地圖點通過五道關卡後還剩多少、擋掉了多少誤匹配；圖 22-2 會示範一件違反直覺的事——閉環閉上了，地圖卻沒修好；圖 22-3 則把共視圖畫出來，說明為什麼它不等於「時間上相鄰的幀」。

圖 22-0　第 22 章的連結地圖



學習目標

第 20、21 章我們備齊了視覺 SLAM 的所有零件：特徵、對極幾何、PnP、三角測量、RANSAC。但零件不等於系統——如何讓它們協同運作、長時間穩定、跟丟能恢復、閉環

能修正？本章剖析 ORB-SLAM2 這個公認最成熟的開源視覺 SLAM 系統，把理論組裝成真正能跑的架構。學完本章之後，同學應能夠達成下列目標：

15. 說明 ORB 特徵在 ORB-SLAM2 中的角色，以及「同一套特徵貫穿三大任務」的設計優點。
16. 說明單目、雙目、RGB-D 三種模式的初始化差異。
17. 說明追蹤執行緒的工作流程與三種追蹤策略。
18. 說明局部建圖執行緒如何新增地圖點、執行局部 BA 與剔除冗餘。
19. 說明閉環執行緒如何偵測閉環、驗證相似變換並修正全域漂移。
20. 說明關鍵幀、地圖點、共視圖與生成樹的資料結構與作用。
21. 規劃並執行 ORB-SLAM2 在公開資料集上的實作與評測。

核心概念

中文名詞	英文名詞	概念說明
關鍵幀	KeyFrame	被保留下來代表某個視角的影像幀，含位姿與特徵。
地圖點	MapPoint	地圖中的三維特徵點，記錄座標、描述子與觀測資訊。
共視圖	Covisibility Graph	連接觀測到共同地圖點的關鍵幀所形成的圖。
生成樹	Spanning Tree	連接所有關鍵幀的最小生成樹，結構輕量。
本質圖	Essential Graph	共視強度高的邊加上生成樹與閉環邊，用於位姿圖優化。
追蹤	Tracking	對每一幀估計相機位姿的前端執行緒。
局部建圖	Local Mapping	管理與最佳化局部地圖的執行緒。
閉環	Loop Closing	偵測重訪並修正全域漂移的執行緒。

中文名詞	英文名詞	概念說明
僅運動 BA	Motion-only BA	固定地圖點、只最佳化相機位姿的束調整。
局部 BA	Local BA	最佳化局部關鍵幀與其所見地圖點的束調整。
全域 BA	Full BA	最佳化所有關鍵幀與地圖點的束調整。
地點辨識	Place Recognition	以詞袋比對判斷是否曾經來過的模組。
重定位	Relocalization	追蹤失敗後在地圖中重新找回位置。
相似變換	Similarity Transform	含尺度的變換，單目閉環修正時使用。

22.1 ORB 特徵提取

ORB-SLAM2 由西班牙薩拉戈薩大學的 Mur-Artal 與 Tardós 開發，是視覺 SLAM 領域最具影響力的開源系統。它的名字直接點出核心：整套系統建立在 ORB 特徵之上。

為什麼選 ORB：三個理由

第 20 章已經談過 ORB 的速度優勢，但在 ORB-SLAM2 中，選擇 ORB 還有更深的系統層面考量。這些特徵對旋轉與尺度具有穩健性，對相機自動增益、自動曝光與光線變化也有良好的不變性；此外它們提取與匹配快速，能支援即時運作，並在詞袋地點辨識中展現良好的精確率與召回率表現。

44. 速度：即時系統的硬性需求（第 20.3 節）。
45. 對曝光與光線變化的不變性：這一點常被忽略卻極為關鍵——相機的自動增益與自動曝光會不斷改變影像亮度，若特徵不具此種不變性，系統會頻繁跟丟。這也正是直接法（第 20.2 節）的軟肋。
46. 適合詞袋：二元描述子能高效地建立詞彙樹，使地點辨識又快又準。

關鍵設計：一套特徵、三種用途

這是 ORB-SLAM2 架構上最優雅的決定之一。系統對追蹤、建圖與地點辨識三項任務，使用的是同一套 ORB 特徵。

這個設計的好處是巨大的：

- 省下重複計算：影像只需提取一次特徵，三個模組共用。若追蹤用 ORB、閉環用 SIFT，則每幀要算兩套特徵，運算量倍增。
- 資料結構統一：地圖點只需儲存一種描述子，關鍵幀只需一種詞袋表示。
- 行為一致：追蹤能匹配上的特徵，閉環也能匹配上，不會出現「追蹤看得見但閉環認不出」的矛盾。

實作細節：金字塔與均勻分布

47. 影像金字塔：為了獲得尺度不變性，ORB-SLAM2 把影像縮放成多層金字塔，在每一層都提取 FAST 角點。這讓系統能匹配遠近不同的相同物體。

48. 均勻分布：直接提取 FAST 角點會造成特徵集中在紋理豐富處（如一幅畫上密集出現），其他區域則空無一物。ORB-SLAM2 用四元樹把影像遞迴切分，確保特徵均勻散布全畫面——這與第 18.2 節 LOAM 分區選取特徵的用意完全相同（避免幾何條件退化）。

22.2 單目、雙目、RGB-D 初始化

ORB-SLAM2 的一大特色是同一套系統支援三種相機。但三者的初始化差異極大，這是理解系統的關鍵之一。

單目：最困難的初始化

單目沒有深度資訊，必須靠兩張影像的視差來建立初始地圖（第 21 章）。而如同第 21 章案例 21-1 所述，若場景是平面或相機純旋轉，對極幾何會退化。

ORB-SLAM2 的解法極為聰明：它平行計算兩個幾何模型——針對平面場景的單應矩陣，以及針對非平面場景的基礎矩陣——然後依據兩者的相對分數選擇其中之一。

49. 若單應矩陣的分數明顯較高，判定場景為平面，以單應矩陣分解出運動。

50. 否則以基礎矩陣（本質矩陣）求解。

51. 若兩者都不夠好（例如視差不足），則放棄本次初始化，等待下一幀重試。

這正是同學實作時「一直初始化不成功」的機制原因：系統其實在保護你，不讓它用不可靠的模型建出錯誤的地圖。解法一如第 21 章所述——平移移動，製造足夠視差。

雙目與 RGB-D：立刻就能開始

雙目與 RGB-D 相機的初始化簡單得多：第一幀就有深度資訊，可以直接把有深度的特

徵點變成地圖點，建立初始地圖，系統立即進入正常追蹤。

52. 雙目：由左右影像視差計算深度（第 9.6 節）。

53. RGB-D：直接讀取深度圖。

54. 共同優點：有絕對尺度、不需等待視差累積、不會因純旋轉而失敗。

近點與遠點的區分

ORB-SLAM2 在雙目與 RGB-D 模式中有一個重要的設計：依深度把特徵點分為近點與遠點。近點（深度小於某個倍數基線）的深度量測可靠，可直接三角化為地圖點；遠點的深度不可靠（第 9.6 節深度誤差隨距離平方成長），則像單目一樣，等到多視角觀測累積足夠視差後再三角化。

這個設計展現了工程上的務實：不是所有量測都同等可信，系統應該依可信度分別處理——這與第 11 章卡爾曼增益「按可信度加權」的精神一脈相承。

22.3 Tracking 執行緒

ORB-SLAM2 以三個平行執行緒運作。系統有三個主要的平行執行緒：追蹤，對每一幀透過與局部地圖的特徵匹配並套用僅運動 BA 最小化重投影誤差來定位相機；局部建圖，管理並最佳化局部地圖，執行局部 BA；閉環，偵測大迴路並以位姿圖優化修正累積漂移。本節先看第一個。

追蹤的任務

追蹤執行緒是前端，它的唯一任務是：對每一張進來的影像，盡快算出「相機現在在哪裡」。它必須即時（與相機幀率同步），因此運算必須輕量。

三種追蹤策略

ORB-SLAM2 依情況採用三種策略，由易到難依序嘗試：

- 依運動模型追蹤：假設相機以等速運動，用上一幀的速度預測當前位姿，再與上一幀的地圖點匹配並精修。最快，也是正常情況下的主要路徑。
- 依參考關鍵幀追蹤：若運動模型失敗（例如相機突然改變運動），改用詞袋加速與最近的參考關鍵幀做特徵匹配，再求解位姿。
- 重定位：若前兩者都失敗（追蹤丟失），啟動地點辨識模組，在整個地圖的關鍵幀資料庫中搜尋相似畫面，以 PnP 加 RANSAC 求解位姿，把自己「找回來」。

追蹤局部地圖

求出初步位姿後，追蹤執行緒還會做一件關鍵的事：把「局部地圖」中的更多地圖點投影到當前幀，尋找額外的匹配，再做一次最佳化。這讓位姿估計使用更多觀測，精度顯著提升。

局部地圖的取得，靠的是共視圖（22.6 節）：找出與當前幀共視程度高的關鍵幀，以及它們所見的地圖點。這些圖結構讓系統能取得關鍵幀的局部視窗，使追蹤與局部建圖能在局部範圍內運作，因而能在大型環境中工作。

關鍵幀的選擇

追蹤執行緒的最後一項工作，是決定「這一幀該不該成為關鍵幀」。判準大致包括：距離上次插入關鍵幀已過足夠幀數、局部建圖執行緒處於閒置、當前幀追蹤到的地圖點數量足夠但與參考關鍵幀的共視比例已下降（表示視角有明顯變化）。

ORB-SLAM2 的策略是「寬鬆插入、嚴格剔除」：先盡量插入關鍵幀以保證追蹤穩健，再由局部建圖執行緒剔除冗餘者（22.4 節）。這個設計讓系統在快速運動時不易跟丟，同時避免關鍵幀無限膨脹。

22.4 Local Mapping 執行緒

局部建圖執行緒是中端，它不需即時，但負責維護地圖的品質。局部建圖執行緒把新的點插入局部地圖，並透過執行局部束調整來最佳化局部地圖。

五個步驟

- 插入關鍵幀：新的關鍵幀被插入共視圖、連接關鍵幀與擁有最多共同點的關鍵幀的生成樹，並建立該關鍵幀的詞袋表示（用於三角化新點時的資料關聯）。
- 剔除劣質地圖點：新建立的地圖點必須通過嚴格檢驗才能留下——必須被足夠多的關鍵幀觀測到、追蹤成功率必須夠高。不合格者立即刪除，避免污染地圖。
- 建立新地圖點：新地圖點由共視圖中相連關鍵幀的 ORB 特徵三角化產生；某關鍵幀中未匹配的 ORB 與其他關鍵幀中未匹配的 ORB 比較，配對必須滿足對極約束才算有效；成對的 ORB 被三角化後，還要檢查在兩幀中的深度是否為正，並檢驗視差、重投影誤差與尺度一致性。
- 局部束調整：對當前關鍵幀、共視圖中與它相連的關鍵幀，以及這些關鍵幀所見的所有地圖點，執行局部 BA（第 12.5 節）。這是精度的主要來源。
- 剔除冗餘關鍵幀：若某關鍵幀所見的地圖點中，有九成以上也被其他至少三個關鍵幀看到，則它是冗餘的，予以刪除。這讓地圖規模不會隨時間無限成長。

嚴格的品質控管

同學應該注意到：步驟 2、3、5 都在「刪東西」。這正是 ORB-SLAM2 精度的祕密——它對地圖品質極為嚴苛，寧可少一些地圖點與關鍵幀，也不要不可靠的資料。

三角化新點時的多重檢驗尤其嚴謹：對極約束、正深度、視差角（第 21.5 節）、重投影誤差、尺度一致性——五道關卡缺一不可。這反映了一個工程原則：錯誤的地圖點比沒有地圖點更危險，因為它會持續誤導後續的位姿估計。

基線的考量

實作上還有一個細節值得一提：三角化新點時，系統會從共視圖中選取鄰近的關鍵幀（單目模式取較多、非單目取較少），並且只使用其中基線足夠大的那些。這正是第 21.5 節「視差角決定三角測量精度」的直接應用——基線太小的關鍵幀對，三角化出來的點深度不可靠。

量一量：五道關卡各擋掉多少

22.4 節說新地圖點必須通過五道檢驗，而且「五道關卡缺一不可」。這句話聽起來很有道理，但到底每一道各擋掉多少？本小節用一個可重現的模擬把它數出來。

設定是一間 14×10 公尺的實驗室，牆面、地板與家具上共有 1,400 個地標；相機沿橢圓軌跡繞行兩圈（第二圈半徑略小，模擬真實的重訪不會走在同一條線上），依 22.3 節的判準選出關鍵幀。接著對系統真的會嘗試的每一對共視關鍵幀，把它們共同看到的地標當作正確候選匹配，再混入 25% 的誤匹配——其中一半是隨機的錯配，另一半刻意製造成「落在對極線上但深度錯誤」的錯配，因為真實的誤匹配往往正是後者（匹配本來就沿著對極線搜尋）。

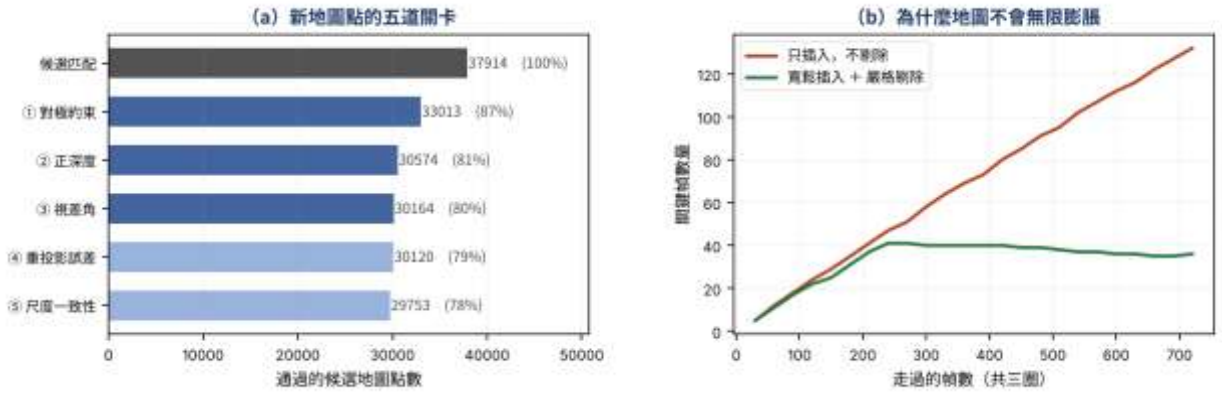


圖 22-1　新地圖點的五道關卡，以及關鍵幀為何不會無限成長

（a）是漏斗。37,914 個候選匹配裡，第①道對極約束擋掉 4,901 個（剩 87%），第②道正深度再擋掉 2,439 個（剩 81%），第③道視差角擋掉 410 個（剩 80%），第④道重投影誤差與第⑤道尺度一致性各擋掉數十到數百個，最後剩 29,753 個（78%）。

這張圖最值得討論的是前三道與後兩道的落差。對極約束、正深度、視差角三道合計擋掉了九成以上被擋的候選，而重投影誤差與尺度一致性加起來只擋掉不到 1.5%。這是不是表示後兩道可以省掉？不行，理由有二。其一，被它們擋掉的那幾百個點，正是「通過了前三道卻仍然錯誤」的點——這種點最危險，因為它看起來完全合理。其二，本模擬沒有建模金字塔層級，而真實系統的尺度一致性檢驗正是比對「觀測距離比」與「金字塔層級差」是否

吻合，在真實資料上它擋掉的比例會高得多。

更重要的是品質而非數量。這 29,753 個存活的候選裡，正確匹配佔 93.9%——比原始的 75% 大幅提升；9,475 個誤匹配中有 7,671 個被擋下（81%），而 28,439 個正確匹配裡只有 490 個被誤殺（1.7%）。這個「寧可嚴格一點」的比例配置，正是 22.4 節「錯誤的地圖點比沒有地圖點更危險」那句話的工程實現。

（b）回答另一個問題：既然關鍵幀是「寬鬆插入」，那麼在同一個地方繞三圈，關鍵幀會不會暴增？紅線是只插入不剔除的情況——三圈下來累積 132 個關鍵幀，而且還在線性成長。綠線加上九成規則的冗餘剔除（若某關鍵幀所見的地圖點有九成以上也被其他至少三個關鍵幀看到，就刪掉），結果在第一圈末尾就停在約 40 個，之後三圈都維持在 36 至 41 之間。

這正是 ORB-SLAM2 能長時間運行的關鍵。地圖的規模由「環境有多大」決定，而不是由「跑了多久」決定——這與第 19.6 節巡檢機器人的地圖維護、第 15 章 Cartographer 的子地圖管理，是同一個思想。

寬鬆插入、嚴格剔除：為什麼不乾脆一開始就嚴格

一個自然的疑問是：既然最後要刪掉七成，為什麼不一開始就少插一點？

因為插入的時機與剔除的時機，掌握的資訊量完全不同。追蹤執行緒在決定「這一幀要不要當關鍵幀」時，只看得到當下；它不知道接下來相機會往哪走、這個視角日後會不會變得重要。而局部建圖執行緒在剔除時，已經看過後續的關鍵幀，能判斷某一幀是否真的冗餘。

所以這不是「先浪費再收拾」，而是「把決策延後到資訊足夠的時候」。同樣的思想在第 14 章的粒子濾波重取樣、第 15 章的子地圖凍結時機都出現過，是系統設計上一個相當通用的原則。

22.5 Loop Closing 執行緒

閉環執行緒是後端，它處理 SLAM 最根本的問題：累積漂移。閉環執行緒負責偵測重訪區域，並使用 g2o 框架以位姿圖優化修正累積誤差；在位姿圖優化之後，它會啟動第四個執行緒執行完整束調整，最佳化整個地圖與關鍵幀之間的軌跡估計。

第一步：偵測候選閉環

ORB-SLAM2 基於 DBoW2 函式庫辨識重訪的地點；它把當前影像的特徵以二元格式，與階層式 DBoW2 資料庫中儲存的二元特徵比較，若當前幀與資料庫中某個表示的相似度達到一定分數，即偵測到閉環。

這裡的關鍵是效率：資料庫可能有數千個關鍵幀，不可能逐一做特徵匹配。詞袋模型把每張影像壓縮成一個「視覺單詞的統計向量」，比對只需計算向量相似度，速度極快——這正是第 20.7 節介紹 DBoW2 的價值所在。

第二步：幾何驗證

詞袋相似只是「看起來像」，不足以確認。系統接著做嚴格的幾何驗證：對候選閉環幀與當前幀做特徵匹配，並求解兩者之間的變換。若相似度得到足夠多內點的支持，閉環才被接受。

這一步至關重要。第 12 章與第 15.4 節都強調過：一條錯誤的閉環會把地圖硬拉到錯誤位置，造成災難性後果。因此寧可漏掉閉環，也不能接受錯誤的閉環。

第三步：相似變換與尺度修正

這裡有一個單目 SLAM 特有的細節。由於單目有尺度漂移（第 20.5 節），閉環時當前的尺度可能與初始尺度不同。因此求解的不是一般的剛體變換（旋轉加平移，第 10 章），而是相似變換——多加一個尺度因子。

修正時，系統會把這個尺度差異也一併攤平到整條軌跡上，同時修正位置與尺度漂移。

雙目與 RGB-D 模式因為有絕對尺度，則只需一般的剛體變換。

第四步：位姿圖優化與全域 BA

55. 位姿圖優化：在本質圖（22.6 節）上執行位姿圖優化（第 12 章），快速把整條軌跡拉正。因為只最佳化位姿（不含地圖點），速度快，能即時完成——這正是第 15 章 Cartographer 的相同策略。
56. 全域 BA：位姿圖優化後，系統啟動第四個執行緒執行完整的束調整，同時最佳化所有關鍵幀位姿與所有地圖點，得到最佳的整體結構與運動解。這一步較慢，因此放在背景執行，不阻塞前端追蹤。

這個「先快速拉正、再精細最佳化」的兩段式設計，同學應該覺得熟悉——它與第 18 章 LOAM 的雙層架構、第 15 章 Cartographer 的雙子系統，是同一個系統設計模式在不同系統中的體現。

量一量：閉環到底修好了什麼

22.5 節說閉環「修正累積漂移」。這句話對，但有一個常被誤解的地方：閉環把頭尾接上，不等於中間那一段就正確了。本小節把這件事量出來。

實驗用一個簡化的平面位姿圖（相機在水平面上繞一圈，因此可用二維位姿表示，這讓實驗能完整重現而不失去要點）。里程計每一步加入雜訊，繞完一圈後累積出明顯的漂移。接著比較三種位姿圖優化的設定：只加一條閉環邊、加上本質圖的邊、以及加上完整共視圖的邊。所有殘差都除以該項量測的標準差（第 11、12 章的資訊加權）。

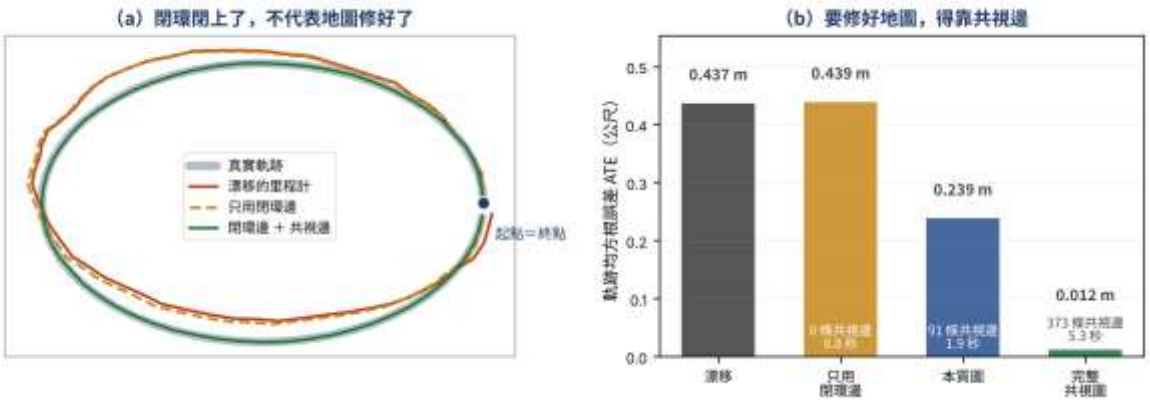


圖 22-2　閉環閉上了，不代表地圖修好了

（a）的灰色粗線是真實軌跡，紅線是漂移後的里程計估計，整條軌跡的均方根誤差（ATE）是 0.437 公尺，走回起點時與真正的起點差了 0.220 公尺。

橘色虛線是「只加一條閉環邊」的結果。它確實把頭尾接上了——閉環處的殘餘落差從 0.220 公尺降到 0.018 公尺，終點誤差也從 0.213 公尺降到 0.017 公尺。從「軌跡有沒有閉合」這個角度看，它完全成功。

但看整條軌跡：橘色虛線幾乎與紅線重疊，ATE 是 0.439 公尺——跟沒優化之前一模一樣。換句話說，閉環只把誤差從終點搬走了，卻沒有把它從中間消掉。原因很直觀：一條閉環邊只約束了「第一幀與最後一幀」的相對關係，中間那幾十個位姿彼此之間除了里程計之外沒有任何其他約束，優化器沒有理由去動它們。

綠線是加上共視邊之後。ATE 降到 0.012 公尺，是漂移的三十六分之一，而且與真實軌跡幾乎完全重合。關鍵在於共視邊把「不相鄰但看到同一批地標」的位姿直接綁在一起，形成一張網；閉環的修正量因此能沿著這張網傳遍整條軌跡，而不是只停在接縫處。

（b）把三種設定的代價也一起列出來。只用閉環邊：0 條共視邊、0.8 秒、ATE 0.439 公尺。本質圖：91 條邊、1.9 秒、ATE 0.239 公尺。完整共視圖：373 條邊、5.3 秒、ATE 0.012 公尺。必須誠實地說：在這個模擬裡，本質圖只把誤差改善到一半左右，離完整共視圖差得很遠。它換到的是四分之一的邊數與不到四成的時間。

這個取捨正是 ORB-SLAM2 的設計：閉環發生時先在本質圖上做位姿圖優化，因為它快到可以即時完成、不會卡住前端追蹤；接著在背景執行緒啟動全域 BA，用完整的資訊把地圖精修到位。先用便宜的方法把形狀大致拉正，再用昂貴的方法補完精度——這與第 18 章 LOAM 的雙層架構、第 15 章 Cartographer 的雙子系統，是同一個模式的第三次出現。

> **一個對專題很實用的診斷**
>
> 如果同學的 SLAM 系統「閉環之後看起來有閉上，但地圖還是歪的」，圖 22-2 就是在描述你的情況。
>
> 該檢查的不是閉環偵測，而是共視關係：系統有沒有正確建立非相鄰關鍵幀之間的邊？共視門檻是不是設得太高，導致圖太稀疏？位姿圖優化用的是本質圖還是只有里程計加閉環？
>
> 反過來，如果閉環處有明顯的「接縫斷裂」，那才是閉環偵測或幾何驗證的問題。兩種症狀的成因完全不同，先分清楚再動手。

22.6 KeyFrame、MapPoint、Covisibility Graph

一個系統的資料結構，決定了它能做什麼。本節說明 ORB-SLAM2 的四個核心資料結構。

關鍵幀（KeyFrame）

關鍵幀是被保留下來的影像幀，代表一個具代表性的視角。它儲存：相機位姿（第 10 章的變換矩陣）、所有 ORB 特徵點與描述子、該幀的詞袋向量、以及與其他關鍵幀的連接關係。

為什麼不保留所有幀？因為相鄰幀的資訊高度重複，全部保留會使 BA 的規模爆炸。關鍵幀是「以最少的幀涵蓋最多視角」的取捨結果。

地圖點（MapPoint）

地圖點是三維空間中的特徵點。它儲存：三維座標、代表性描述子（從所有觀測中選出最具代表性的那個）、平均觀測方向、以及「哪些關鍵幀看到了我」的觀測清單。

最後一項至關重要：它建立了「地圖點↔關鍵幀」的雙向連結，使系統能快速查詢「這個關鍵幀看到哪些點」與「這個點被誰看到」。共視圖正是由這個關係推導出來的。

共視圖（Covisibility Graph）

系統維護一個共視圖，連接任何兩個觀測到共同地圖點的關鍵幀，以及一個連接所有關鍵幀的最小生成樹。

57. 節點：每個關鍵幀。
58. 邊：兩個關鍵幀若共同觀測到足夠多的地圖點，就連一條邊；邊的權重是共同觀測的地圖點數量。

共視圖的作用是定義「鄰居」：與當前幀共視程度高的關鍵幀，就是空間上鄰近、視角相似的幀。追蹤與局部建圖都只在這個局部範圍內運作，因此能在大型環境中維持即時性（22.3 節引述的原理）。

生成樹與本質圖

59. 生成樹（Spanning Tree）：連接所有關鍵幀的最小生成樹，每個關鍵幀連到「與它共視最多的那個關鍵幀」。它極為輕量，保證所有關鍵幀連通。
60. 本質圖（Essential Graph）：閉環時做位姿圖優化的結構。它包含生成樹的所有邊、共視權重高於門檻的邊，以及閉環邊。相較於完整共視圖，本質圖的邊少得多，使位姿圖優化能快速完成。

這個「完整共視圖用於局部操作、精簡本質圖用於全域優化」的分層設計，是 ORB-SLAM2 能兼顧精度與效率的關鍵。這些圖結構在閉環時，也作為位姿圖優化所使用的結構。

資料結構	內容	主要用途
KeyFrame	位姿、ORB 特徵、詞袋向量、連接關係	地圖的骨架、BA 的變數
MapPoint	三維座標、描述子、觀測清單	追蹤的參考、BA 的變數
共視圖	關鍵幀節點 + 共視權重邊	取得局部地圖、定義鄰居
生成樹	最小生成樹的邊	保證連通、輕量結構
本質圖	生成樹 + 強共視邊 + 閉環邊	閉環時的位姿圖優化

表 22-1　ORB-SLAM2 的核心資料結構

把共視圖畫出來

共視圖是本章最抽象的概念，也最容易被誤解成「時間上相鄰的關鍵幀」。本小節把它實際畫出來。使用的是與圖 22-1 相同的模擬：相機在實驗室裡繞兩圈，第二圈的半徑比第一圈小三成。



圖 22-3　共視圖、局部地圖與本質圖

（a）是完整的共視圖：88 個關鍵幀、536 條邊（共視地圖點數達 15 個以上才連線，線的粗細代表共視的強度）。請注意內外兩圈之間密密麻麻的連線——那些就是「跨圈」的共視邊，連接的是時間上相隔數十個關鍵幀、但空間上看著同一面牆的兩幀。這張圖最重要的訊息是：共視關係由空間決定，不是由時間決定。

（b）是局部地圖。紅色星號是當前關鍵幀，橘色是它的局部地圖——共視最強的鄰居，加上這些鄰居的鄰居，一共 23 個關鍵幀。追蹤執行緒只需要把這 23 個關鍵幀所見的地圖點投

影到當前幀，而不是把全部 88 個關鍵幀的地圖點都拿來試。請注意橘色的點同時橫跨內外兩圈：機器人此刻能用上第一圈經過這裡時建立的地圖點，這正是 SLAM 與純里程計的根本差別。

更重要的是，局部地圖的大小不隨地圖成長。這張圖裡是 23 / 88；如果機器人繼續繞十圈、關鍵幀變成四百個，局部地圖仍然只會是當前位置附近的那二十幾個。這就是 22.6 節說共視圖「使系統能在大型環境中工作」的具體意思——追蹤的計算量與地圖總規模無關，只與局部密度有關。

（c）是本質圖：從 536 條邊精簡到 180 條，約三分之一。保留的是最小生成樹（確保每個關鍵幀都連得上）加上共視強度最高的那些邊。它保留了圖的整體連通結構與主要的跨圈連接，卻丟掉了大量冗餘的短邊——這就是閉環時位姿圖優化所使用的圖，也是圖 22-2(b) 裡那 91 條邊的來源。

三張圖對應三個執行緒

（a）完整共視圖：由局部建圖執行緒維護，是整個地圖的連接結構。

（b）局部地圖：由追蹤執行緒使用，決定每一幀要拿哪些地圖點來匹配。

（c）本質圖：由閉環執行緒使用，是位姿圖優化的對象。

同一份連接關係，三個執行緒各取所需——這就是 22.6 節「資料結構決定了系統能做什麼」的意思。如果共視圖建錯了，三個執行緒會同時出問題，而且症狀各不相同。

22.7 ORB-SLAM2 實作與資料集測試

本節說明實作要點，為第 30 章的完整實作與第 31 章專題預作準備。

編譯與相依套件

ORB-SLAM2 是 C++ 專案，編譯前需備妥幾個相依套件：

61. OpenCV：影像處理與特徵提取（第 9、20 章）。
62. Eigen3：線性代數運算。
63. Pangolin：三維視覺化，用於顯示地圖與軌跡。
64. g2o 與 DBoW2：已包含在專案的 Thirdparty 資料夾中，分別負責圖優化（第 12 章）與地點辨識（第 20.7 節）。

實作提醒：ORB-SLAM2 發布於數年前，在較新的作業系統與 OpenCV 版本上可能需要修改少量程式碼才能編譯成功。建議參考社群維護的分支版本，或使用容器化環境以避免相依性問題——這是同學實作時最常卡關的地方。

組態檔：相機參數

執行前必須準備一個 YAML 組態檔，內容包括：

65. 相機內參：fx、fy、cx、cy（第 9.1 節的 K 矩陣）。
66. 畸變係數：k1、k2、p1、p2（第 9.2 節）。
67. 影像尺寸與幀率。
68. 雙目的基線乘焦距、RGB-D 的深度縮放因子。
69. ORB 參數：每幀特徵數（常用 1000 至 2000）、金字塔層數與縮放比例、FAST 門檻。

這裡再次強調第 9 章的重要性：相機校正不準，整個系統的精度就無從談起。務必用自己的相機做過完整校正，不要直接套用範例參數。

公開資料集

資料集	相機類型	場景	特色
TUM RGB-D	RGB-D	室內辦公室	有動作捕捉真值，適合精度評測
KITTI	雙目	戶外道路	自駕車場景，大範圍長軌跡
EuRoC MAV	雙目 + IMU	室內機房與廠房	無人機視角，含快速運動序列

表 22-2　常用的視覺 SLAM 公開資料集

建議的學習順序是：先跑 TUM RGB-D 的簡單序列（緩慢運動、室內、有深度）建立成功經驗，再挑戰 EuRoC 的快速運動序列，最後嘗試 KITTI 的大範圍戶外場景。

評測方法

70. 絕對軌跡誤差（ATE）：把估計軌跡與真值軌跡對齊後，計算對應時刻位置差的均方根。這是最常用的整體精度指標。
71. 相對位姿誤差（RPE）：計算固定時間間隔內相對運動的誤差，反映局部漂移程度。
72. 工具：evo 是目前最方便的軌跡評測工具，支援多種資料集格式，能自動對齊、計算指標並繪圖。
73. 單目特別注意：單目軌跡沒有絕對尺度（第 20.5 節），評測時需先與真值做尺度對齊（Sim3 對齊），否則誤差會毫無意義。

觀察重點

執行時，建議同學在 Pangolin 視窗中留意幾件事：

- 初始化過程：單目模式下，觀察系統需要多少幀、多大的運動才能成功初始化（對照 22.2 節）。
- 關鍵幀的分布：觀察關鍵幀在軌跡上如何分布——是否在轉彎與視角變化大處較密集？

- 共視關係：視覺化中關鍵幀之間的連線就是共視圖，觀察它的結構。
- 閉環時刻：繞回起點時，觀察軌跡與地圖被拉正的瞬間（第 12、15 章的震撼畫面，在此再現）。
- 追蹤丟失與重定位：刻意遮住相機數秒再放開，觀察系統如何丟失又如何重定位回來——這是 ORB-SLAM2 最令人印象深刻的能力之一。

至此，一個完整的視覺 SLAM 系統已經完整呈現。但 ORB-SLAM2 仍有兩個限制：它是純視覺的（沒有 IMU，因此單目有尺度問題、快速運動易跟丟），而且一旦嚴重跟丟且無法重定位，系統就無法繼續。下一章的 ORB-SLAM3 正是為解決這兩點而生——緊耦合的視覺慣性融合，以及能在跟丟時開啟新地圖、日後再合併的多地圖系統。

22.8 ORB-SLAM2 之後：2024–2026 年視覺 SLAM 的兩條岔路

ORB-SLAM2 發表於 2017 年。近十年過去，它仍是比較新方法時最常用的基準線——這本身就說明了它的份量。但視覺 SLAM 的研究確實往兩個方向岔了出去，本節說明這兩條路各自要解決什麼，以及它們與本章內容的關係。

一、第一條路：把稀疏地圖換成可算繪的稠密地圖

ORB-SLAM2 的地圖是稀疏的——幾萬個三維點，足以定位，卻不足以「看」。近年的一條主線是把三維高斯潑濺（3D Gaussian Splatting）這類可算繪的表示法接進 SLAM，讓系統在定位的同時也建出能重新算繪成影像的稠密地圖。

值得同學注意的是，這類系統並沒有丟掉本章的東西。2024 年底的一項代表性工作明確地把 ORB 特徵與高斯潑濺緊耦合在同一套最佳化裡：追蹤時同時最小化光度誤差與重投影誤差，後端保留輕量的束調整來抑制累積漂移。作者的理由很直白——純粹依賴光度的高斯潑濺 SLAM 容易產生假影與雜訊、視角選擇不佳、而且缺乏全域最佳化，而這三件事正是本章的關鍵幀策略、共視圖與閉環在處理的。

效果上，該工作在 TUM 資料集的平均軌跡誤差是 0.91 公分，ORB-SLAM2 是 1.00 公分—

—定位精度只是略勝，真正的差別在算繪品質（相較純高斯潑濺的基線提升約 3.93 分貝）。作者也誠實指出限制：訓練耗時可觀，系統尚無法做到即時定位。這是一個很重要的提醒——稠密地圖的代價目前仍然很高。

二、第二條路：把前端換成學習式的三維重建模型

第二條路更激進：連特徵匹配與對極幾何都交給網路。2025 年出現的一類方法以學習式的三維重建模型作為前端，直接從兩張影像預測稠密的三維對應，再接上 SLAM 的後端。

它最引人注目的特性是不需要相機校正——系統使用通用的中心相機模型，因此可以直接處理來源不明的影片，這對第 9 章花了整節講內參校正的同學會有點衝擊。速度方面，在高階顯示卡上約每秒 15 幀，在中階卡上約每秒 5 到 7 幀；精度則在部分資料集上與 ORB-SLAM3 及其他稠密方法互有勝負。

三、本章的東西還剩多少

	ORB-SLAM2（本章）	高斯潑濺 SLAM	學習式前端 SLAM
地圖形式	稀疏特徵點	可算繪的稠密場	稠密點雲
前端	ORB 匹配 + PnP	光度 + 特徵	神經網路預測對應
需要內參	要	要	可以不要
關鍵幀機制	有	有	有
共視圖與閉環	有	多半保留	多半保留
即時性	CPU 即可即時	多半尚未即時	需高階 GPU
目前定位	成熟的基準線	算繪品質優先	免校正、泛化優先

表 22-3　三條路線的比較

從這張表可以看出一件事：兩條新路線換掉的是「地圖怎麼表示」與「對應怎麼找」，但關鍵幀、共視關係、閉環偵測與位姿圖優化——也就是本章 22.3 到 22.6 節的全部內容——幾

乎都原封不動地保留著。

理由不難理解。這些機制解決的不是「怎麼算出位姿」，而是「怎麼在長時間運行中管理資訊」：哪些幀值得記住、哪些資料彼此相關、什麼時候該承認自己回到了舊地方、以及誤差該怎麼重新分配。不論前端換成什麼，這些問題都還在。這也是本章值得仔細讀的理由——它教的是系統設計，而系統設計比演算法活得久。

公式或流程

一、三種 BA 的比較

ORB-SLAM2 在不同階段使用三種束調整（第 12.5 節），差別在於「哪些變數被最佳化」：

$$\text{僅運動 BA：} \operatorname{argmin}_{R,t} \Sigma_i \rho(\| u_i - \pi(K, R, t, P_i) \|^2)$$

固定地圖點 P_i，只求相機位姿（R, t）。用於追蹤執行緒的每一幀，速度最快。

$$\text{局部 BA：} \operatorname{argmin}_{R,t,P} \Sigma \text{局部關鍵幀與其地圖點的重投影誤差}$$

同時最佳化局部關鍵幀的位姿與它們所見的地圖點。用於局部建圖執行緒，是精度的主要來源。

$$\text{全域 BA：} \operatorname{argmin}_{\text{全部 } R,t,P} \Sigma \text{所有關鍵幀與地圖點的重投影誤差}$$

最佳化整張地圖，運算量最大，只在閉環後的第四個執行緒中執行。式中 ρ 為穩健核函數（第 12、15 章），用來降低離群觀測的影響。

二、位姿圖優化（閉環修正）

閉環時在本質圖上執行位姿圖優化，只最佳化關鍵幀位姿（不含地圖點）：

$$\operatorname{argmin} \Sigma_{(i,j)\in \text{本質圖}} \| \log(\Delta T_{ij}^{-1} \cdot T_i^{-1} \cdot T_j) \|^2_{\Sigma}$$

其中 ΔT_{ij} 是邊所記錄的相對位姿量測、T_i 與 T_j 是待最佳化的關鍵幀位姿。單目模式下 T

為含尺度的相似變換（Sim3），雙目與 RGB-D 則為剛體變換（SE3）。這正是第 12 章位姿圖優化的具體實現。

三、三執行緒協同流程

圖 22-4　ORB-SLAM2 的三執行緒架構

【追蹤執行緒】即時，每幀執行

提取 ORB → 依運動模型 / 參考幀 / 重定位求初始位姿

→ 追蹤局部地圖 → 僅運動 BA 精修 → 判斷是否插入關鍵幀

↓（新關鍵幀）

【局部建圖執行緒】非即時，關鍵幀觸發

插入共視圖與生成樹 → 剔除劣質地圖點 → 三角化新地圖點

→ 局部 BA → 剔除冗餘關鍵幀

↓（新關鍵幀）

【閉環執行緒】非即時，背景執行

DBoW2 偵測候選 → 幾何驗證 → 求解相似變換（Sim3）

→ 本質圖位姿圖優化（拉正軌跡）

↓

【第四執行緒】全域束調整，最佳化整張地圖與軌跡

四、地圖點三角化的五道檢驗

圖 22-5 新地圖點必須通過的檢驗

① 對極約束：配對的 ORB 必須滿足對極幾何（第 21 章）

↓

② 正深度檢驗：三角化後在兩幀中的深度都必須為正

↓

③ 視差角檢驗：視差必須足夠大，否則深度不可靠（第 21.5 節）

↓

④ 重投影誤差檢驗：投影回兩張影像的誤差必須夠小

↓

⑤ 尺度一致性檢驗：觀測到的尺度與距離必須合理

↓

五道全過 → 成為地圖點；任一失敗 → 捨棄

案例

案例 22-1 遮住相機再放開：重定位的魔術時刻

這是展示 ORB-SLAM2 能力最有戲劇效果的示範，幾乎每個課堂都值得做一次。

讓系統正常運行、地圖已建立一段之後，用手完全遮住相機鏡頭。畫面全黑，特徵瞬間歸零，系統立刻顯示追蹤丟失（TRACK LOST），軌跡停止更新。此時把手移開，讓相機重新看到之前走過的區域——不到一秒，系統就「跳」回正確的位置，軌跡無縫接續。

這一秒鐘裡發生了什麼？系統啟動了地點辨識模組：把當前影像的 ORB 特徵轉成詞袋向量，在關鍵幀資料庫中快速搜尋最相似的候選，對候選做特徵匹配，再以 PnP 加 RANSAC（第 21 章）求解位姿，通過驗證後恢復追蹤。

更有趣的變化是：遮住相機後，把相機搬到房間的另一個角落再放開。系統同樣能找回自己——這正是第 3 章「綁架機器人問題」的視覺版本。相較之下，若是純視覺里程計（沒有地圖與地點辨識），遮擋後就永遠回不來了。

這個示範讓同學親身體會「SLAM 與 VO 的差別」：VO 只能一路往前推算，SLAM 卻擁有地圖與記憶，能從失敗中恢復。這也是第 20.1 節流程圖中「閉環偵測」模組價值的最佳註腳。

案例 22-2　寬鬆插入、嚴格剔除：一個工程哲學

ORB-SLAM2 對關鍵幀的處理策略，乍看有些矛盾：插入時判準寬鬆（常常插入），但局部建圖又會積極剔除冗餘的關鍵幀。為什麼不乾脆一開始就嚴格一點？

原因在於兩者服務的目標不同。插入時的目標是追蹤穩健：若關鍵幀太稀疏，快速運動或大幅轉向時，當前幀與最近關鍵幀的視角差異過大，匹配就會失敗導致跟丟。寬鬆插入等於為系統買了保險。

剔除時的目標則是地圖精簡：關鍵幀無限增加會使 BA 規模膨脹、記憶體耗盡、系統逐漸變慢。所以在確認某個關鍵幀的資訊已被其他關鍵幀充分涵蓋（九成地圖點被至少三個其他關鍵幀觀測到）之後，就把它移除。

這個「先求穩、後求精」的策略，是一個值得記住的工程哲學：在不確定的當下寧可多做（保留選項），等資訊充足後再回頭精簡。同學會發現這與第 12 章「圖優化能回頭修正歷史」的優勢相呼應——正因為系統能事後修正，當下才敢寬鬆。

案例 22-3　從 ORB-SLAM2 長出的家族

ORB-SLAM2 開源之後，成為無數研究的基礎。觀察這個「技術家族」的開枝散葉，能看出視覺 SLAM 的研究地形。

74. 加入其他特徵：有研究在點特徵之外加入線段特徵。PL-SLAM 提出的系統同樣基於三個執行緒：視覺里程計、局部建圖與閉環；其地圖由關鍵幀、偵測到的三維地標（同時包含關鍵點與線段）、共視圖與生成樹組成。線特徵在人造環境（走廊、建築）中特別有價值，因為那裡點特徵可能不足但線條豐富。

75. 加速與輕量化：有研究提出免描述子的變體。FastORB-SLAM 的結構基於 ORB-SLAM2 並實作三個執行緒，但改以灰階匹配而非描述子匹配來建立相鄰幀之間的關鍵點對應，再以兩階段、與描述子無關的關鍵點匹配方法處理離群點問題，最後透過最小化重投影誤差恢復相機運動。

76. 應用延伸：有研究把 ORB-SLAM2 用於視障者的穿戴式導航裝置——由明眼人配戴裝置行走以建立「虛擬導盲路徑」，再供視障者依循。該研究說明追蹤執行緒能透過與局部地圖的特徵匹配定位相機，局部建圖執行緒把新點插入並以局部 BA 最佳化，閉環執行緒偵測大迴路並最佳化位姿圖。

77. 持久化地圖：也有研究為自駕車的視覺定位加上地圖儲存功能，讓建好的地圖能存檔重用，系統在定位模式下關閉局部建圖與閉環執行緒，僅執行追蹤。

這些延伸的共同點是：它們都保留了 ORB-SLAM2 的三執行緒骨架，只替換或增添其中的模組。這說明了一個好架構的價值——它不只解決當下的問題，更成為後續創新的平台。同學若要做視覺 SLAM 專題，以 ORB-SLAM2 為基礎進行改良，是一條務實且有前人可循的路徑。

實作活動

實作 22-1　ORB-SLAM2 系統實作與評測

目的：完整編譯並執行 ORB-SLAM2，在公開資料集上評測精度，觀察三執行緒的協同運作，並實驗系統的極限。這是第六篇最重要的實作，也是第 30、31 章的基礎。

第一部分：環境建置與資料集執行（全體）

28. 安裝相依套件（OpenCV、Eigen3、Pangolin），編譯 ORB-SLAM2。若遇到編譯錯誤，記錄問題與解法——這是實務工程的必經訓練。

29. 下載 TUM RGB-D 資料集的一個簡單序列（建議從 freiburg1_xyz 或 freiburg2_desk 開始），以 RGB-D 模式執行，觀察 Pangolin 視窗中的地圖與軌跡。

30. 同一段序列改用單目模式執行，比較兩者的差異：初始化耗時、軌跡完整度、地圖點密度。

31. 觀察並記錄：系統建立了多少關鍵幀？多少地圖點？關鍵幀在軌跡上的分布有什麼規律？

第二部分：三執行緒行為觀察（全體）

32. 初始化觀察（單目）：記錄系統從啟動到成功初始化所需的幀數與相機移動距離，對照 22.2 節的原理。

33. 共視圖觀察：在視覺化中觀察關鍵幀之間的連線，判斷哪些關鍵幀共視程度高，並與相機軌跡對照。

34. 閉環觀察：選一段含迴路的序列（如 KITTI 序列 00），觀察閉環觸發瞬間軌跡與地圖被拉正的過程，錄影記錄。

35. 重定位實驗：執行時人為中斷影像輸入數秒（或選用含遮擋的序列），觀察追蹤丟失與重定位的完整過程，重現案例 22-1。

第三部分：精度評測（大學部）

36. 安裝 evo 工具，把 ORB-SLAM2 輸出的軌跡與資料集真值比較，計算絕對軌跡誤差（ATE）與相對位姿誤差（RPE）。

37. 繪製軌跡對照圖與誤差隨時間變化圖，找出誤差最大的區段，並回頭觀察該區段的影像，推測原因（快速運動？弱紋理？動態物體？）

38. 單目評測注意：記得先做尺度對齊（Sim3），否則誤差無意義。比較「有無尺度對齊」的誤差數值差異，親身理解尺度不確定性。

39. 三模式比較：若資料集支援，分別以單目、雙目、RGB-D 執行同一場景，比較三者的 ATE 與運算負擔，填入你自己的比較表。

第四部分：參數實驗與極限測試（大學部）

40. 特徵數實驗：把每幀提取的 ORB 特徵數分別設為 500、1000、2000，比較精度、幀率與地圖點數量的變化，找出你場景的合適值。

41. 弱紋理挑戰：自行錄製一段包含大面積白牆的影片，觀察系統在哪裡跟丟，驗證第 9、20 章的「白牆詛咒」。

42. 快速運動挑戰：錄製一段快速轉向的影片，觀察追蹤是否失敗，並思考 IMU 能如何幫助（預告第 23 章）。

討論問題

11. 單目模式的初始化需要多少幀？若相機只做純旋轉，結果如何？這印證了 22.2 節與第 21.5 節的哪些論述？

12. 系統剔除了多少關鍵幀？若關閉冗餘關鍵幀剔除功能，長時間運行會發生什麼？

13. 閉環拉正的瞬間，軌跡上哪些區段位移最大？為什麼是那些區段？（提示：第 12 章「把矛盾均攤」）

14. 三種模式的 ATE 差多少？RGB-D 是否一定最準？在什麼情況下雙目會勝出？

15. （大學部）特徵數從 500 增至 2000，精度提升多少？幀率下降多少？這個取捨對即時系統意味著什麼？

延伸挑戰（大學部）

其一：閱讀 ORB-SLAM2 的 Tracking.cc 原始碼，找出三種追蹤策略（運動模型、參考關鍵幀、重定位）的實作位置，並在關鍵處加上輸出訊息，執行時觀察系統在什麼情況下切換策

略、切換頻率如何。這是把本章理論與真實程式碼對應起來的最佳練習。其二：嘗試用自己的相機（手機或網路攝影機）執行 ORB-SLAM2——先完成第 9 章的相機校正、撰寫 YAML 組態檔、再錄製影片或即時輸入，把你們系館的教室或走廊建成地圖。這份成果可直接作為第 31 章專題的素材。

延伸閱讀

15. R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo and RGB-D Cameras," IEEE T-RO, 2017.（本章主要依據的原始論文）
16. R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: A Versatile and Accurate Monocular SLAM System," IEEE T-RO, 2015.（前身之作，單目版本的完整設計）
17. ORB-SLAM2 開源專案（GitHub: raulmur/ORB_SLAM2）.（原始程式碼、編譯說明與資料集執行範例）
18. D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," IEEE T-RO, 2012.（DBoW2，閉環與重定位的技術基礎）
19. H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," RSS, 2010.（共視圖與 Sim3 閉環修正的來源）
20. 高翔、張濤，《視覺 SLAM 十四講》第 9 講（後端）與實踐章節.（華文最佳的 ORB-SLAM 系統剖析輔助教材）
21. evo 軌跡評測工具（GitHub: MichaelGrupp/evo）.（ATE 與 RPE 計算、軌跡繪圖，實作評測必備）

小結

15. ORB-SLAM2 對追蹤、建圖與地點辨識三項任務使用同一套 ORB 特徵；ORB 對旋轉、尺度、相機自動增益與曝光及光線變化具良好不變性，提取匹配快速，並在詞袋地點辨識中有良好的精確率與召回率。
16. 單目初始化平行計算單應矩陣（平面場景）與基礎矩陣（非平面場景），依相對分數擇一；雙目與 RGB-D 第一幀即有深度，可立即建圖，並依深度把特徵分為可靠的近點與需累積視差的遠點。
17. 追蹤執行緒以僅運動 BA 最小化重投影誤差來定位相機，依序嘗試運動模型、參考關鍵幀與重定位三種策略，並藉共視圖取得局部地圖做進一步精修；關鍵幀採「寬鬆插入」策略。
18. 局部建圖執行緒插入關鍵幀至共視圖與生成樹並建立詞袋、剔除劣質地圖點、由相連關鍵幀三角化新點（須通過對極約束、正深度、視差、重投影誤差與尺度一致性五道檢驗）、執行局部 BA，並剔除冗餘關鍵幀。
19. 閉環執行緒以 DBoW2 比對二元特徵偵測重訪，經幾何驗證取得足夠內點後接受閉環，求解相似變換（單目含尺度修正），在本質圖上做位姿圖優化，並啟動第四執行緒執行全域 BA。
20. 核心資料結構為關鍵幀、地圖點、共視圖（連接觀測共同地圖點的關鍵幀）、最小生成樹與本質圖；共視圖用於局部操作以支援大型環境，精簡的本質圖則用於閉環的位姿圖優化。
21. 實作需備妥 OpenCV、Eigen3、Pangolin，並以 YAML 提供精確的相機內參與畸變係數；常用資料集為 TUM RGB-D、KITTI 與 EuRoC，評測以 ATE 與 RPE 為指標，單目須先做尺度對齊。

承先啟後

本章把第 20、21 章的零件組成了一個真能跑的系統，也量出了幾個關鍵數字：新地圖點的

五道關卡把候選從 37,914 個篩到 29,753 個，正確率由 75% 提升到 93.9%，而正確匹配只誤殺了 1.7%；九成規則的冗餘剔除讓繞三圈的關鍵幀數停在 36 個而非 132 個；圖 22-2 則示範了一件違反直覺的事——只有一條閉環邊時，軌跡確實閉合了（接縫落差 0.220 降到 0.018 公尺），整體誤差卻完全沒改善（ATE 0.437 對 0.439 公尺）；要把誤差真正消掉，得靠共視邊把修正量傳遍整條軌跡（ATE 降到 0.012 公尺）。

ORB-SLAM2 是視覺 SLAM 成熟度的標竿，但它仍有兩個明確限制：純視覺架構使單目有尺度問題、快速運動易跟丟；而且嚴重跟丟且無法重定位時，系統便無以為繼。下一章的 ORB-SLAM3 正是為此而生——以最大後驗估測為基礎的緊耦合視覺慣性融合（第 7 章的 IMU 預積分在此登場），以及跟丟時開新地圖、日後再無縫合併的多地圖系統。讀第 23 章時可以帶著一個問題：圖 22-3 的共視圖只有一張，當系統同時持有好幾張互不相連的地圖時，這套機制要怎麼改？第六篇的最高峰，即將登場。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 23 章

ORB-SLAM3 與 Visual-Inertial SLAM

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

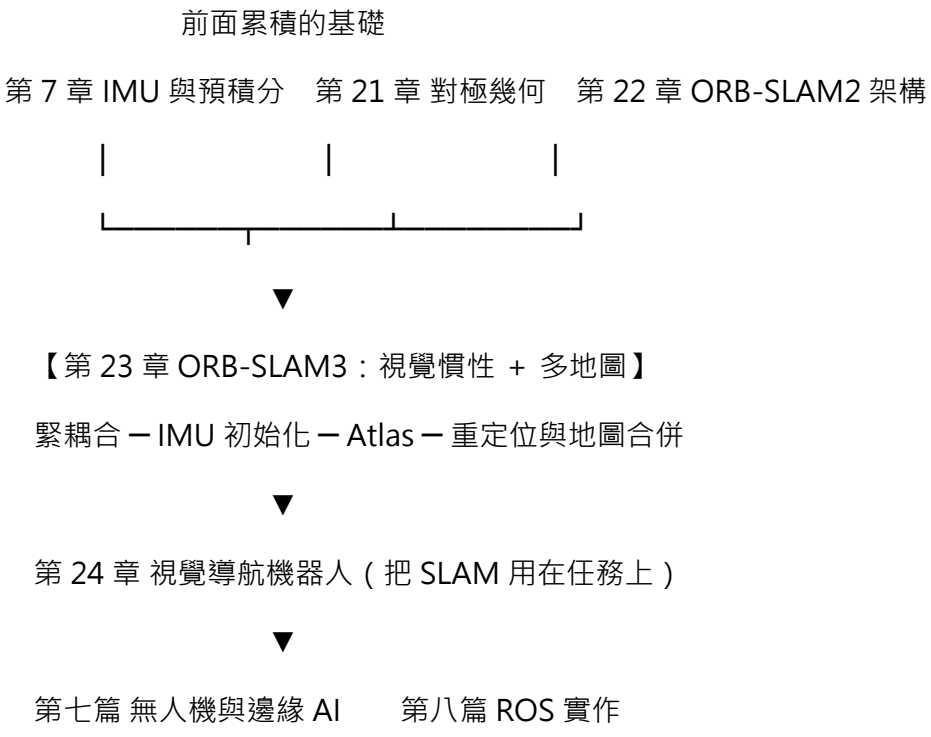
本章在全書的位置

第 22 章的 ORB-SLAM2 已經是一套完整的系統，但它有兩個硬傷：單目沒有絕對尺度，而且一旦嚴重跟丟又重定位不了，整個系統就到此為止。本章的 ORB-SLAM3 正是針對這兩件事而生——前者靠 IMU 解決，後者靠多地圖解決。

這也是第六篇的最高峰，以及第二篇與第六篇的交會點。第 7 章講了一整章的 IMU：它為什麼漂、零偏是什麼、為什麼需要運動激勵才能初始化——那些結論在本章全部派上用場，而且會被量化。

本章的三張圖分別回答三個問題：圖 23-1 量出 IMU 在視覺失效時能撐多久，以及零偏估不準的代價；圖 23-2 把「等速直線時尺度不可觀測」從一句口訣變成可以算出來的條件數；圖 23-4 則比較跟丟之後的三種命運——停住、開新圖不合併、以及 Atlas 的無縫合併。

圖 23-0　第 23 章的連結地圖



學習目標

上一章的 ORB-SLAM2 已是視覺 SLAM 成熟度的標竿，但它仍有兩個限制：純視覺架構使

單目有尺度問題、快速運動易跟丟；而嚴重跟丟且無法重定位時，系統便無以為繼。ORB-SLAM3 正是為此而生。本章是第六篇的最高峰，也是全書視覺路線的集大成。學完本章之後，同學應能夠達成下列目標：

22. 說明 ORB-SLAM3 相對 ORB-SLAM2 的三項核心突破。
23. 說明四種資料關聯（短期、中期、長期、多地圖）的意義與價值。
24. 說明單目慣性與雙目慣性 SLAM 的架構與 IMU 帶來的貢獻。
25. 說明以最大後驗估測為基礎的 IMU 初始化三階段流程。
26. 說明 Atlas 多地圖系統的組成與地圖合併的兩階段策略。
27. 說明追蹤失敗後的重定位與地圖合併機制。
28. 舉例說明視覺慣性 SLAM 在手機、無人機與穿戴裝置上的應用。

核心概念

中文名詞	英文名詞	概念說明
視覺慣性 SLAM	Visual-Inertial SLAM	緊耦合相機與 IMU 的同步定位與建圖。
多地圖系統	Atlas	同時維護多張地圖的架構，含一張活躍地圖與多張非活躍地圖。
活躍地圖	Active Map	追蹤執行緒當前用來定位的地圖。
非活躍地圖	Inactive Map	Atlas 中暫時未使用、但保留待日後合併的地圖。
短期資料關聯	Short-term	匹配最近數秒內的地圖元素。
中期資料關聯	Mid-term	匹配相機附近、帶有輕微累積漂移的地圖元素。
長期資料關聯	Long-term	以地點辨識匹配先前造訪區域，不受累積漂移影響。
多地圖資料關聯	Multi-map	匹配並使用先前建圖階段的地圖元素。

中文名詞	英文名詞	概念說明
最大後驗估測	MAP Estimation	在已知觀測下尋找機率最大的狀態估計。
IMU 初始化	IMU Initialization	估出尺度、重力方向、零偏與速度的啟動程序。
地圖合併	Map Merging	把兩張地圖無縫融合為一的操作。
焊接視窗	Welding Window	地圖合併時先行融合的局部關鍵幀範圍。
多階段作業	Multi-session	跨多次執行、重複使用先前地圖的運作模式。

23.1 ORB-SLAM3 的系統特色

ORB-SLAM3 由 Campos、Elvira、Rodríguez、Montiel 與 Tardós 於 2021 年發表，是第 7.7 節已初步介紹過的系統。它是特徵法 SLAM 這條血脈的集大成之作，從 PTAM、ORB-SLAM、ORB-SLAM2 一路演進而來，發布時是當時最完整、最精確的開源 SLAM 函式庫。

三項核心突破

78. 緊耦合的視覺慣性 SLAM：第一個以最大後驗估測（MAP）為基礎、連 IMU 初始化階段也建立在 MAP 之上的緊耦合系統（第 7.7 節）。這解決了單目的尺度問題與快速運動下的穩健度。
79. 多地圖系統（Atlas）：跟丟時不再崩潰，而是開啟新地圖繼續運作，日後重回舊區域時再無縫合併。這解決了 ORB-SLAM2「跟丟即結束」的致命弱點。
80. 四種資料關聯的全面運用：這是最根本的架構創新。ORB-SLAM3 是第一個在所有演算法階段都運用短期、中期、長期與多地圖資料關聯的系統。

支援的感測器組態

第 7.7 節提過，ORB-SLAM3 支援極為廣泛的組態，這是它實用價值的重要來源：

- 相機類型：單目、雙目、RGB-D。
- 鏡頭模型：針孔與魚眼（第 9 章）。魚眼支援對室內大視角應用特別有價值。
- 是否含 IMU：每一種相機模式都有對應的視覺慣性版本。

把這些組合起來，系統支援單目、單目慣性、雙目、雙目慣性、RGB-D、RGB-D 慣性等多種模式——同一套程式碼，換個參數即可。這讓它成為研究與教學的絕佳平台：同學能在完全相同的框架下，比較不同感測器組態的表現。

精度表現

第 7.7 節已引述過關鍵數據，此處補充多階段作業的表現。在 EuRoC 多階段實驗中，

Atlas 的地圖合併使精度較 CCM-SLAM 與 VINS-Mono 提升一倍以上；相對 VINS-Mono 的優勢從單階段的 2.6 倍成長到多階段的 3.2 倍。整體而言，論文摘要宣稱其精度較先前方法提升二至十倍；開源釋出後，它成為數百篇後續論文的標準比較基準。

這個「單階段已勝出、多階段優勢更大」的現象，清楚說明了多地圖架構的價值：能重複使用先前的地圖，精度自然更高。這對實際部署的機器人（每天在同一場域運行）意義重大。

23.2 四種資料關聯：系統的架構主軸

本節是理解 ORB-SLAM3 設計哲學的關鍵。作者把「資料關聯」——也就是「判斷不同時刻的觀測是否對應同一個地圖元素」——分為四個層次，並主張先前的系統往往只用了其中一兩種。

類型	匹配對象	技術手段	解決什麼
短期	最近數秒內的地圖元素	投影匹配	基本的逐幀追蹤
中期	相機附近、有輕微漂移的元素	局部地圖投影匹配	減少漂移累積，提升精度
長期	先前造訪區域（不論漂移多大）	地點辨識（DBoW2）	閉環修正、重定位
多地圖	先前建圖階段的地圖元素	地點辨識 + 地圖合併	跨階段重用、跟丟後恢復

表 23-1 四種資料關聯的比較

短期與中期：日常運作的基礎

幀與活躍地圖之間的短期與中期資料關聯，是由追蹤與建圖執行緒例行找出的——把地圖點投影到估計的相機位姿，並在影像中僅數個像素的視窗內搜尋匹配。

中期關聯是 ORB-SLAM3 精度優勢的重要來源。實驗顯示，在 EuRoC 資料集的單目慣性單階段運作中，ORB-SLAM3 的精度是 VINS-Mono 的 2.6 倍，這歸功於它使用中期資料關聯的能力。

為什麼中期關聯如此重要？因為許多視覺里程計系統只做短期關聯（只與前幾幀比對），一旦某個地圖點暫時離開視野再回來，系統就把它當成新的點——這浪費了資訊，也讓漂移持續累積。ORB-SLAM3 則能重新匹配上這些「還在附近但有些漂移」的舊點，等於不斷地做小規模修正。

長期與多地圖：恢復與重用

為了達成重定位與閉環偵測所需的長期資料關聯，ORB-SLAM 使用 DBoW2 詞袋地點辨識系統，這個方法也被多數近期的 VO 與 SLAM 系統所採用。

多地圖關聯則是 ORB-SLAM3 獨有的第四層。它讓系統能匹配「不同建圖階段」的地圖元素——無論是同一次執行中跟丟後開的新地圖，還是昨天建好存下來的地圖。這開啟了多階段作業的可能。

觀念總結

這四層關聯可以用一個記憶的比喻來理解：短期是「剛剛看到的」、中期是「這附近熟悉的」、長期是「以前來過的」、多地圖是「別次經歷的」。一個好的 SLAM 系統，應該像人一樣善用所有層次的記憶——這正是 ORB-SLAM3 的設計哲學。

23.3 Monocular / Stereo Visual-Inertial SLAM

第 7.6 節已經說明過 IMU 在視覺慣性 SLAM 中的四大貢獻（絕對尺度、重力方向、填補視覺失效空隙、高頻輸出），本節聚焦 ORB-SLAM3 如何實現緊耦合。

為什麼需要緊耦合

IMU 量測角速度與線加速度，但存在顯著漂移，使得單靠 IMU 資料的二次積分無法用

於長期位姿估計。這正是第 7.3 節的核心結論。

反過來說，純視覺在快速運動、弱紋理、光線劇變時會失效。因此兩者必須融合——而如第 7.6 節所述，緊耦合（把 IMU 預積分約束與視覺重投影誤差放進同一個最佳化問題）的精度顯著優於鬆耦合。

ORB-SLAM3 的緊耦合實現

系統的三個執行緒都因 IMU 而擴充：

- 追蹤執行緒：除了最小化重投影誤差，還加入 IMU 預積分的慣性誤差項（第 7.5 節），同時估計位姿、速度與 IMU 零偏。
- 局部建圖執行緒：局部建圖模組把關鍵幀與地圖點加入當前活躍地圖、移除冗餘的幀，並在當前幀的鄰近關鍵幀上，以視覺束調整或視覺慣性束調整最佳化地圖。
- 閉環與地圖合併執行緒：在慣性模式下，閉環修正還需處理重力方向與尺度的一致性。

狀態向量的擴充

與純視覺相比，視覺慣性 SLAM 每個關鍵幀要估計的量變多了：

- 純視覺：位姿（旋轉 R、平移 t），共 6 個自由度。
- 視覺慣性：位姿 + 速度 + 加速度計零偏 + 陀螺儀零偏，共 15 個自由度。

狀態變多雖然增加了運算量，但換來的是：絕對尺度（單目不再有尺度不確定性）、重力方向已知（地圖自動擺正，滾轉與俯仰角可觀測）、以及在視覺短暫失效時仍能維持追蹤數秒的能力。這個交換在絕大多數應用中都極為划算。

雙目慣性：最強健的組態

若不計成本與體積，雙目慣性是 ORB-SLAM3 最強健的組態：雙目提供絕對尺度與單幀深度（不需等待視差累積）、IMU 提供高頻運動與快速運動下的穩健度。第 7.7 節引述的 EuRoC 3.5 公分精度，正是雙目慣性模式的成績。對校園的巡檢機器人或無人機專題，若條

件允許，這是首選組態。

量一量：IMU 到底能撐多久

23.3 節說緊耦合能「在視覺短暫失效時仍能維持追蹤數秒」。「數秒」是多少秒？本小節把它算出來，順便回答一個更重要的問題：這個能力取決於什麼。

實驗設定：讓載體做一段含加減速與旋轉的運動，IMU 以 200 Hz 輸出比力讀值（含零偏與雜訊）。從某一刻起假設視覺完全失效，持續 τ 秒，期間只靠 IMU 二次積分推算位置，最後比較推算位置與真實位置的誤差。關鍵變因是「系統對加速度計零偏估得多準」，因為推算時要先把估計的零偏扣掉。

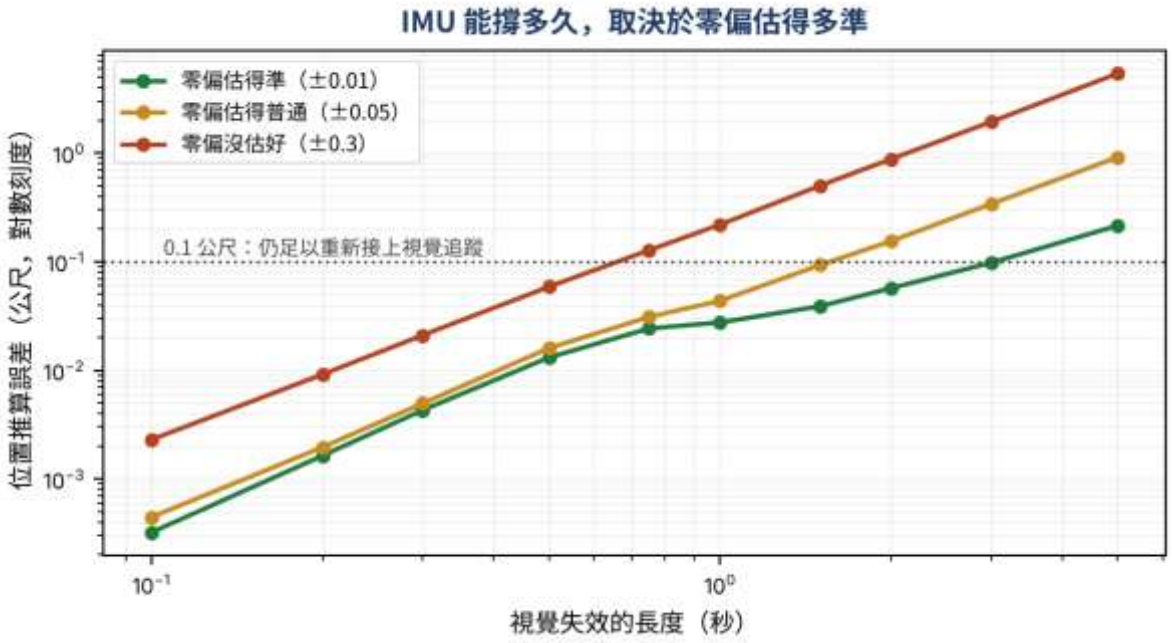


圖 23-1　IMU 能撐多久，取決於零偏估得多準

三條線的形狀都是二次成長——這是第 7.3 節的必然結果：加速度的誤差積分兩次變成位置誤差，所以誤差隨時間平方累積。真正的差別在於三條線之間的垂直距離。

若零偏估到 ±0.01 公尺 / 秒² 的水準（綠線），視覺失效 1 秒的位置誤差是 0.028 公尺、2 秒是 0.057 公尺、即使 5 秒也只有 0.21 公尺。這個精度足以讓系統在視覺恢復時直接把地圖點投影回畫面重新接上——也就是「使用者幾乎不會察覺中斷」的技術基礎。

若零偏只估到 ±0.3 公尺 / 秒²（紅線），同樣的 1 秒誤差是 0.22 公尺、2 秒是 0.88 公尺、5 秒是 5.39 公尺。與綠線相比，2 秒時差了 15 倍。0.88 公尺的誤差意味著重新投影時完全對不上原來的地圖點，系統只能走重定位流程，甚至直接跟丟。

這組數字解釋了一件事：IMU 在視覺慣性 SLAM 裡不是「備援感測器」那麼簡單。它能不能救場，取決於零偏有沒有被持續而正確地估計——而零偏的估計又回頭依賴視覺提供的位置約束。兩者是互相依賴的，這正是 23.3 節強調「緊耦合」而非「鬆耦合」的根本理由：把 IMU 當成獨立模組接上來，零偏就永遠估不準。

> **為什麼零偏比雜訊重要得多**
>
> IMU 的隨機雜訊是零均值的，積分之後會互相抵消一部分，誤差大致隨 $\sqrt{t}$ 成長。但零偏是常數偏移，積分兩次之後誤差隨 t^2 成長——兩者完全不是同一個量級。
>
> 以本實驗的數字為例：加速度計雜訊是 0.02 公尺 / 秒²，而未校正的零偏可達 0.1 公尺 / 秒²以上。五秒之後，雜訊貢獻的誤差不到幾公分，零偏貢獻的卻是公尺級。
>
> 所以同學在調校視覺慣性系統時，若發現「靜止時位置會慢慢飄走」，要查的是零偏估計而不是雜訊參數。

23.4 IMU 初始化與預積分

第 7.7 節已介紹過 IMU 初始化的困難與運動激勵的必要性，本節深入其三階段流程——這是 ORB-SLAM3 最重要的技術貢獻之一。

初始化要解出什麼

系統啟動時，有一堆彼此耦合的未知數必須同時確定：

- 尺度：單目模式下地圖的真實大小（雙目與 RGB-D 已知，故較單純）。
- 重力方向：哪個方向是「下」，用來從加速度計讀值中扣除重力（第 7.1 節的比力觀念）。

- IMU 零偏：加速度計與陀螺儀的偏移（第 7.3 節）。
- 初始速度：每個關鍵幀的速度。

三階段流程

ORB-SLAM3 把初始化拆成三個階段，逐步逼近：

81. 純視覺 MAP 估測：先只用視覺，建立一段短時間的地圖與軌跡（單目模式下尺度未知）。這提供了後續慣性估計所需的運動基準。
82. 純慣性 MAP 估測：固定視覺軌跡不動，只用 IMU 量測來估計尺度、重力方向、零偏與速度。因為視覺軌跡已定，這個子問題變得相對簡單。
83. 聯合視覺慣性 BA：最後把所有變數（位姿、速度、零偏、尺度、重力、地圖點）放進同一個最佳化問題聯合精修。

這個「先分開解、再聯合精修」的策略，大幅提升了初始化的速度與可靠度。

初始化的表現

IMU 初始化在 2 秒內達到 5% 的尺度誤差，經精修後約為 1%。這相較第 7.7 節提到的早期系統（ORB-SLAM-VI 需約 15 秒）是巨大的進步——2 秒對使用者體驗而言幾乎無感，15 秒則明顯令人不耐。

實作提醒：運動激勵

再次強調第 7.7 節的關鍵提醒：IMU 初始化需要足夠的運動激勵。若載體只是等速直線移動或完全靜止，加速度計讀到的幾乎只有重力，系統無法把「尺度」與「零偏」區分開來（數學上不可觀測）。

實務操作：啟動後拿著裝置做數秒帶有加減速與旋轉的運動（小幅度前後左右移動並轉動），系統才能順利完成初始化。這與第 21 章「純旋轉無法三角測量」並列為視覺慣性 SLAM 兩大實作陷阱——同學在第 30 章實作時務必記得。

量一量：為什麼非得晃一晃不可

23.4 節說「若載體只是等速直線移動，系統無法把尺度與零偏區分開來（數學上不可觀測）」。這句話很重要，但「不可觀測」聽起來像個抽象的名詞。本小節把它變成一個可以算出來的數字。

做法是把初始化的第二階段（固定視覺軌跡、純慣性估計）寫成一個線性方程組。未知數是：每個關鍵幀的速度、重力向量、尺度因子、以及加速度計零偏。方程式則來自相鄰關鍵幀之間的預積分關係——位置式與速度式各一組。把這個方程組的係數矩陣做奇異值分解，最大與最小奇異值的比值（條件數）就直接告訴我們問題好不好解。



圖 23-2　運動激勵決定了初始化能不能解

（a）畫出四秒內的三種運動：等速直線（紅）、輕微晃動（橘，橫向擺動約 3 公分）、以及充分激勵（綠，含前後加減速、橫向與上下移動、加上偏航與俯仰的轉動）。請注意這三條軌跡在視覺上都「有在動」——光看軌跡，很難預料它們的初始化結果會差這麼多。

（b）是結果，縱軸是對數刻度。等速直線的尺度誤差是 100.3%——不是「不夠準」，是完全解不出來；實際解出的尺度甚至可能是負值。條件數高達 3,820。輕微晃動把尺度誤差拉到 1.2%，充分激勵則是 0.2%，條件數降到 157，比等速直線小了二十四倍。

為什麼等速直線會完全失敗？道理其實很直白。加速度計量到的是比力，也就是「真實加速度減去重力」在機體座標系下的投影。等速直線運動時真實加速度是零，而姿態又固定不變，所以加速度計讀到的永遠是同一個常數向量。這個常數可以解釋成「重力的投影」，也可以

解釋成「零偏」，兩種解釋在數學上完全等價——方程組根本沒有足夠的資訊把它們分開。

這裡還有一個值得注意的細節：輕微晃動雖然把尺度救回來了（1.2%），但它的條件數仍有 3,559，幾乎與等速直線一樣高，而且加速度計零偏的誤差反而是三者中最大的。換句話說，稍微晃一下足以讓尺度變得可觀測，卻不足以把零偏分離出來。這正好呼應圖 23-1 的結論——零偏估不準，IMU 就撐不過視覺空隙。

（c）回答另一個問題：初始化要花多久。固定「充分激勵」的運動，把時間窗從 0.5 秒掃到 10 秒。0.5 秒時尺度誤差是 15.6%，0.8 秒降到 0.9%，2 秒時是 0.7%，之後曲線明顯轉平——10 秒才降到 0.06%。也就是說，兩秒左右是報酬遞減的轉折點：再等下去精度改善有限，使用者的耐心卻是線性消耗的。這就是 23.4 節引述「2 秒內達到 5% 尺度誤差」之所以被視為重要突破的原因。

本實驗與真實系統的差距

本實驗為了能完整重現，做了幾個簡化：沒有陀螺儀零偏、視覺軌跡的雜訊是獨立同分布、預積分對零偏的敏感度只取一階近似。因此這裡的絕對數字（例如 2 秒時 0.7%）比真實系統樂觀。

真實系統在 2 秒時約為 5%，再經第三階段的聯合視覺慣性 BA 精修到約 1%。但三個定性結論完全一致：等速直線不可觀測、晃一下就能救回尺度、兩秒之後報酬遞減。

同學若要在自己的裝置上重做這個實驗，最有價值的產出不是絕對數字，而是「你的裝置需要晃多大、晃多久」——這會直接寫進你的操作手冊。

23.5 Multi-Map 多地圖系統（Atlas）

這是 ORB-SLAM3 的第二項核心創新，也是最能體現「工程智慧」的設計。

Atlas 的組成

Atlas 是由一組離散地圖構成的多子地圖系統。它維護一張活躍地圖，供追蹤執行緒定位當前幀，而局部建圖執行緒持續以新的關鍵幀最佳化並更新該地圖；Atlas 中的其他地圖則為非活躍地圖。系統並建立一個以詞袋模型為基礎的關鍵幀資訊資料庫，用於重定位、閉環偵測與地圖合併。

- 活躍地圖（Active Map）：正在使用的那一張，追蹤與局部建圖都在它上面運作。
- 非活躍地圖（Inactive Map）：曾經建立、目前未使用，但完整保留在記憶體中等待日後合併。
- 共用的詞袋資料庫：所有地圖的關鍵幀都登錄在同一個資料庫中——這是能跨地圖比對的關鍵。

跟丟時發生什麼

這正是 Atlas 存在的理由。回顧 ORB-SLAM2 的處境：嚴重跟丟且重定位失敗時，系統就停在那裡，之後的資料全部浪費。

ORB-SLAM3 的做法是：跟丟後若短時間內無法重定位，就把當前地圖設為非活躍、開啟一張全新的地圖，從零開始繼續建圖與追蹤。使用者幾乎不會察覺中斷。而當機器人日後走回舊地圖涵蓋的區域，地點辨識會偵測到跨地圖的匹配，系統隨即把兩張地圖合併，恢復成一張完整一致的地圖。

與其他多地圖方案的差異

值得注意的是，也有其他系統提出 ORB-SLAM2 的多地圖擴充，但它們把子地圖保留為分離的實體；ORB-SLAM3 則執行無縫的地圖合併，建立更精確的全域地圖。

這個差異很關鍵。若只是把地圖各自存著不合併，雖然資料沒丟，但你會得到好幾張互不相干的地圖，無法規劃跨區域的路徑，精度也無法互相提升。無縫合併才能真正發揮多地圖的價值。

多階段作業的價值

Atlas 的另一個重要應用是跨執行階段的重用：今天建好的地圖存起來，明天啟動時載入，系統能直接在舊地圖中重定位並繼續擴充。這對每天在同一場域運行的服務機器人、巡檢機器人極為實用，也解釋了 23.1 節提到「多階段精度優勢更大」的現象。

23.6 失去追蹤後的重定位與地圖融合

本節說明 Atlas 運作的技術細節：如何偵測跨地圖的關聯，以及如何合併。

地點辨識：高召回率的設計

每當建圖執行緒建立一個新的關鍵幀，地點辨識就會啟動，嘗試偵測與 Atlas 中任何既有關鍵幀的匹配。若找到的匹配關鍵幀屬於活躍地圖，就執行閉環；否則就是多地圖資料關聯，此時活躍地圖與匹配地圖會被合併。

這是一個優雅的統一設計：閉環與地圖合併，其實是同一個機制的兩種結果——差別只在於匹配到的關鍵幀屬於同一張地圖還是不同地圖。

精確率與召回率的取捨

地點辨識面臨一個經典的兩難：

- 召回率要高：必須盡量找到所有真實的匹配，否則會錯過閉環與合併的機會。
- 精確率必須是百分之百：一個錯誤的匹配會把地圖硬拉到錯誤位置，造成災難（第 12、15、22 章反覆強調）。

ORB-SLAM3 的做法是：為了達到高召回率，對每個新的活躍關鍵幀，向 DBoW2 資料庫查詢 Atlas 中數個相似的關鍵幀；而為了達到百分之百的精確率，每一個候選都要通過數個步驟的幾何驗證。

幾何驗證透過直接的 Sim(3) 或 SE(3) 對齊執行（使用 RANSAC 與 Horn 方法）。這裡再次看到第 21 章 RANSAC 的身影，以及第 22 章單目需用相似變換（含尺度）的道理。

地圖合併的兩階段策略

確認跨地圖關聯後，合併就要開始了。但這裡有一個工程難題：非活躍地圖可能很大，若一次全部合併，運算會阻塞系統。

ORB-SLAM3 的解法是把合併拆成兩個步驟：首先，在由匹配關鍵幀與其鄰居所定義的焊接視窗中執行合併；第二階段，再透過位姿圖優化把修正傳播到合併後地圖的其餘部分。

圖 23-3　地圖合併的兩階段策略

偵測到跨地圖關聯：活躍地圖 Ma 的關鍵幀 Ka ↔ 非活躍地圖 Mm 的關鍵幀 Km

↓ 估出對齊變換 Tam

【第一階段：焊接視窗】

在 Ka 與 Km 的鄰居所構成的局部視窗中執行合併

在此視窗中密集搜尋中期資料關聯，提升精度

→ 局部立即可用，追蹤執行緒不中斷

↓

【第二階段：傳播修正】

以位姿圖優化把修正傳播到合併後地圖的其餘部分

→ 全域一致性恢復

這個「先局部焊接、再全域傳播」的策略，讓合併能即時進行而不阻塞追蹤。同學應該注意到，這與第 15 章 Cartographer「局部即時、全域背景」、第 18 章 LOAM「高頻里程計、低頻建圖」的架構思想完全一致——這是機器人系統設計的普遍模式。

一個細節：中期關聯的強化

作者提出的第二項創新是：一旦估出新關鍵幀與匹配地圖之間的相對位姿，便以匹配關鍵幀及其在共視圖中的鄰居定義一個局部視窗，並在此視窗中密集搜尋中期資料關聯，以提

升精度。

這再次呼應 23.2 節的主軸：ORB-SLAM3 的精度優勢，很大程度來自它在每一個環節都盡可能運用中期資料關聯。

量一量：跟丟之後的三種命運

23.5 與 23.6 節描述了 Atlas 的運作。本小節用一個模擬把它的價值量化：同一條軌跡、同一次跟丟，三種系統設計各自會得到什麼。

設定是機器人在室內繞兩圈共 120 個關鍵幀，在第 66 至 78 幀之間視覺完全失效（例如快速轉動或面對白牆）。三種情況分別是：ORB-SLAM2 的做法（重定位失敗後停住）、開新地圖但不合併、以及 Atlas 的無縫合併。最後用兩個指標比較：涵蓋了多少條軌跡，以及最終地圖的均方根誤差。

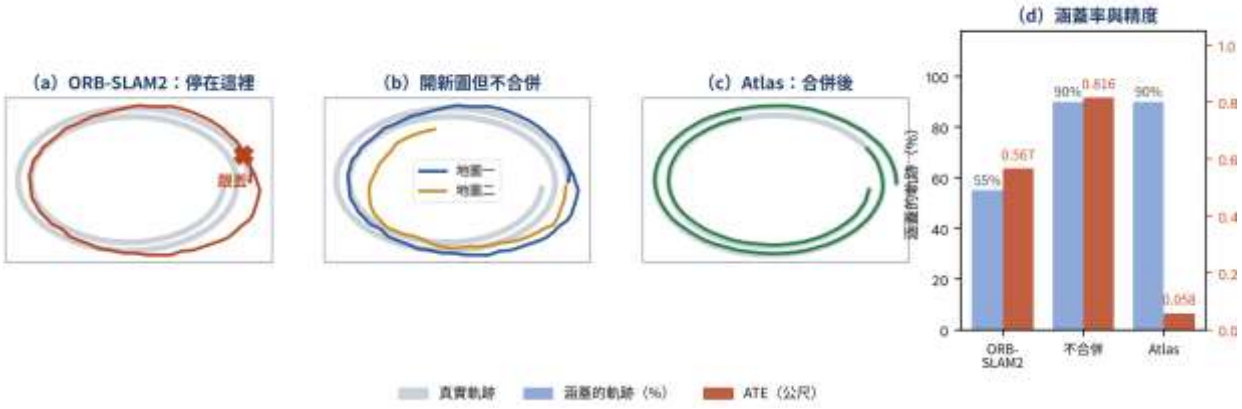


圖 23-4　跟丟之後的三種命運

（a）是 ORB-SLAM2 的處境。系統走到第 66 幀跟丟，重定位失敗後就停在那裡，只涵蓋了 55% 的軌跡，之後的資料全部浪費。已建好的部分 ATE 是 0.567 公尺——注意這個誤差不算小，因為它沒有機會靠第二圈的閉環來修正。

（b）是「開新地圖但不合併」。涵蓋率提升到 90%，資料確實沒丟。但兩張地圖各自有自己的原點，把它們放在同一個座標系下看，整體 ATE 是 0.816 公尺——比只建一半還糟。圖上藍色與橘色兩段明顯錯開，這正是 23.5 節說「無法規劃跨區域路徑」的視覺呈現。

（c）是 Atlas。機器人走回第一張地圖涵蓋的區域時，地點辨識偵測到跨地圖的匹配，系統把兩張圖合併成一張。涵蓋率同樣是 90%，但 ATE 降到 0.058 公尺——是不合併的十四分之一，也比只建一半的 ORB-SLAM2 好了十倍。圖上可以看到合併後的綠色軌跡與真實軌跡幾乎重合，只在跟丟的那一段留下缺口。

（d）把兩個指標並排。這張圖要傳達的重點是：Atlas 的價值不只是「資料沒丟」，更是「資料能互相校正」。第二張地圖之所以能被拉回正確位置，靠的是它與第一張地圖之間的跨地圖共視關係——這與第 22 章圖 22-2 的結論完全一致：誤差要靠圖的連通性才能重新分配。多地圖如果不合併，就只是把資料存起來而已。

> **為什麼「合併」比「保留」難得多**
>
> 把跟丟後的資料另存成一張新地圖，工程上很容易——開一個新容器就好。真正困難的是合併：兩張地圖的座標系不同、尺度可能不同（單目）、而且合併時不能卡住正在運行的追蹤執行緒。
>
> 這就是 23.6 節那個兩階段策略存在的理由：先在焊接視窗內做局部融合（範圍小、可即時完成），再把修正量沿著位姿圖往外傳播（範圍大、放背景做）。同樣是「先快速拉正、再精細最佳化」——這個模式在本書已經出現第四次了。

23.7 GoPro、手機、無人機上的視覺慣性 SLAM

視覺慣性 SLAM 已不只是研究題目，它就在你我的日常裝置中運作。本節說明幾個代表性應用。

手機與 AR / VR

第 20.7 節提過 AR 的「由內而外追蹤」，其技術本質正是視覺慣性 SLAM。手機的相機加上內建 IMU（第 7 章），構成完美的視覺慣性組合：相機提供不漂移的環境參考、IMU 提供高頻運動與快速轉頭時的穩健度。

ORB-SLAM3 論文中一個特別有說服力的數據，正是針對這類應用。在魚眼的 TUM-VI 基準測試上，它整體表現優於 VINS-Mono 與 Basalt；在類似 AR / VR 的房間序列中，雙目慣性模式的平均絕對軌跡誤差為 0.009 公尺，單目慣性模式為 0.011 公尺。

毫米級的精度——這正是把虛擬物件穩定「釘」在真實桌面上所需要的水準。同學可以想想：你手機裡那個能量測桌子長度的 AR 尺，背後就是這個等級的技術。

穿戴式相機與運動攝影

運動攝影機（如 GoPro）配戴在頭盔或胸前，在騎車、跑步、攀岩時錄影。這類影片對 SLAM 極具挑戰：劇烈震動、快速轉向、動態模糊、強烈的光線變化（進出樹蔭、隧道）。

純視覺方法在這類序列上幾乎必然跟丟，而視覺慣性 SLAM 因為有 IMU 在視覺失效時撐住數秒（第 7.6 節的第三項貢獻），得以維持連續軌跡。這類應用的典型用途包括：運動路線的三維重建、極限運動的軌跡分析、以及第五篇提過的背包式空間測繪。

無人機

無人機是視覺慣性 SLAM 最重要的應用之一，也是第 26 章的主題。它的特殊需求包括：

- 重量與功耗極度受限：相機加 IMU 是唯一可行的組合（第 9.5 節）。
- 六自由度快速運動：純視覺難以應付，IMU 不可或缺。
- GPS 拒止環境：室內、橋下、隧道中，視覺慣性是唯一的定位手段（第 5 章案例 5-3）。
- 即時性要求嚴苛：定位延遲直接影響飛行穩定與安全。

第 7 章案例 7-3 提過的 EuRoC 資料集，正是以微型飛行器錄製，而 ORB-SLAM3 在其上的優異表現（第 7.7 節的 3.5 公分精度），說明了它在無人機應用上的實用性。

機器人與校園場域

對本書讀者而言，幾個具體的專題方向值得考慮：

- 視覺導航機器人：以雙目慣性 ORB-SLAM3 建立校園地圖，再結合第 24 章的視覺導航與第 32 章的語意理解。
- 多階段建圖：利用 Atlas 的多階段能力，分多次建立整棟系館的地圖，驗證地圖合併的效果。
- 跟丟恢復實驗：設計一個「刻意讓系統跟丟」的實驗流程，量化 Atlas 相對 ORB-SLAM2 在恢復能力上的優勢——這是一個很好的比較型研究題目。
- 與光達 SLAM 對照：在同一場域分別執行 ORB-SLAM3（視覺慣性）與第 18 章的 LIO 系統（光達慣性），比較兩者在不同環境（明亮走廊、昏暗機房、玻璃帷幕大廳）的表現。這個對照能讓同學深刻體會第 8、9 章反覆強調的感測器互補性。

至此，第六篇的核心系統已完整介紹。下一章我們把視野從「定位建圖」擴展到「導航與應用」——視覺導航機器人如何利用這些能力執行任務？線追蹤、視覺伺服、AprilTag 導航、YOLO 語意導航、乃至於 VLM 輔助導航，將把本書的技術與真實的機器人任務串接起來。

23.8 視覺慣性之後：2024–2026 年的兩個方向

ORB-SLAM3 發表於 2021 年，至今仍是視覺慣性 SLAM 的標準比較對象。這幾年的進展主要往兩個方向走，而且方向幾乎相反。

一、往下：擠進毫瓦級的晶片

第一個方向是把視覺慣性里程計縮小到極限。2026 年的一項工作把完整的單目視覺慣性管線（ORB 特徵、八點法與 EPnP 搭配 RANSAC、IMU 預積分、滑動視窗的位姿圖優化）部署到一顆 RISC-V 的超低功耗晶片上，時脈只有 370 MHz。

成績是每秒 20 幀、功耗低於 100 毫瓦、記憶體用量不到 1 MB（第一層約 110 KB、第二層約 850 KB）。把這幾個數字放在一起看相當驚人——一套完整的視覺慣性 SLAM，跑在比手機弱兩個數量級的晶片上。

但代價也很誠實。該系統省掉了閉環偵測，在標準的 EuRoC 資料集上平均軌跡誤差約 3.46 公尺，而 ORB-SLAM3 是 0.25 公尺左右——差了一個數量級。作者自己指出，原因正是缺少全域修正。這組對照很適合放在本章末尾：它說明了閉環與全域最佳化不是錦上添花，而是長時間一致性的唯一來源。第 25 章的毫瓦級平台會再回到這個主題。

二、往上：讓學習式模組接手前端

第二個方向是把前端的某些環節交給神經網路：用學習式的特徵與匹配取代 ORB、用學習式的光流一致性檢驗取代純幾何的外點剔除、或在光線劇變的場景中加入學習式的線特徵。

這些工作有一個共同點值得同學注意：它們換掉的幾乎都是「對應怎麼找」這一層，而 IMU 預積分、緊耦合的最佳化、關鍵幀與地圖管理——也就是本章 23.3 到 23.6 節的全部內容——大多原封不動。這與第 22 章 22.8 節觀察到的現象完全相同。

	ORB-SLAM3（本章）	毫瓦級 VIO	學習式前端 VIO
前端	ORB + RANSAC	ORB + RANSAC（精簡）	學習式特徵 / 光流
IMU 融合	緊耦合，含零偏估計	緊耦合（滑動視窗）	緊耦合
閉環與多地圖	有	無	多半保留
典型硬體	桌機或 Jetson	RISC-V，370 MHz	需要 GPU
功耗	數瓦至數十瓦	低於 100 毫瓦	數十瓦
EuRoC 平均誤差	約 0.25 公尺	約 3.46 公尺	與 ORB-SLAM3 相當
適用	一般機器人與 AR	微型無人機、穿戴式	光線劇變、弱紋理場景

表 23-2　三條路線的比較

從這張表可以讀出一個對專題選題很實用的判斷：如果你的載體有數瓦的功耗預算，ORB-

SLAM3 仍然是最省事也最可靠的選擇；如果你做的是克級的微型平台，那麼你需要的是第 25 章的技術，並且要接受沒有閉環的後果；如果你的場域光線條件惡劣，學習式前端值得一試，但系統架構仍是本章這一套。

公式或流程

一、視覺慣性 SLAM 的狀態向量

視覺慣性模式下，每個關鍵幀 i 需估計的狀態為：

$$S_i = \{ R_i, p_i, v_i, b_g_i, b_a_i \}$$

其中 R_i 為旋轉、p_i 為位置（合計 6 自由度）、v_i 為速度（3）、b_g 與 b_a 為陀螺儀與加速度計零偏（各 3），共 15 個自由度——相較純視覺的 6 個自由度多出速度與零偏。單目慣性模式在初始化階段還額外要估尺度 s 與重力方向 R_wg。

二、緊耦合的聯合最佳化目標

ORB-SLAM3 的視覺慣性束調整，同時最小化視覺與慣性兩類誤差：

$$\min \Sigma \rho (\| e_視覺 \|^2_\Sigma) + \Sigma \| e_慣性 \|^2_\Sigma + \| e_先驗 \|^2$$

- 視覺誤差 e_視覺：重投影誤差（第 21、22 章），要求地圖點投影位置與實際觀測相符。
- 慣性誤差 e_慣性：IMU 預積分殘差（第 7.5 節），要求相鄰關鍵幀的位姿、速度變化與 IMU 量測一致。
- 先驗誤差 e_先驗：保留先前最佳化累積的資訊。

ρ 為穩健核函數。這條式子正是第 7.6 節「緊耦合」的具體形式，也是第 12 章最小平方框架的又一次應用——同學至此應能看出，整本書的數學是高度統一的。

三、IMU 初始化的三階段

圖 23-5 IMU 初始化流程

① 純視覺 MAP 估測

只用相機建立短時間的地圖與軌跡（單目尺度未知）

↓ 提供運動基準

② 純慣性 MAP 估測（固定視覺軌跡）

估計：尺度 s、重力方向、陀螺儀零偏、加速度計零偏、各幀速度

↓ 取得慣性參數初值

③ 聯合視覺慣性束調整

所有變數（位姿、速度、零偏、尺度、重力、地圖點）一起精修

↓

完成：2 秒內尺度誤差約 5%，精修後約 1%

【前提】必須有足夠的運動激勵，否則尺度與零偏不可觀測

四、閉環與地圖合併的統一流程

圖 23-6 地點辨識觸發的兩種結果

建圖執行緒建立新關鍵幀

↓

地點辨識：向 DBoW2 資料庫查詢 Atlas 中的相似關鍵幀（高召回率）

↓

幾何驗證：多步驟驗證 + Sim（3）/ SE(3) 對齊（RANSAC + Horn 方法）

↓ （追求 100% 精確率）

匹配幀屬於哪張地圖？

├─ 活躍地圖 → 【閉環】位姿圖優化 + 全域 BA（第 22 章）

└─ 其他地圖 → 【地圖合併】焊接視窗融合 + 位姿圖傳播修正

案例

案例 23-1 跟丟不再是終點：Atlas 的實測體驗

這是對比 ORB-SLAM2 與 ORB-SLAM3 最有戲劇性的實驗，強烈建議在課堂上並排演示。

實驗設計很簡單：讓兩套系統執行同一段影片，而這段影片中間有一段嚴重的視覺失效——例如相機被完全遮住十秒、或快速甩過一大片白牆（第 9 章的白牆詛咒）。

ORB-SLAM2 的表現：跟丟後嘗試重定位，但因為離開原區域太遠而失敗，系統就此停擺。畫面凍結，之後的影片內容完全沒有被利用——即使後面有精彩的場景，也全部浪費。

ORB-SLAM3 的表現：跟丟後短暫嘗試重定位，失敗後隨即開啟一張新地圖，繼續正常建圖與追蹤，使用者幾乎察覺不到中斷。而當相機日後繞回舊區域，地點辨識偵測到跨地圖關聯，兩張地圖無縫合併——螢幕上會看到「兩段原本分離的軌跡忽然接在一起」的震撼

畫面。

這個對比讓同學直觀理解「穩健度」在真實系統中的價值。實驗室裡的乾淨資料集上，兩者精度差距或許只是數字；但在真實世界的挑戰性場景中，差別是「能不能用」。這也呼應第 15 章 Cartographer 的觀察：產品化的系統，穩健度往往比極致精度更重要。

案例 23-2　兩秒鐘的價值：初始化速度為何是關鍵

IMU 初始化從十五秒縮短到兩秒，聽起來只是「快了一點」。但這十三秒的差距，在實務上是決定性的。

想像一位使用者打開手機的 AR 應用。若需要拿著手機晃動十五秒才能開始使用，絕大多數人會在第五秒就關掉 App——這個產品根本無法上市。而兩秒鐘的初始化，使用者通常還在調整握姿，幾乎無感。

對機器人也一樣。一台無人機起飛前若要在地上晃十五秒，不僅浪費時間，在某些場景（例如緊急救援）根本不可行；而兩秒的初始化，可以在起飛滑行的過程中順帶完成。

更深一層看，快速初始化還帶來一個間接好處：當系統跟丟並需要在新地圖上重新初始化時（23.5 節），快速初始化意味著中斷時間極短。若初始化要十五秒，Atlas 的「無縫切換」就名不副實了。這說明了系統設計中各項技術的相互支撐——初始化速度、多地圖架構、資料關聯，三者共同構成了 ORB-SLAM3 的穩健度。

對同學的啟示是：評估一項技術改進時，不要只看單一指標的數字變化，要問「這個改變讓什麼原本不可能的事變成可能？」——這才是技術價值的真正衡量。

案例 23-3　從 PTAM 到 ORB-SLAM3：一條血脈的二十年

ORB-SLAM3 不是憑空出現的，它是特徵法視覺 SLAM 這條技術血脈二十年演進的結晶。它是自 PTAM 與 ORB-SLAM 開始的特徵法 SLAM 血脈的集大成之作，發布時是當時最完整、最精確的開源 SLAM 函式庫。

- PTAM(2007)：首創「追蹤與建圖分離為兩個執行緒」的架構，讓即時視覺 SLAM 成為可能。這個雙執行緒思想，是後續所有系統的基礎。
- ORB-SLAM(2015)：以 ORB 特徵統一追蹤、建圖與地點辨識三項任務（第 22.1 節），加入閉環執行緒，並引入共視圖與本質圖（第 22.6 節）。
- ORB-SLAM2(2017)：擴展到雙目與 RGB-D，成為最廣泛使用的開源視覺 SLAM（第 22 章）。
- ORB-SLAM3(2021)：加入緊耦合視覺慣性與 Atlas 多地圖，並全面運用四種資料關聯。

觀察這條血脈，同學能學到一個重要的研究方法論：每一代都不是推翻前代，而是保留有效的架構、針對明確的弱點做突破。PTAM 的雙執行緒、ORB-SLAM 的共視圖與詞袋、ORB-SLAM2 的多相機支援——這些設計在 ORB-SLAM3 中全都還在。

這也給有志研究的同學一個務實的建議：與其追求「發明全新的東西」，不如深入理解一個成熟系統，找出它明確的弱點，提出針對性的改良。ORB-SLAM3 之所以能提出 Atlas，正是因為作者深知 ORB-SLAM2「跟丟即結束」的痛點。理解限制，才能突破限制。

實作活動

實作 23-1　ORB-SLAM3 視覺慣性 SLAM 實作

目的：執行 ORB-SLAM3 的多種模式，親身體驗 IMU 帶來的差異、Atlas 的跟丟恢復能力，並完成與 ORB-SLAM2 的對照實驗。這是第六篇的總結實作，也是第 30、31 章的重要基礎。

第一部分：環境建置與基本執行（全體）

43. 編譯 ORB-SLAM3（相依套件與第 22 章類似：OpenCV、Eigen3、Pangolin；注意版本相容性問題）。
44. 下載 EuRoC 資料集的一個序列（建議從 MH_01_easy 開始），先以純雙目模式執行，記錄軌跡與地圖。

45. 同一序列改以雙目慣性模式執行，比較兩者的軌跡、地圖點數量與系統穩定度。

46. 觀察初始化：單目慣性模式下，記錄從啟動到 IMU 初始化完成所需的時間與影像幀數，對照 23.4 節的「兩秒」數據。

第二部分：IMU 的價值（全體或大學部）

47. 挑戰序列：選一段快速運動的困難序列（如 EuRoC 的 V1_03_difficult 或 MH_05_difficult），分別以純視覺與視覺慣性模式執行，比較是否跟丟、跟丟幾次。

48. 尺度驗證：單目模式與單目慣性模式各執行一次，用 evo 評測時，前者需做尺度對齊、後者不需要——親身驗證 IMU 提供絕對尺度的價值（第 20.5 節）。

49. 運動激勵實驗：若使用自己的相機與 IMU，分別嘗試「靜止不動」「等速直線移動」「帶加減速與旋轉的移動」三種啟動方式，觀察哪一種能成功初始化，驗證 23.4 節的論述。

第三部分：Atlas 多地圖實驗（核心，大學部）

50. 人為製造跟丟：執行資料集時，以程式暫時中斷影像輸入數秒（或選用含嚴重遮擋的序列），觀察系統是否開啟新地圖。

51. 觀察 Atlas 狀態：在終端輸出或視覺化中，確認當前有幾張地圖、哪一張是活躍地圖。

52. 地圖合併：讓相機回到舊地圖涵蓋的區域，觀察合併發生的瞬間，並錄影記錄——這是本章最值得展示的畫面。

53. 對照實驗：對同一段含嚴重跟丟的序列，分別執行 ORB-SLAM2 與 ORB-SLAM3，比較最終軌跡的完整度，重現案例 23-1。

54. 多階段實驗：利用 EuRoC 同一場景的多個序列（如 MH_01 至 MH_05），依序執行並讓系統合併，觀察多階段作業如何逐步擴充地圖。

討論問題

16. 純雙目與雙目慣性的精度（ATE）差多少？在哪一種序列上差距最大？為什麼？
17. 困難序列中，純視覺跟丟了幾次？加入 IMU 後改善多少？對照第 7.6 節，IMU 的哪一項貢獻在此發揮了作用？
18. IMU 初始化在你的實驗中花了多久？若初始化時載體靜止不動，會發生什麼？這印證了 23.4 節的哪一項論述？
19. 地圖合併發生時，軌跡與地圖有什麼變化？這與第 22 章的閉環修正有何異同？（提示：23.6 節的統一流程）
20. （大學部）依 23.2 節的四種資料關聯，說明 ORB-SLAM3 相對 VINS-Mono 精度優勢的主要來源是哪一種關聯？為什麼它如此重要？

延伸挑戰（大學部）

其一：校園場域實測——用自備的雙目慣性相機（或含 IMU 的深度相機）錄製系館走廊與大廳的資料，執行 ORB-SLAM3 建圖。刻意設計一條「經過昏暗機房造成跟丟、再回到明亮走廊」的路線，測試 Atlas 的合併能力，並以第 13.7 節的方法評估地圖品質。其二：跨模態比較——在同一場域同時錄製視覺慣性與光達慣性資料，分別執行 ORB-SLAM3 與第 18 章的 LIO 系統，在明亮走廊、昏暗機房、玻璃帷幕大廳三種環境下比較兩者表現，量化驗證第 8、9 章的感測器互補論述。這個題目兼具教學與研究價值，適合作為專題主軸。

延伸閱讀

22. C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM," IEEE T-RO, 2021.（本章主要依據）
23. R. Elvira, J. D. Tardós, and J. M. M. Montiel, "ORBSLAM-Atlas: A Robust and Accurate Multi-Map System," IROS, 2019.（Atlas 多地圖架構的前導工作）

24. C. Campos, J. M. M. Montiel, and J. D. Tardós, "Inertial-Only Optimization for Visual-Inertial Initialization," ICRA, 2020.（IMU 初始化三階段流程的理論來源）

25. C. Forster et al., "On-Manifold Preintegration for Real-Time Visual-Inertial Odometry," IEEE T-RO, 2017.（IMU 預積分，第 7.5 節的理論基礎）

26. ORB-SLAM3 開源專案（GitHub: UZ-SLAMLab/ORB_SLAM3）.（原始程式碼、編譯說明與各模式執行範例）

27. T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," IEEE T-RO, 2018.（另一套經典視覺慣性系統，可與本章對照）

28. EuRoC MAV 與 TUM-VI 資料集官方網站.（本章實驗與論文評測所用的標準基準）

小結

22. ORB-SLAM3 有三項核心突破：以最大後驗估測為基礎的緊耦合視覺慣性 SLAM、Atlas 多地圖系統、以及在所有演算法階段全面運用四種資料關聯；支援單目 / 雙目 / RGB-D 與針孔 / 魚眼鏡頭的多種組態。

23. 四種資料關聯為短期（最近數秒）、中期（附近有輕微漂移的元素）、長期（以地點辨識匹配先前造訪區域）與多地圖（跨建圖階段）；中期關聯是精度優勢的關鍵來源，使其在 EuRoC 單目慣性單階段運作中精度達 VINS-Mono 的 2.6 倍。

24. 視覺慣性模式下狀態向量從純視覺的 6 自由度擴充為 15 自由度（加上速度與兩組零偏）；IMU 帶來絕對尺度、重力方向、視覺失效時的支撐與高頻輸出，而視覺回報以不漂移的觀測校正 IMU 零偏。

25. IMU 初始化採三階段：純視覺 MAP 估測 → 固定視覺軌跡的純慣性 MAP 估測 → 聯合視覺慣性 BA;2 秒內達 5% 尺度誤差、精修後約 1%，但必須有足夠的運動激勵，否則尺度與零偏不可觀測。

26. Atlas 由一張活躍地圖與多張非活躍地圖組成，共用以詞袋為基礎的關鍵幀資料庫；跟丟時開啟新地圖繼續運作，日後偵測到跨地圖關聯時執行無縫合併——不同於其他方案把子地圖保留為分離實體。

27. 地點辨識對每個新關鍵幀查詢 Atlas 中的相似幀以求高召回率，再以多步驟幾何驗證與 Sim（3）/ SE(3) 對齊確保百分之百精確率；匹配屬活躍地圖則閉環，否則合併。合併分焊接視窗融合與位姿圖傳播兩階段，以免阻塞追蹤。

28. 視覺慣性 SLAM 廣泛應用於手機 AR、穿戴式相機與無人機；在類似 AR / VR 的 TUM-VI 房間序列中，雙目慣性平均 ATE 為 0.009 公尺、單目慣性為 0.011 公尺，達到毫米級精度。

承先啟後

本章把第 7 章的 IMU 與第 22 章的系統架構接了起來，也量出了三組數字：IMU 要撐過視覺空隙，關鍵在零偏——零偏估到 ±0.01 時兩秒的推算誤差是 0.057 公尺，估到 ±0.3 時是 0.88 公尺，差了十五倍；等速直線運動下初始化的條件數高達 3,820、尺度誤差 100%，充分激勵則把條件數降到 157、誤差降到 0.2%；而跟丟之後，Atlas 的合併把 ATE 從不合併的 0.816 公尺壓到 0.058 公尺，同時涵蓋 90% 的軌跡。

第六篇的核心系統至此完整介紹。但 SLAM 本身不是目的——機器人建了地圖、知道自己在哪之後，要拿它來做什麼？下一章我們把視野從「定位建圖」擴展到「導航與任務」：視覺導航與 SLAM 有何不同？線追蹤、視覺伺服、人工標記導航如何運作？物件偵測與視覺語言模型如何讓機器人理解「請去教務處」這樣的指令？機器狗與校園巡檢機器人如何整合這一切？第 24 章將把本書的技術與真實的機器人任務串接起來。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 24 章

視覺導航機器人

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

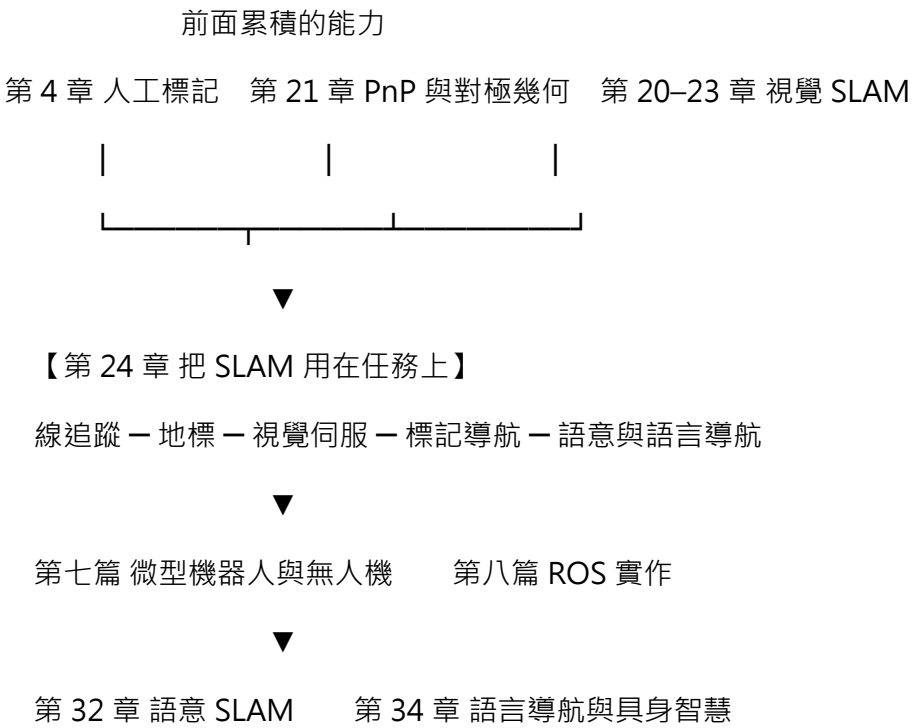
本章在全書的位置

第 20 到 23 章把視覺 SLAM 從特徵一路建到完整系統。本章問的是下一個問題：機器人知道自己在哪之後，要拿它來做什麼。

這一章的性格與前四章很不一樣。前四章在追求精度與穩健，本章則在談取捨——很多導航任務根本不需要 SLAM。貼一條線、貼幾個標記、或者直接對著目標做視覺伺服，在對的場合比完整的 SLAM 更便宜也更可靠。工程判斷的重點不是「用上最強的技術」，而是「用剛好夠的技術」。

本章的三張圖都在量化這些取捨：圖 24-1 比較兩種視覺伺服的收斂行為，說明為什麼精密對接要挑對方法；圖 24-2 量出標記的位姿精度如何隨距離與視角變化；圖 24-3 則回答一個很實際的問題——標記到底該多久貼一個。

圖 24-0　第 24 章的連結地圖



學習目標

前四章我們建立了完整的視覺 SLAM 能力：機器人能知道自己在哪、環境長什麼樣。但

SLAM 本身不是目的——建了地圖之後，要拿它來做什麼？本章是第六篇的最後一章，把技術與真實的機器人任務串接起來，從最樸素的線追蹤，一路走到能聽懂人話的 VLM 導航。學完本章之後，同學應能夠達成下列目標：

29. 說明視覺導航與視覺 SLAM 的差異與互補關係。
30. 比較線追蹤、地標導航與視覺伺服三種基本視覺導航策略。
31. 說明 AprilTag / ArUco 在機器人導航中的四種典型用途。
32. 說明 YOLO 物件辨識如何支援語意導航，並理解其在機器人視角下的限制。
33. 說明 VLM 視覺語言模型如何讓機器人理解自然語言導航指令。
34. 說明機器狗視覺導航的特殊挑戰與對策。
35. 規劃一個校園巡檢或資源回收辨識機器人的系統架構。

核心概念

中文名詞	英文名詞	概念說明
視覺導航	Visual Navigation	以視覺資訊引導機器人移動到目標的技術。
線追蹤	Line Following	沿地面標線行進的最基本視覺導航方式。
視覺伺服	Visual Servoing	以影像回授直接控制機器人運動的方法。
影像式視覺伺服	IBVS	直接以影像特徵誤差作為控制訊號。
位置式視覺伺服	PBVS	先解出三維位姿誤差再控制。
地標導航	Landmark Navigation	依據可辨識的地標決定行進方向。
語意導航	Semantic Navigation	依物體與場景意義而非幾何座標導航。
物件目標導航	Object-Goal Navigation	接受「去找某個物體」的任務指令。
視覺語言導航	VLN	依自然語言指令在環境中導航。

中文名詞	英文名詞	概念說明
視覺語言模型	VLM	能同時理解影像與語言的多模態 AI 模型。
視覺語言動作模型	VLA	直接由影像與語言產生動作指令的模型。
邊界探索	Frontier Exploration	朝已知與未知交界處前進以探索環境。
零樣本	Zero-Shot	未經任務特定訓練即可執行新任務的能力。

24.1 視覺導航與 SLAM 的差異

這是本章第一個必須釐清的觀念。許多同學把「視覺 SLAM」與「視覺導航」混為一談，但兩者回答的是不同的問題。

兩個不同的問題

84. 視覺 SLAM 回答的是：「我在哪裡？環境長什麼樣？」它產出的是位姿與地圖——這是第 20 至 23 章的主題。

85. 視覺導航回答的是：「我該往哪走？」它產出的是運動指令——這是本章的主題。

用第 3 章的導航三問來對照：SLAM 專門解決第一問（我在哪裡），而視覺導航則涵蓋第二、三問（我要去哪裡、我該怎麼去）。

視覺導航不一定需要 SLAM

這是一個重要但常被忽略的事實。有些視覺導航方法完全不建地圖：

86. 線追蹤機器人只看地上的線，不知道自己在哪，也不需要知道（24.2 節）。

87. 視覺伺服直接把「影像中目標的偏移」轉成馬達指令，中間不經過任何地圖。

88. 依循視覺地標的走廊機器人，只需認得「看到滅火器就右轉」，不需全域座標。

這些方法屬於第 3 章介紹的反應式架構——感知直接驅動行為，不建模、不規劃。它們簡單、快速、穩健，在受控環境中非常實用。

SLAM 讓導航能力升級

但反應式導航有明顯的天花板：無法規劃全域路徑、無法處理「去一個沒有標線通往的地方」、無法在中途改變目標。加上 SLAM 之後，機器人才具備：

- 全域路徑規劃：在地圖上算出最佳路線（第 3、29 章）。
- 任意目標導航：能前往地圖上任何座標，而非只能沿預設路線。
- 斷點恢復：跟丟後能重定位並繼續任務（第 22、23 章）。

- 語意任務：結合語意地圖，能理解「去教務處」這種指令（24.5 節）。

比較項目	視覺 SLAM	視覺導航
核心問題	我在哪？環境長什麼樣？	我該往哪走？
輸出	位姿與地圖	運動指令（速度、方向）
是否需地圖	產出地圖	不一定需要
架構類型	審議式（需建模）	可反應式，也可審議式
對應導航三問	第一問	第二、三問
本書章節	第 19–22 章	本章、第 29 章

表 24-1　視覺 SLAM 與視覺導航的對照

實務系統多半兩者並用：以 SLAM 提供全域定位與規劃，以反應式視覺導航處理即時避障與精確對接。這正是第 3 章「混合式架構」的體現。

24.2　線追蹤、地標導航、視覺伺服

本節介紹三種不需要完整 SLAM 的基本視覺導航策略，由簡到繁。

一、線追蹤（Line Following）

這是最樸素、也是無數同學第一個做出來的機器人：在地面貼一條黑色膠帶，機器人以向下的相機或紅外線感測器偵測線的位置，若線偏左就左轉、偏右就右轉。

89. 原理：把影像轉為灰階或二值化（影像處理的基本功），找出線的中心與畫面中心的偏差，轉為轉向指令。
90. 優點：成本極低、程式極簡、極為穩健、不需地圖與定位。
91. 限制：只能走預設路線、無法繞開障礙、環境必須改造（貼線）。

92. 實務應用：工廠 AGV 的磁條或色帶導引至今仍廣泛使用——在可控環境中，它便宜可靠得無可取代（呼應第 4 章亞馬遜倉儲案例的觀念：環境越可控，技術選型越能簡化）。

二、地標導航（Landmark Navigation）

比線追蹤進階一階：不鋪連續的線，而是在關鍵位置放置可辨識的地標，機器人「看到地標 A 就右轉、看到地標 B 就停」。

地標可以是人工的（第 4.6 節的 AprilTag、QR Code、色塊）或自然的（門、滅火器、消防栓）。前者可靠但需改造環境，後者不需改造但辨識較難——這正是 YOLO 等 AI 辨識技術的用武之地（24.4 節）。

地標導航本質上建立了一張拓樸地圖（第 3 章）：節點是地標，邊是「從這個地標到下一個地標的行進動作」。它不需要精確的座標，卻能完成相當複雜的路線任務。

三、視覺伺服（Visual Servoing）

視覺伺服是三者中最精密的，它把相機直接放進控制迴路：持續比較「當前影像」與「目標影像」的差異，並產生運動指令以縮小差異，直到兩者一致。

它有兩大流派：

93. 影像式視覺伺服（IBVS）：直接以影像上的特徵誤差（例如目標點應該在畫面中央，現在偏左 30 像素）作為控制訊號。優點是不需重建三維資訊、對相機校正誤差不敏感；缺點是機器人的實際運動軌跡可能不直觀。

94. 位置式視覺伺服（PBVS）：先由影像解出「相機相對目標的三維位姿誤差」（用第 21 章的 PnP），再據此規劃運動。優點是運動軌跡在三維空間中合理；缺點是需要準確的相機校正與目標模型。

視覺伺服的典型應用是精確對接：機器人手臂抓取物件、掃地機器人回充電座對位、無人機精準降落。它能達到公釐級的相對定位精度——遠高於 SLAM 的全域定位精度，因為

它只關心「相對於眼前目標」的位置。

方法	需要環境改造	精度	彈性	典型應用
線追蹤	需要（貼線）	中	低（固定路線）	工廠 AGV、教學機器人
地標導航	視地標而定	中	中（拓樸路線）	走廊巡邏、展場導覽
視覺伺服	不需要	極高（公釐級）	低（僅對目標）	精確對接、抓取、降落

表 24-2　三種基本視覺導航策略比較

量一量：兩種視覺伺服差在哪

24.2 節說影像式視覺伺服「機器人的實際運動軌跡可能不直觀」，位置式則「運動軌跡在三維空間中合理」。這兩句話聽起來像是風格差異，其實是會決定成敗的實質差別。本小節把它算出來。

實驗設定：相機要從一個偏離的起始位姿，移動到正對目標、距離 0.65 公尺的位置。目標是一個邊長 20 公分的方形上的四個角點。兩種控制律都用標準形式——影像式以影像雅可比矩陣的偽逆乘上特徵誤差，位置式則先解出三維位姿誤差再直接命令速度。關鍵變因是起始時相機繞光軸轉了多少度。

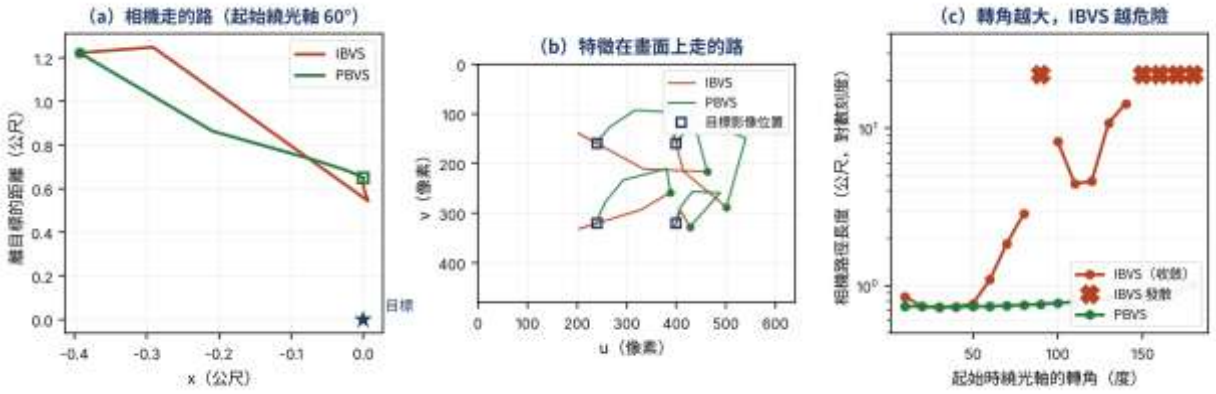


圖 24-1　影像式與位置式視覺伺服的收斂行為

（a）是相機在空間中走的路，起始時繞光軸轉了 60 度。位置式（綠）平順地由遠而近收斂

到目標距離；影像式（紅）則先衝過頭、距離一度縮到 0.54 公尺（比目標的 0.65 還近），再退回來。路徑長度分別是 0.74 與 1.09 公尺。這個「先衝過頭再退回」正是影像式視覺伺服著名的相機後退現象。

（b）是同一次過程在影像平面上的樣子，而這裡的勝負完全相反。影像式的四條特徵軌跡幾乎是直線——這正是它的設計目標：它最小化的是影像上的誤差，所以特徵會沿著畫面上的最短路徑移動。位置式的特徵軌跡則明顯繞彎，因為它只關心三維位姿，不管特徵在畫面上怎麼走。

這組對照說明了兩者的本質：影像式保證畫面上的行為漂亮（特徵不容易跑出視野），但相機的實際運動可能很怪；位置式保證相機的運動漂亮，但特徵可能在過程中跑出畫面外。選哪一個，取決於你的失敗模式是什麼——手臂在狹窄空間作業怕撞到，就該選位置式；目標只佔畫面一小塊、怕追丟，就該選影像式。

（c）把起始轉角從 10 度掃到 180 度。位置式的路徑長度全程維持在 0.74 至 1.03 公尺之間，幾乎就是起點到終點的直線距離，而且每一次都收斂。影像式在 50 度以內同樣漂亮（約 0.73 公尺），超過之後路徑迅速膨脹——60 度是 1.09 公尺、80 度是 2.86 公尺、140 度是 14.4 公尺，而在 90、150、160、170、180 度這幾個設定上直接發散。

180 度那個著名的反例

影像式視覺伺服最廣為人知的失效案例，正是「目標相對相機剛好轉了 180 度」。此時影像雅可比矩陣的偽逆會命令相機沿光軸直直後退——因為對影像而言，退得越遠，四個角點就越靠近畫面中心，誤差「看起來」在減少。結果是相機一路退到無窮遠，誤差卻始終降不下來。

本實驗在 150 度以上重現了這個現象：路徑長度動輒數百萬公尺，那不是機器人真的走了那麼遠，而是控制律已經失控。

實務上的對策有三種：切換式控制（遠距離用位置式、近距離用影像式）、混合式視

覺伺服（平移用影像式、旋轉用位置式）、或最簡單的——先用其他方式把姿態大致對正，再交給視覺伺服收尾。

24.3 AprilTag / ArUco 導航

第 4.6 節介紹過視覺基準標記的原理——由四個角點與相機內參解出完整 6-DoF 位姿（第 21 章的 PnP 問題）。本節聚焦它們在導航任務中的實際用法。

四種典型用途

- 絕對位姿校正：在走廊、電梯口、實驗室門口貼標記。機器人只要看到任一標記，就能取得不漂移的絕對位姿，一舉消除 SLAM 的累積誤差。這是最常見也最實用的用法（第 4.7 節的融合思想）。
- 精確對接：充電站、取放料點、電梯按鈕前貼標記，配合視覺伺服（24.2 節）完成公釐級的最後對位。SLAM 的定位精度通常在數公分，不足以應付需要精密對接的動作。
- 樓層與區域識別：第 5.7 節提過，光達 SLAM 無法分辨長得一樣的不同樓層。在每層樓的電梯口貼上不同 ID 的標記，問題迎刃而解。
- SLAM 精度評估：在場域中密集布置已知座標的標記，可作為近似的真值軌跡，用來評測 SLAM 系統的精度（第 22.7 節的 ATE 評測需要真值）。這對沒有動作捕捉系統的教學實驗室極為實用。

實作要點

95. 標記尺寸與偵測距離：偵測距離大致與標記邊長成正比。走廊用的標記若太小，機器人要走到很近才看得到，失去校正的意義。實務上會依需求製作不同尺寸。
96. 安裝高度與角度：標記應貼在機器人相機視野的合理高度，並盡量正對走道。斜視角度過大時，四個角點的偵測精度下降，解出的位姿誤差變大。

97. 光線與反光：標記應避免直接反光（護貝的標記在燈光下可能完全無法辨識）。建議用消光材質列印。

98. 唯一性管理：每個標記的 ID 與其世界座標必須建檔管理。這份「標記地圖」本身就是一種簡易的語意地圖（第 3 章）。

與 SLAM 的融合層次

第 4.7 節提過三種融合層次，在此具體化：

99. 鬆耦合：SLAM 正常運作，只在看到標記時把位姿「拉」到標記給出的值。實作最簡單，適合教學。

100. 緊耦合：把標記觀測當作約束加入 SLAM 的後端最佳化（第 12 章的一條邊）。精度最佳，是研究級做法。

101. 輔助初始化與重定位：最務實——SLAM 平時自主運作，只在啟動時或跟丟時用標記提供絕對參考。以極低成本解決最頭痛的問題。

量一量：標記貼多大、貼多斜、貼多密

24.3 節的「實作要點」給了三條原則：標記太小看不遠、斜視角度過大精度會下降、以及唯一性要管理。前兩條都是可以量化的，而量化之後會得到比原則更有用的東西——可以直接寫進規劃文件的數字。

實驗方法：取一個邊長 16 公分的標記，以 520 像素焦距、640×480 解析度的相機觀測，四個角點加入定位雜訊後解 PnP，與真實位姿比較。角點雜訊的模型是：標記在畫面上投影得越小（距離遠或斜視造成前縮），雜訊越大——這比「固定雜訊」更貼近真實偵測器的行為。

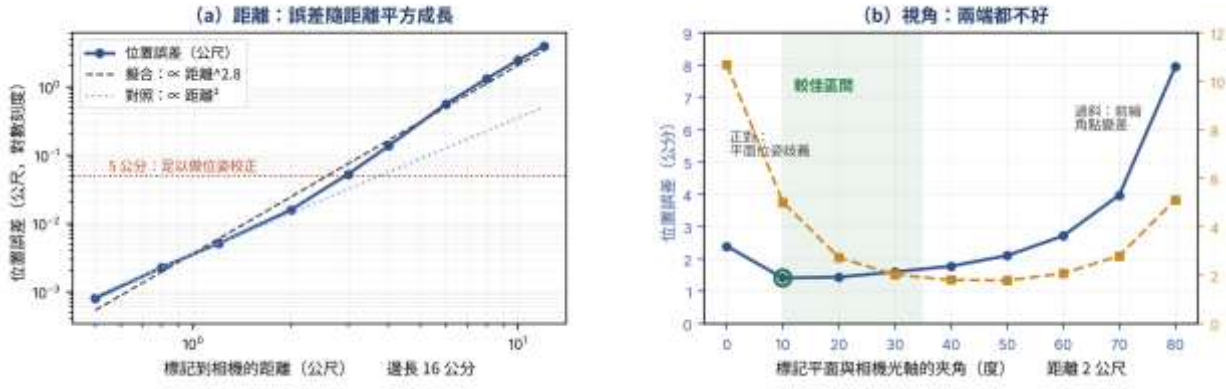


圖 24-2　標記的位姿精度如何隨距離與視角變化

（a）是距離的影響，兩軸都是對數刻度。0.5 公尺時位置誤差只有 0.8 公釐，1.2 公尺時 5 公釐，3 公尺時 5.2 公分，6 公尺時 0.56 公尺，12 公尺時 3.9 公尺。擬合出的指數是 2.76——比單純的距離平方更陡。多出來的那 0.76 次方來自角點偵測本身：標記在畫面上越小，角點位置就越難定準，雜訊與幾何放大效應相乘。

這條曲線給出一個很實用的設計規則。若把「位置誤差 5 公分」當作足以做位姿校正的門檻（SLAM 本身的定位誤差通常也在數公分），那麼 16 公分的標記有效距離大約是 3 公尺。走廊若要在 6 公尺外就能用，標記邊長得加倍到 32 公分左右。這就是 24.3 節「偵測距離大致與標記邊長成正比」那句話的具體換算。

（b）是視角的影響，而結果比 24.3 節原本的說法更細緻。原則上「斜視角度過大時位姿誤差變大」是對的——80 度時位置誤差是 8 公分，比最佳值差了五倍以上。但完全正對也不好：0 度時角度誤差高達 10.7 度，是整條曲線的最差點。

正對之所以糟，是因為平面目標的位姿歧義：當標記正對相機時，「標記往左傾一點」與「標記往右傾一點」投影出來的四個角點幾乎一模一樣，旋轉方向在數學上接近不可觀測。稍微斜一點，這個對稱性就被打破了。兩端各有各的問題，最佳區間落在大約 10 至 35 度。

一條可以直接用的安裝規則

把圖 24-2 的兩個結論合起來，走廊標記的安裝建議是：邊長取「預期偵測距離除以

20」（16 公分對應 3 公尺、32 公分對應 6 公尺）；不要正貼在走道正前方的牆上，而是略微側貼或讓機器人斜向經過，使視角落在 10 至 35 度之間。

這兩件事都不花錢，卻能讓同一批標記的精度差上好幾倍。同學在規劃場域時值得先用自己的相機跑一次圖 24-2 的實驗——不同的焦距與解析度會得到不同的數字。

量一量：標記該多久貼一個

前面量的是單一標記的精度，接下來是更實際的系統問題：一條巡邏路線要貼幾個標記才夠？

實驗設定是一條 90 公尺的ㄇ字形走廊巡邏路線，共 360 個關鍵幀。里程計每一步都有雜訊，繞完整條路線會累積出明顯漂移。標記每隔固定距離貼一個，量測到的絕對位姿帶有雜訊（位置標準差 2 公分、角度 2 度，由圖 24-2 的結果推得）。比較三種做法：完全不用標記、鬆耦合（看到標記就把位姿拉過去）、以及緊耦合（把標記觀測當成最佳化的約束，與里程計一起解）。

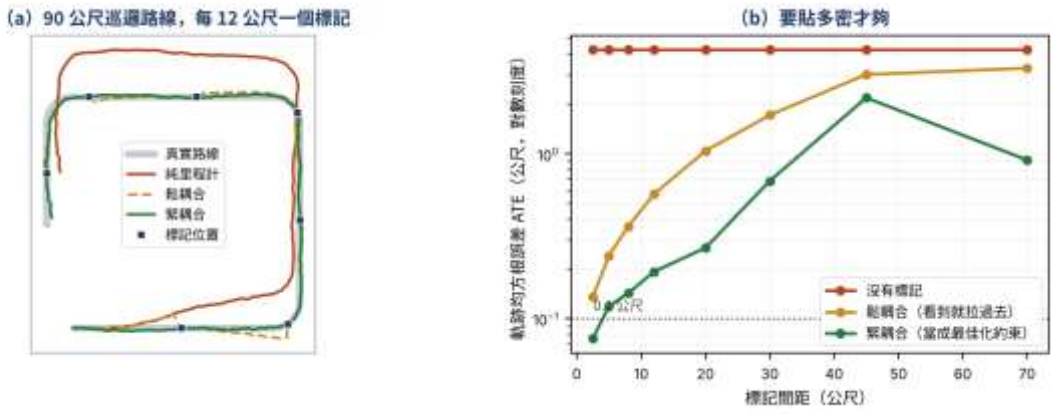


圖 24-3　標記間距對定位精度的影響

（a）畫出每 12 公尺貼一個標記時的實際軌跡。純里程計（紅）在走完一圈後已經明顯偏離路線；鬆耦合（橘虛線）在每個標記處被拉回來，但兩個標記之間仍會漂開，形成鋸齒狀的軌跡；緊耦合（綠）則全程貼合真實路線。

（b）掃描標記間距。沒有標記時 ATE 固定是 4.29 公尺。鬆耦合在間距 2.5 公尺時是 0.136 公尺，5 公尺時 0.241、12 公尺時 0.571、20 公尺時 1.04 公尺。緊耦合在相同間距下分別

是 0.076、0.119、0.193、0.269 公尺——每一點都比鬆耦合好，而且間距越大、差距越明顯：20 公尺間距時緊耦合的誤差只有鬆耦合的四分之一。

為什麼緊耦合在稀疏的情況下優勢更大？因為鬆耦合只能「在看到標記的那一瞬間」修正，修正量無法往回傳給前面那些已經走過的位姿；緊耦合則把標記當成最佳化裡的一條約束，誤差會沿著整條位姿鏈重新分配。這與第 22 章圖 22-2 的結論是同一件事——修正量要靠圖的連通性才能傳開。

必須誠實地指出一個細節：在 45 公尺與 70 公尺的間距下，整條路線只剩一個標記，結果就由「那個標記剛好貼在哪裡」主宰，而不再反映間距本身。圖上 70 公尺那一點的誤差反而比 45 公尺低，就是這個原因。所以這張圖只在「標記數量足夠形成鏈」的範圍內可以直接讀。

給場域規劃的三個數字

以本實驗的設定（90 公尺路線、數公分級的里程計雜訊）為例：

要把 ATE 壓在 10 公分以內，緊耦合需要約 5 公尺的間距、鬆耦合則需要 2.5 公尺以內——也就是說，採用緊耦合可以讓標記數量減半。

若只要求 30 公分以內（多數巡檢任務足夠），緊耦合可以放寬到約 20 公尺的間距，整條路線只需要四、五個標記。這個差別直接決定了布建成本與維護工作量。

24.4 YOLO 物件辨識與語意導航

前三節的導航，機器人都不「理解」它看到的東西——線就是線、標記就是編號。加入物件辨識之後，機器人才開始理解場景的意義。

從幾何到語意

第 1、3 章提過語意地圖的概念：在幾何地圖之上，標註「這是門、那是樓梯、那邊是滅火器」。YOLO 這類即時物件偵測模型，正是產生這些標註的工具。

有了語意，導航任務的表達方式就徹底改變了：

102. 幾何導航：「前往座標（12.5, 8.3）」——使用者必須知道座標。

103. 語意導航：「前往最近的飲水機」——使用者只需說出物體名稱。

後者才是人類自然的溝通方式，也是服務機器人真正可用的前提。

YOLO 在導航中的三種角色

- 語意標註：偵測到的物體連同其三維位置（由 RGB-D 或雙目提供深度）一起寫入地圖，形成語意地圖。
- 動態物體處理：辨識出「人」「車」等會移動的物體，在 SLAM 中把這些區域的特徵剔除，避免動態物體干擾位姿估計（第 9.5 節提過的視覺 SLAM 弱點）。這是一個極為實用的技巧。
- 任務目標偵測：巡檢機器人辨識儀表、裂縫、垃圾；資源回收機器人辨識瓶罐種類——這是任務本身的核心功能。

機器人視角的挑戰

這裡有一個容易被忽略、但在實務上極為關鍵的問題：大多數視覺模型是用「人類視角」的照片訓練的，而機器人的視角往往截然不同。

研究指出，四足機器人的自主導航需要穩健的語意感知，然而由於這類平台在低於 50 公分的高度運作，以人類中心視角訓練的視覺模型往往難以良好運作。

這個問題對掃地機器人、機器狗、小型輪型機器人都成立：從 30 公分高往上看的椅子、從地面看到的門，與訓練資料中的照片差異很大，辨識率因此下降。

104. 對策一：蒐集機器人視角的資料重新訓練或微調模型——這正是第 31 章專題四階段四（訓練 YOLO 辨識巡檢目標）與第 32 章語意標註的用意。

105. 對策二：使用具備開放詞彙能力的較新模型，它們的泛化能力較強。

106. 對策三：調整相機安裝高度與俯仰角，盡量接近訓練資料的視角。

這個案例再次印證本書反覆強調的觀念：技術的效能高度依賴部署情境，不能只看論文上的數字。

24.5 VLM 視覺語言模型輔助導航

YOLO 讓機器人認得物體，但它仍然只能回答「畫面中有什麼」。VLM（視覺語言模型）則更進一步——它能理解場景、推理常識、並接受自然語言指令。這是機器人導航正在發生的典範轉移。

VLM 帶來什麼

VLM 在大規模影像與文字語料上預訓練，展現出統合視覺與語言模態的強大能力，對需要豐富語意感知的導航任務日益重要。早期模型把影像與文字特徵對齊到共同的嵌入空間，而 CLIP 引入對比學習實現開放詞彙的視覺辨識，大幅提升多模態表徵的泛化能力；近期的 VLM 更進一步提供視覺語言推理與決策支援的統一介面。

對導航而言，VLM 的關鍵價值在於常識推理。舉例來說，若指令是「去找洗手台」，機器人不需事先標註過洗手台的位置——VLM 知道洗手台通常在廁所或廚房，因此會優先探索這類區域。這種能力稱為語意先驗。

三種應用模式

- 視覺語言導航（VLN）：接受自然語言指令，如「走到走廊盡頭，右轉後在第二扇門停下」。系統需理解指令、對應到當前視野、逐步執行。
- 物件目標導航：接受「去找一個垃圾桶」這類目標，系統需在未知環境中探索並找到目標。VLM 的常識推理讓探索更有效率。
- 社交感知導航：理解場景中的人與情境，決定合宜的行為（不從人群中硬穿、在教室外放慢速度）。

零樣本能力：不需任務特定訓練

VLM 導航最具突破性的特質，是零樣本能力——不需要為每個新環境或新任務重新訓練。

相關研究以 VLM 作為全域規劃器，把多個機器人不同視角的子地圖聚合為統一的全域地圖，編碼機器人狀態與邊界區域；VLM 據此在機器人之間分配邊界，促成協調而高效的探索。實驗顯示這類方法在成功率與導航效率上優於既有基準，且不需要任務特定的訓練；消融實驗進一步證實了 VLM 語意先驗的重要性。

這與傳統的學習式導航形成鮮明對比：後者往往需要在目標環境中蒐集大量資料訓練，換個場域就要重來。

前沿方向：視覺語言動作模型（VLA）

更新的發展是 VLA（Vision-Language-Action）模型——不只理解，而是直接由影像與語言產生動作指令。

近期研究提出能實現全模態目標條件化的訓練框架：以高容量的視覺語言動作骨幹，同時以 2D 位姿、第一人稱影像與自然語言三種目標模態（以及它們的組合）進行訓練。這呼應了人類的能力——我們能靈活地解讀並組合不同形式的目標指定，無論是語言指令、空間座標還是視覺參照。

另有研究展示同一套代理核心能部署到四種異質的真實機器人上——雙臂輪式平台、四足機器人、人形機器人與自建的四旋翼無人機——只需更換底層控制器。這種跨平台的通用性，是機器人基礎模型（Robotics Foundation Model）願景的具體展現。

實務限制：別忘了工程現實

VLM 導航令人興奮，但同學務必認識它的限制：

107. 運算負擔：大型 VLM 推論耗時，難以在機器人上即時運行。因此出現了各種效率優化研究，例如以注意力為基礎的權杖重要性評估與查詢引導的時空過濾，在不重新

訓練或修改預訓練模型的前提下保留導航相關資訊，並在真實的四足機器人上驗證了可靠且低延遲的指令跟隨導航。

108. 幻覺風險：VLM 可能「看到」不存在的東西或做出錯誤推理。若機器人完全信任它，可能撞牆或走錯路。實務系統必須保留幾何層的安全檢查（第 29 章的避障）。

109. 延遲與安全：VLM 適合做高階決策（去哪裡），不適合做即時控制（閃避突然出現的障礙）。分層架構是必然選擇——這正是第 3 章混合式架構的現代版本。

這些限制正是第 32 章要深入討論的主題。同學此刻只需建立一個核心認知：VLM 讓機器人「聽得懂、想得通」，但「走得穩、不撞人」仍然要靠前面二十三章的幾何與控制技術。兩者缺一不可。

24.6 機器狗視覺導航

四足機器人（機器狗）近年成為視覺導航研究與應用的熱門平台。本節說明它的特殊挑戰與對策。

為什麼選機器狗

110. 地形適應力：能上下樓梯、跨越門檻與碎石、走過不平地面——這是輪型機器人做不到的（第 3.6 節）。

111. 校園與工廠場域適用：有樓梯、斜坡、戶外通道的校園場域，機器狗的通行能力明顯優於輪型。

112. 平台成熟：商用四足機器人已相當普及，並提供開放的開發介面，適合作為研究與教學平台。

四項特殊挑戰

- 步態震動：四足行走時機身持續上下起伏與左右擺動，造成影像模糊與劇烈的位姿變化。這對視覺 SLAM 是嚴峻考驗——這正是第 23 章 IMU 緊耦合不可或缺的原因，也呼應第 18 章機器狗需要光達慣性緊耦合的論述。

- 低視角問題：如 24.4 節所述，機器狗在低於 50 公分的高度運作，以人類中心視角訓練的視覺模型往往難以良好運作。語意感知的準確度因此下降。
- 三維地形理解：輪型機器人只需知道「前方能不能走」，機器狗還需要知道「這個台階多高、踏得上去嗎、落腳點穩不穩」。導航從 2D 升級為 2.5D 甚至完整 3D（第 3.6 節）。
- 續航與運算限制：四足行走的能耗遠高於輪型，可搭載的運算平台也受限。這使得輕量化的演算法與邊緣運算格外重要（第七篇的主題）。

對策與系統設計

113. 感測器組合：3D 光達或深度相機（地形）+ IMU（姿態與震動補償）+ RGB 相機（語意）。多感測器融合在此不是選項而是必需。
114. 緊耦合估測：以第 18 章的 LIO 或第 23 章的視覺慣性 SLAM 為骨幹，IMU 是穩定度的關鍵。
115. 分層控制：上層做語意決策與路徑規劃（可用 VLM），中層做地形評估與落腳點規劃，底層做步態控制。各層時間尺度不同，這是第 3 章混合式架構的典型實現。
116. 語意模型的在地化：蒐集機器狗視角的資料微調模型，是提升辨識率最直接的方法。

研究前沿

機器狗已成為驗證前沿導航技術的標準平台。近年的視覺語言導航研究，普遍會在真實的四足機器人上驗證方法的實用性——因為它同時具備真實世界的複雜度與可控的實驗條件。已有研究把專為腿式機器人設計的視覺語言動作導航模型作為代表性工作提出，顯示這個方向的活躍程度。

24.7 校園巡檢與資源回收辨識機器人

本節把本章技術整合成兩個具體的專題規劃，作為第六篇的收尾，也為第 31 章的專題

預作準備。

專題一：校園巡檢機器人

這是第 5.7 節、第 18.7 節都提過的主軸，本節從視覺導航的角度完整規劃。

圖 24-4 校園巡檢機器人的系統架構

【感知層】RGB-D 或雙目相機 + IMU +（可選）光達、GNSS

↓

【定位建圖】ORB-SLAM3 視覺慣性 SLAM（第 23 章）

+ AprilTag 絕對位姿校正（24.3 節）

↓

【語意理解】YOLO 物件偵測 → 語意地圖（24.4 節）

↓

【任務決策】巡檢點清單 / VLM 自然語言任務（24.5 節）

↓

【路徑規劃】全域規劃（地圖）+ 局部避障（即時感知）

↓

【執行與紀錄】移動至巡檢點 → 拍照 / 量測 → 標上三維座標存檔

巡檢任務的視覺重點

117. 儀表判讀：偵測並讀取壓力表、電表、溫度計的數值。這需要精確對位（視覺伺服，24.2 節）加上數字辨識。

118. 異常偵測：裂縫、漏水痕跡、鏽蝕、雜物堆放——這些多半只有視覺看得出來（第 8 章提過光達給不了語意）。

119. 變化偵測：比較不同時間的同一位置影像，找出「昨天沒有、今天出現」的異常。這需要精確的重複定位能力（第 23 章 Atlas 的多階段作業正好適用）。

120. 門禁與安全：偵測門是否關好、消防通道是否堵塞、滅火器是否在位。

專題二：資源回收辨識機器人

這是一個兼具技術挑戰與教育意義的題目，特別適合高中與大學部合作。

121. 任務定義：機器人在校園中巡行，辨識並記錄（或收集）可回收物，分類為寶特瓶、鋁罐、紙類、一般垃圾。

122. 視覺挑戰：物體姿態多變（躺著、立著、壓扁的）、材質反光（鋁罐）、透明（寶特瓶，這對深度相機也是難題，呼應第 9 章的限制）、光線多變（戶外）。

123. 資料集建置：這是專題的核心工作——在校園實地拍攝各種角度、光線、背景下的回收物，建立標註資料集，訓練 YOLO 模型。這正是第 31 章專題四階段四的方法。

124. 系統整合：SLAM 提供位置（知道在哪裡發現的）、YOLO 提供辨識、導航讓機器人自主巡行、資料庫記錄成果。

教學層次的規劃

階段	內容	適合對象
一	線追蹤與 AprilTag 導航，理解基本視覺導航	高中
二	訓練 YOLO 辨識校園物件，建立資料集	高中進階 / 大學部
三	SLAM 建圖 + 語意標註，產生語意地圖	大學部
四	自主巡檢：規劃路線、執行任務、產出報告	大學部專題
五	VLM 自然語言任務、跨階段變化偵測	專題 / 研究生

表 24-3　視覺導航專題的五個教學階段

這個層次規劃的優點是：每個階段都有可展示的成果，學生不必等到最後才看到東西動起來；而且各階段技術逐步疊加，前面的成果不會白費。

至此，第六篇「Visual SLAM 與視覺導航機器人」全部完成。我們從第 20 章的特徵基礎出發，經過第 21 章的幾何、第 22 章的 ORB-SLAM2、第 23 章的 ORB-SLAM3，到本章把技術落實為真實任務。下一篇，我們把視野轉向極端受限的平台——微型機器人與無人機。當運算資源只有毫瓦等級、當載重只有數十公克時，SLAM 該如何實現？第七篇「微型機器人、無人機與邊緣 AI SLAM」將帶來截然不同的思考。

24.8　語言導航的現況：2025 年的一次誠實評估

24.5 節描述了視覺語言模型輔助導航的願景。本節補上一件同樣重要的事：以目前的技術，它做得到什麼程度。

一、一個具身導航的基準

2025 年底發表的一個基準提供了相當清楚的答案。它的設計是：給模型一張真實場景的照片、一段文字指令，以及一個載體類型（人、四足機器人、輪型機器人、腳踏車），要模型在影像上畫出應該走的軌跡。資料集有 1,000 個情境與超過 3,000 條專家軌跡，評分同時考慮軌跡形狀、終點準確度，以及「這個載體不該走的地方」的懲罰。

方法	分數
人類專家	75.40
只預測路徑（直接告訴它目標在哪）	56.55
已知目標的直線基準	51.89
完整軌跡預測（最佳模型）	34.38

只預測目標點	29.65

表 24-4　具身導航基準的評估結果（分數越高越好）

二、三個應該被記住的數字

第一，最佳模型的 34.38 分與人類專家的 75.40 分差了 41.02 分。這不是「再過半年就追上」的差距。

第二，也是最值得玩味的：一條「已知目標的直線」拿到 51.89 分，遠高於最佳模型完整軌跡的 34.38 分。一條不避障、不繞路、完全不看環境的直線，勝過當代最強模型的完整規劃。這說明模型的失分主要不在路徑形狀，而在它沒搞清楚目標在哪。

第三點佐證了這個判斷：當目標被直接告知時，同一個模型只預測路徑能拿到 56.55 分；反過來只預測目標點則掉到 29.65 分。模型「會走」但「不會認」——把語言裡的目標對應到影像中正確的位置，才是目前真正的瓶頸。

三、這對本章的分層架構意味著什麼

這三個數字直接支持本章 24.1 到 24.3 節的立場：既然模型最弱的環節是目標定位，那麼能用標記確定的位置，就不要讓模型去猜。一個貼在教務處門口的標記，比任何視覺語言模型都更可靠地回答「教務處在哪裡」。

反過來說，模型相對擅長的事情應該交給它：理解口語指令、判讀畫面內容、生成回報、以及在已知目標的情況下描述路線。務實的問題不是「語言模型能不能做導航」，而是「導航的哪些部分適合交給它」。

這也呼應案例 24-2 的低視角問題：模型在機器人視角下的表現，與在人類視角下有明顯落差。第 32 章會回到資料蒐集與微調的做法，第 34 章則會把這整條語言到行動的鏈路完整展開。

公式或流程

一、線追蹤的比例控制

設影像寬度為 W、偵測到的線中心像素為 x_line、畫面中心為 x_center = W / 2，則橫向偏差為：

$$e = x_line - x_center$$

最基本的比例控制器直接把偏差轉為角速度指令：

$$\omega = -K_p \cdot e$$

其中 K_p 為比例增益。負號表示線偏右（e 為正）時應右轉。實務上常加入微分項以抑制震盪（PD 控制），或依偏差大小調整線速度——偏差大時放慢、走直線時加速。這是最簡單的視覺回授控制，也是同學理解視覺伺服的最佳起點。

二、影像式視覺伺服的控制律

IBVS 的目標是讓當前影像特徵 s 逼近目標特徵 s*。定義特徵誤差：

$$e = s - s^*$$

控制律為（以指數方式收斂誤差）：

$$v = -\lambda \cdot L^{+} \cdot e$$

其中 v 是相機的六自由度速度指令、λ 是收斂速率、L 是影像雅可比矩陣（描述「相機動一單位，影像特徵會移動多少」）、L^{+} 為其偽反矩陣。這條式子的精神與第 11、12 章的最佳化一脈相承：定義誤差、求出誤差對控制量的敏感度、沿誤差下降方向移動。

三、語意導航的目標轉換

語意導航的核心是把「語意目標」轉為「幾何目標」：

圖 24-5 從語言指令到運動指令的完整流程

使用者指令：「請去最近的飲水機」

↓

① 語言理解（VLM / LLM）：解析出目標物件類別 = 飲水機

↓

② 語意地圖查詢：地圖中是否已標註飲水機？

├─ 有 → 取得其三維座標，直接進入 ④

└─ 無 → 進入 ③ 探索

↓

③ 語意探索：以 VLM 常識推理選擇有希望的探索方向

（飲水機通常在走廊、茶水間附近）→ 邊界探索 + YOLO 偵測

↓

④ 幾何路徑規劃：在地圖上算出到目標的路徑（第 3、29 章）

↓

⑤ 局部避障與運動控制：即時感知，輸出速度指令

↓

⑥ 抵達後精確對接（視覺伺服，如需要）

這條流程清楚呈現了本書的整體架構：上層由 AI 理解語意，中層由 SLAM 與規劃提供幾何，底層由控制執行動作。三層缺一不可——這也是 24.5 節末尾強調的核心認知。

四、AprilTag 位姿校正的融合

當機器人偵測到已知世界座標的標記時，可由標記推算自身位姿：

$$T_{world\leftarrow robot} = T_{world\leftarrow tag} \cdot T_{tag\leftarrow camera} \cdot T_{camera\leftarrow robot}$$

其中 T_world←tag 由標記地圖查得（已知）、T_tag←camera 由 PnP 解出（第 21 章）、T_camera←robot 為外參（第 10 章的相機安裝位置）。三個變換相乘即得機器人在世界座標中的位姿——這正是第 10 章「變換組合就是矩陣連乘」的直接應用。

案例

案例 24-1　從貼一條線到聽懂人話：視覺導航的四十年

把本章的技術依時間排列，會看到一條清晰的演進脈絡，而且每一步都對應著「機器人理解世界的深度」在提升。

125. 1980 年代——線追蹤：機器人只認得亮度差異，環境必須為它改造（貼線）。它不知道自己在哪、也不知道要去哪，只是不斷修正偏差。

126. 2000 年代——地標與視覺伺服：機器人開始能辨識特定圖案，並精確地相對定位。環境改造需求降低（只需貼標記），但仍需改造。

127. 2010 年代——視覺 SLAM：機器人能自行建立地圖並定位，不再需要改造環境。但它理解的是「幾何」，不是「意義」。

128. 2020 年代——語意與 VLM 導航：機器人開始理解「那是門、那是飲水機」，並能接受自然語言指令。它終於能用人類的方式溝通。

觀察這條脈絡，同學會發現一個有趣的規律：每一代技術降低的，都是「人類為機器人做的準備工作」。從改造環境、到標註地標、到只需下達指令——機器人越來越能適應人類的世界，而非要求人類遷就它。

但這裡有一個重要提醒：新技術並未淘汰舊技術。今天的工廠 AGV 仍大量使用磁條導引，因為在可控環境中它最可靠、最便宜；精確對接仍需視覺伺服，因為 SLAM 的精度不夠。工程師的任務不是永遠用最新的技術，而是為每個問題選擇最合適的技術——這是本書從第 8 章開始就反覆強調的判斷力。

案例 24-2 低視角的困境：當模型不認識機器人眼中的世界

這是一個看似技術細節、實則影響深遠的問題，也是同學做專題時最容易踩到的坑。

狀況是這樣的：同學下載了一個在標準資料集上表現優異的物件偵測模型，興沖沖裝到機器狗或掃地機器人上，結果辨識率慘不忍睹——明明是椅子，模型卻認不出來；明明前方是門，模型標成了牆。

原因如研究所指出：四足機器人在低於 50 公分的高度運作，而以人類中心視角訓練的視覺模型往往難以良好運作。訓練資料中的椅子照片，多半是站著的人以約 150 公分高度拍攝的；從 30 公分高往上看，椅子的形狀、透視、遮蔽關係都完全不同。

這個問題還有幾個變形：掃地機器人看到的是家具的下半部與地板紋理；無人機看到的是物體的頂視圖；水下機器人面對的是色偏嚴重的影像。共同點是：部署情境與訓練情境的分布不一致。

解法沒有捷徑——蒐集目標視角的資料重新訓練或微調。這聽起來麻煩，但正是第 31 章專題四階段四的核心，也是專題最有價值的部分：一份「在地的機器人視角校園物件資料集」，本身就是可發表、可延續的研究成果。

這個案例給同學的更深啟示是：AI 模型不是萬用的黑盒子。理解它的訓練資料分布與部署情境的差異，是負責任地使用 AI 的基本功。

案例 24-3 多機器人協作探索：VLM 作為全域規劃器

本章 24.5 節提到的一項研究，值得單獨展開，因為它展示了 VLM 導航的完整潛力。

該研究提出的框架把視覺語言模型整合為全域規劃器，實現具備常識推理的多機器人視覺目標導航。它把多個機器人不同視角的子地圖聚合為統一的全域地圖，編碼機器人狀態與邊界區域；VLM 據此在機器人之間分配邊界，促成協調而高效的探索。

這個設計精彩之處在於：VLM 不是用來做低階控制，而是扮演「指揮官」的角色。它

看著全域地圖，運用常識判斷「這幾個未探索方向中，哪一個最可能找到目標」，再把任務分配給不同的機器人。低階的移動與避障，仍由傳統的規劃與控制負責。

實驗結果顯示，這個方法在成功率與導航效率上都優於既有基準，且不需要任務特定的訓練；消融實驗進一步證實了 VLM 語意先驗的重要性。研究團隊也在真實場景中以四足機器人驗證了這個框架。

對校園場域的啟發是：這正是「多台機器狗校園巡檢」專題（第 27、31 章）的技術路徑。多台機器人分頭探索、VLM 統籌分配、各自以 SLAM 定位建圖、最後合併成完整的校園語意地圖——這是一個層次豐富、可分工進行、且與國際前沿接軌的專題題目。

實作活動

實作 24-1　從線追蹤到語意導航的階梯式實作

目的：依照 24.7 節的五階段規劃，逐步建立視覺導航能力。本實作可依班級程度與資源選擇進行到哪一階段，每一階段都有可展示的成果。

階段一：線追蹤（高中適用，不需 SLAM）

55. 準備一台附相機的教學小車（或在模擬環境中），在地面貼出一條含直線與轉彎的路線。
56. 以 OpenCV 讀取影像、轉灰階、二值化，找出線的中心位置。
57. 依公式一實作比例控制器，把橫向偏差轉為轉向指令，讓小車自動循線。
58. 參數實驗：調整比例增益 K_p，觀察太小（反應遲鈍、出彎道）與太大（左右震盪）的現象，找出合適值。
59. 挑戰：在路線中加入十字交叉或分岔，設計規則讓小車做出選擇。

階段二：AprilTag 導航與位姿校正（高中進階 / 大學部）

60. 在教室四周貼上數個 AprilTag，量測並記錄每個標記的世界座標（建立標記地圖）。

61. 用第 9 章校正過的相機偵測標記，以 PnP 解出相機相對標記的位姿（第 21 章）。

62. 依公式四，把三個變換相乘，算出機器人在世界座標中的位姿，並即時顯示。

63. 與 SLAM 對照：同時執行視覺 SLAM，把兩條軌跡畫在一起——觀察 SLAM 軌跡平滑但漸漸偏離、標記軌跡跳動但始終正確（重現第 4 章案例 4-3）。

64. 簡易融合：實作鬆耦合校正——平時用 SLAM，每當看到標記就把位姿拉回標記給出的值，觀察漂移是否被抑制。

階段三：YOLO 語意標註（大學部）

65. 蒐集資料：用機器人視角的相機（注意安裝高度）在校園拍攝目標物件——門、飲水機、滅火器、垃圾桶、樓梯口，每類至少 200 張，涵蓋不同角度與光線。

66. 標註並訓練 YOLO 模型，在驗證集上評估準確率。

67. 視角對照實驗：另外拍一組「人類視角」（約 150 公分高）的測試影像，比較模型在兩種視角下的辨識率，量化重現案例 24-2 的現象。

68. 語意標註：把 YOLO 偵測結果與 RGB-D 深度結合，計算物件的三維位置，標註到 SLAM 建好的地圖上，產生語意地圖。

階段四：語意導航任務（大學部專題）

69. 建立語意地圖後，實作一個簡單的語意導航：輸入物件名稱（如「飲水機」），系統查詢地圖中該物件的座標，規劃路徑並導航前往。

70. 加入動態物體處理：用 YOLO 偵測「人」，在 SLAM 中剔除人身上的特徵點，觀察定位穩定度是否改善（24.4 節的第二種角色）。

71. 實作巡檢任務：設定數個巡檢點，讓機器人依序前往、拍照記錄、標上三維座標，最後產出巡檢報告。

階段五：VLM 輔助（專題 / 研究生）

72. 把機器人當前影像送入 VLM，以自然語言提問：「前方是否可通行？」「畫面中有樓梯嗎？」「哪裡可能是門？」記錄回答的正確率。

73. 比較 VLM 與 YOLO：在同一批影像上，比較兩者對「開放詞彙物件」（訓練時沒見過的類別）的辨識能力。

74. 延遲量測：記錄 VLM 推論的耗時，評估它適合放在系統的哪一層（高階決策？即時控制？），對照 24.5 節的限制討論。

討論問題

21. 階段一中，比例增益太大與太小各造成什麼現象？這與控制理論的哪些概念相關？

22. 階段二中，SLAM 軌跡與 AprilTag 軌跡的差異隨時間如何變化？加入校正後改善多少？

23. 階段三的視角對照實驗中，兩種視角的辨識率差多少？這對「直接套用網路上的預訓練模型」有什麼警示？

24. 階段四中，剔除人身上的特徵點後，定位穩定度改善了嗎？若場景中人很多（如下課時間走廊），系統還能運作嗎？

25. （專題）VLM 的推論延遲是多少？若機器人以每秒 0.5 公尺移動，這個延遲代表機器人在「思考」期間移動了多遠？這對安全設計有什麼意涵？

延伸挑戰（大學部）

其一：資源回收辨識機器人——依 24.7 節專題二的規劃，建立校園回收物資料集（特別注意透明寶特瓶與反光鋁罐的拍攝），訓練辨識模型，並整合 SLAM 讓機器人記錄「在哪裡發現了什麼」。這個題目同時涵蓋 AI、SLAM 與環境教育，展示效果佳。其二：多機器人協作探索的簡化版——用兩台機器人（或兩個模擬實例）分頭探索同一場域，各自建圖後合併（可用第 23 章 Atlas 的概念），並比較「協作探索」與「單機探索」完成覆蓋所需的時間，呼應案例 24-3 的研究方向。

延伸閱讀

29. F. Chaumette and S. Hutchinson, "Visual Servo Control, Part I: Basic Approaches" 與 "Part II: Advanced Approaches," IEEE Robotics & Automation Magazine, 2006–2007.（視覺伺服的權威教程，IBVS 與 PBVS 的完整說明）

30. E. Olson, "AprilTag: A Robust and Flexible Visual Fiducial System," ICRA, 2011.（第 4、24 章共同的基準標記理論來源）

31. A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision (CLIP)," ICML, 2021.（開放詞彙視覺辨識的里程碑，VLM 導航的基礎）

32. B. Yu et al., "Co-NavGPT: Multi-Robot Cooperative Visual Semantic Navigation Using Vision Language Models," arXiv, 2023（2025 修訂）.（案例 24-3 的來源，多機器人 VLM 協作探索）

33. N. Yokoyama et al., "VLFM: Vision-Language Frontier Maps for Zero-Shot Semantic Navigation," ICRA, 2024.（零樣本語意導航的代表作）

34. Ultralytics YOLO 官方文件（docs.ultralytics.com）.（物件偵測模型的訓練與部署，第 31 章專題四與第 32 章語意標註的主要工具）

35. OpenCV 官方文件之 ArUco 模組與影像處理教學（docs.opencv.org）.（線追蹤與標記偵測的實作基礎）

小結

29. 視覺 SLAM 回答「我在哪、環境長什麼樣」，視覺導航回答「我該往哪走」；視覺導航不一定需要 SLAM（線追蹤、視覺伺服屬反應式架構），但加上 SLAM 後才具備全域規劃、任意目標導航、斷點恢復與語意任務的能力。

30. 三種基本策略：線追蹤成本極低但只能走預設路線；地標導航建立拓樸地圖、彈性較高；視覺伺服以影像回授直接控制，能達公釐級的相對定位精度，分 IBVS 與 PBVS 兩流派。

31. AprilTag / ArUco 在導航中有四種用途：絕對位姿校正、精確對接、樓層與區域識別、以及作為 SLAM 精度評估的近似真值；與 SLAM 的融合可分鬆耦合、緊耦合與輔助初始化三個層次。

32. YOLO 在導航中扮演語意標註、動態物體剔除與任務目標偵測三種角色；但因四足與小型機器人在低於 50 公分的高度運作，以人類中心視角訓練的模型往往難以良好運作，必須蒐集機器人視角資料微調。

33. VLM 在大規模影像文字語料上預訓練，提供語意先驗與常識推理，使機器人能接受自然語言指令並零樣本執行新任務；但受限於運算負擔、幻覺風險與延遲，適合做高階決策而非即時控制。

34. 機器狗視覺導航面臨四項特殊挑戰：步態震動、低視角語意問題、三維地形理解與續航運算限制；對策是多感測器緊耦合估測與分層控制架構。

35. 校園巡檢與資源回收辨識是整合本篇技術的理想專題，可依線追蹤、AprilTag、YOLO 語意、自主巡檢、VLM 任務五階段規劃，兼顧高中到研究生的不同層次。

承先啟後

本章把前四章的技術帶進真實任務，也量出了三組可以直接用的數字：視覺伺服方面，位置式的相機路徑全程維持在 0.74 至 1.03 公尺、每次都收斂，影像式則在起始轉角超過 50 度後路徑迅速膨脹、並在數個角度上直接發散；標記方面，16 公分的標記在 3 公尺處的位置誤差約 5 公分，誤差隨距離的 2.76 次方成長，而最佳視角落在 10 至 35 度（正對會遇到平面位姿歧義）；布建方面，緊耦合在 20 公尺間距下的 ATE 是 0.269 公尺，只有鬆耦合的四分之一，等於讓標記數量減半。

第六篇「Visual SLAM 與視覺導航機器人」至此全部完成。我們從特徵與幾何出發，建立了完整的視覺 SLAM 系統，最後把它落實為真實的機器人任務。但前面所有的討論，都預設機器人有足夠的運算資源與電力。下一篇，我們走向另一個極端：當機器人只有 44 公克、運算功耗只有毫瓦等級時，SLAM 還可能嗎？第七篇「微型機器人、無人機與邊緣 AI SLAM」將展示這個領域最令人驚嘆的工程成就。